

JavaSE

Java 中的集合有哪些？

Java 的集合框架主要分为两大类：Collection 和 Map。

Collection 接口下有三大常用子接口，List、set 和 queue，list 特点是有序、可重复。

最常用的是 ArrayList，基于动态数组实现，所以查询快、增删慢，线程不安全。

另一个是 LinkedList，基于双向链表实现，因此增删快、查询慢，同时它还可以作为栈或队列使用。

Set 核心是唯一、无序。

最常用的是 HashSet，基于 HashMap 实现，利用哈希表来去重，查询效率非常高。

如果需要保证插入顺序，可以用 LinkedHashSet。

如果需要自动排序，则用 TreeSet，基于红黑树实现。

Queue 和 Deque 表示队列。

像 ArrayDeque 是基于数组的高效双端队列，常用来替代旧的 Stack 类。而 PriorityQueue 则是优先级队列，元素按优先级顺序出队。

Map 接口存储键值对，键是唯一的。

最核心的是 HashMap，基于哈希表，允许 Null 键和值，效率很高，但无序。

HashMap 的一个重要子类是 LinkedHashMap，在 HashMap 基础上维护了一个链表，从而保证了迭代顺序。

如果需要对 Key 进行自然排序，则使用基于红黑树的 TreeMap。

我们知道 HashMap 存储的是 键值对 (Key-Value)，而 HashSet 只存储 元素 (Element)。HashSet 的做法是把你的元素作为 HashMap 的 Key。把一个固定的虚拟对象 (PRESENT) 作为 HashMap 的 Value。

ArrayList 内部是怎么实现的？

内部实现核心是一个动态扩容的数组。当不断调用 add 方法添加元素时，一旦当前元素数量 size 快要达到数组容量，就会自动触发扩容，创建一个新的、更大的数组。新数组的容量计算规则通常是旧容量的 1.5 倍。

然后使用 Arrays.copyOf() 方法，将旧数组里的所有元素全部拷贝到新数组里。

最后，让内部的 elementData 引用指向这个新数组，旧的数组就会被 GC 回收。

因为有这么个扩容和数据拷贝的过程，所以如果能预知数据量的大小，最好在创建 ArrayList 时就直接指定一个初始容量，这样可以避免在添加过程中反复扩容，提升性能。

ArrayList 查询什么速度比较快？

通过索引查询元素时速度非常快，时间复杂度为 $O(1)$ 。

因为 ArrayList 底层通过连续的内存空间存储元素，就像数组一样，可以直接通过索引计算出元素在内存中的位置，实现随机访问。

而通过元素值查询时速度较慢，时间复杂度为 $O(n)$ 。

这种情况下需要从列表头部开始逐个遍历元素进行比较，直到找到目标元素或遍历完所有元素，效率与列表长度成正比。

ArrayList 初始化时数据的长度是多少？

取决于初始化方式，如果使用无参构造器初始长度为 0，使用指定长度的构造器长度为指定值，通过集合初始化则初始长度为集合的长度。

往初始化后的 ArrayList 插入 16 个元素后，再插入第十七个元素会怎样？

往初始化后的 ArrayList 插入第 17 个元素时，会触发自动扩容机制，但具体过程要结合 ArrayList 的初始创建方式来看。

ArrayList 底层是动态数组，有容量和实际元素数量两个概念，容量是底层数组的长度，决定了当前能容纳的最大元素，实际元素数量是当前存储的元素个数，通过 size() 方法就能获取。当实际元素数量要超过容量时，就会触发扩容，这是后续操作的基础。

如果是用无参构造器初始化的，在 JDK 1.8 及以后版本里，初始容量其实是 0，第一次添加元素时才会先扩容到 10。当我们往里面插第 11 个元素时，因为 10 的容量已经满了，就会触发第一次扩容，按 1.5 倍的扩容规则，容量会从 10 变成 15。接着插第 16 个元素时，15 的容量又满了，就会触发第二次扩容，新容量是 15 乘以 1.5，也就是 22。之后 ArrayList 会创建一个长度为 22 的新数组，把原来数组里的 15 个元素复制到新数组中，最后再把第 16 个元素插进去，此时实际元素数量变成 16，容量则是 22。

如果是用指定初始容量的构造器初始化的，那初始容量就是 16，刚好能容纳前 16 个元素。当要插第 17 个元素时，实际元素数量要超过容量，就会直接触发扩容：按 1.5 倍规则，新容量是 16 乘以 1.5 等于 24。后续同样是创建新数组、复制原有 16 个元素，再插入第 17 个元素，最终实际元素数量 17，容量 24。

BIO、NIO 和 AIO 三种 IO 的区别是什么？

BIO、NIO 和 AIO 是 Java 中的三种 I/O 模型，核心区别主要体现在处理方式和适用场景上。

BIO 是阻塞式 I/O，属于同步阻塞模型。特点是每个 I/O 操作都会阻塞线程，直到完成。一个连接需要一个线程处理，哪怕连接没数据，线程也得等着，这样在高并发时会创建大量线程，资源利用率低，但好处是编程简单，适合连接数少的场景。

NIO 是非阻塞式 I/O，属于同步非阻塞模型。引入了通道、缓冲区和选择器，一个线程可以通过选择器管理多个通道。操作时不会阻塞线程，线程可以轮询查看哪些通道有数据需要处理，这样单线程就能处理多个连接，资源利用率更高，适合高并发场景，但编程复杂度会高一些。

AIO 是异步非阻塞模型，它是发起操作后线程就可以去做别的事，等操作完成后，系统会主动通知应用程序。这种方式完全不需要线程等待，资源利用率最高，但实现更复杂，在 Java 里是 JDK7 之后才引入的，实际应用相对较少。

当操作系统把数据准备好，并且搬运到用户缓冲区后，会主动通知应用程序

从线程效率看，BIO 最低，NIO 中等，AIO 最高。

NIO 底层用了什么机制？

NIO 底层主要依赖操作系统提供的 I/O 多路复用机制 来实现高效的非阻塞 I/O 操作。

不同操作系统对多路复用机制有不同的实现，在 Linux 系统中，NIO 底层使用的是 epoll 模型，在 macOS 或 FreeBSD 上用的是 kqueue。而 Windows 系统则使用 IOCP。这些机制的核心思想是通过一个多路复用器来管理多个 I/O 通道。应用程序只需通过一个线程监听多路复用器，由操作系统内核负责检测哪些通道有可读 / 可写等事件发生，复用器会将就绪的通道通知给应用程序，程序再针对性地处理这些就绪通道的数据。

这种设计避免了传统 BIO 中 “一个连接一个线程” 的资源浪费，实现了少量线程高效处理多连接，大幅提升了高并发场景下的 I/O 处理能力。这也是 NIO 能支持高吞吐量网络通信的关键底层原理。

AOP 只有动态代理一种实现机制吗？还有其他的吗？

AOP 并非只有动态代理一种实现机制，动态代理是运行时实现 AOP 的主流方式，但除此之外还有其他实现方式，主要可以从编译期和类加载期两个阶段来区分。

编译期织入，在代码编译成字节码的过程中，直接将切面逻辑嵌入到目标类的字节码中。典型的如 AspectJ 框架的编译时织入，通过自定义编译器在编译阶段完成切面与目标类的融合，生成的字节码已经包含了增强逻辑，运行时无需额外处理，性能较好。

类加载期织入，在目标类被类加载器加载到 JVM 时，通过修改字节码的方式植入切面逻辑。例如 AspectJ 的加载时织入，在类加载阶段对字节码进行增强，属于静态织入的一种。

string、stringbuffer、stringbuilder 区别？

String、StringBuffer 和 StringBuilder 都是 Java 中处理字符串的类，它们的主要区别体现在可变性和线程安全性上。

String 是不可变的字符串类。每次对 String 进行拼接、修改等操作时，都会创建新的 String 对象，原对象不会被改变。这导致频繁修改字符串时会产生大量临时对象，占用内存且效率较低，但它是线程安全的。

StringBuffer 是可变的字符串类，提供了 append、insert 等方法直接修改内部字符数组，不会创建新对象。同时方法加了 synchronized 关键字，是线程安全的，但因此带来了额外的性能开销，适合多线程场景。

StringBuilder 同样是可变的字符串类，功能与 StringBuffer 基本一致，但方法没有加同步锁，线程不安全。不过正因为去掉了同步开销，执行效率比 StringBuffer 更高，适合单线程下频繁操作字符串的场景。

说说 tcp、udp 的落地场景？

Tcp 的落地场景有 HTTP/HTTPS 协议，UDP 协议的落地场景有视频通话和语音通话，以及直播等对实时性要求高、能容忍少量数据丢失的场景。

string 为什么是 final 类型？

String 被设计为 final 类型，可以保证字符串不可变，final 修饰类意味着 String 不能被继承，结合其内部字符数组被 private 和 final 修饰，从根本上保证了字符串的不可变性。

内部字符数组被 final 修饰。在 Java 中，final 修饰引用类型意味着该引用不能指向新的数组对象。真正保证内容不变的是 String 类没有提供任何修改内部数组的方法（比如没有 setCharAt），且所有看似修改的操作（如 replace）其实都是返回了一个全新的 String 对象。

这种不可变性让 String 可以安全地作为哈希表的键，也避免了多线程环境下的并发问题。由于内容不变，String 的 hashCode 只需在第一次计算时生成，之后直接读取缓存。这让 String 在作为 HashMap 的 Key 时，效率高得惊人。

以及提高性能和安全性，不可变的 String 可以被 JVM 缓存，多个引用可以指向同一个对象，减少内存消耗。同时，在涉及网络通信 Socket、文件操作 FilePATH 等场景中，不可

变的字符串能避免内容被意外篡改，保证数据安全。

以及简化设计和优化，final 类的方法默认是 final 的，JVM 可以对其进行更多优化，放心地把你的代码直接嵌入到调用处（内联）。此外，不可变特性让 String 的各种操作的实现更简单可控。

知道你不会变，我可以直接引用你的内存地址，不需要到处做昂贵的“深拷贝(Deep Copy)”

Java 常用的一些关键字 final 有什么作用，还有应用场景？

final 修饰的类无法被继承，final 修饰的方法不能被子类重写。

final 修饰的局部变量和成员变量一旦赋值就不能再修改，相当于常量。修饰的引用变量本身的引用地址不可变，但对象内部的属性可以修改。在设计不可变类、固定核心方法、定义常量等场景中经常用。

你考虑过 String 类用 final 修饰数组的原因吗？

保证字符串不可变。final 修饰数组意味着引用地址不可变，结合 private 访问控制，外部无法直接修改数组内容。这种设计让字符串一旦创建就无法被修改，比如 substring、concat 等方法都会返回新的 String 对象，而非修改原对象。这种不可变性是 String 类安全、高效的基础。

以及支持哈希值缓存。由于字符串不可变，其哈希值在第一次计算后就可以被缓存，后续调用 hashCode 方法无需重复计算。这对 HashMap 等依赖哈希值的集合非常重要，能提升查找效率。如果字符串可变，哈希值会随内容变化，缓存就失去意义，甚至导致集合操作异常。还有确保线程安全。不可变对象天然线程安全，多线程环境下无需额外同步机制就能安全共享字符串。这在并发场景中很重要，比如多个线程同时操作同一个字符串时，不用担心内容被意外修改导致的一致性问题。

此外，这种设计也让 String 可以安全地作为 HashMap 的键、用于类加载器的路径等场景，因为其不可变性保证了这些场景下的稳定性和安全性。

关键字 finally 有什么作用，还有应用场景？

核心作用是保证资源释放或清理操作的执行。常见应用场景有打开了文件流、数据库连接、网络连接等资源时清理释放资源。

如果要重写一个对象的 equals 方法，需要怎么去重写或需要注意什么？

首先，要确保满足 equals 方法的自反、对称、传递、一致、非空五大特性，其次，重写时通常需要结合对象的关键属性进行比较。先判断是否为同一对象，若是直接返回 true。再检查参数是否为 null 或类型是否一致，若不是则返回 false。最后逐一比较对象的核心属性，只有所有关键属性都相等时才返回 true。

另外，按照 Java 的规范，当重写 equals 方法时，必须同时重写 hashCode 方法。这是因为如果两个对象通过 equals 比较为相等，它们的 hashCode 必须相同。反之，若 hashCode 不同，则 equals 一定返回 false，否则会导致 HashMap 等集合类出现异常行为。

还要注意避免在 equals 方法中使用可变属性，否则当属性变化后，对象在集合中的位置可能变得不正确。同时，比较引用类型属性时要使用 equals 而非 ==，而基本类型则直接使用 == 比较。

五大特性分别是什么意思？

自反性，即对象必须等于自身；对称性，若 a 等于 b，则 b 必须等于 a；传递性，若 a 等

于 b 且 b 等于 c，则 a 必须等于 c；一致性，多次调用 equals 方法应返回相同结果，除非用于比较的属性被修改；还有非空性，任何对象都不能等于 null。

HashMap 的 get 和 put 一个元素，大概是一个什么流程或者逻辑？

HashMap 的 get 和 put 操作核心是基于哈希表实现的键值对存储与查找。

put 元素时，首先会计算 key 的哈希值，通过哈希算法确定其在数组中的索引位置。如果该位置为空，直接将键值对存入；如果已有元素，会判断是链表还是红黑树结构：若是链表，遍历链表比较 key 是否存在，存在则更新 value，不存在则新增节点，当链表长度超过阈值且数组长度大于等于 64 时，会将链表转为红黑树；若是红黑树，则按照树的插入规则处理。此外，当元素数量达到负载因子与数组长度的乘积时，会触发扩容，将数组容量翻倍，并重新计算所有元素的存储位置。

get 元素时，同样先计算 key 的哈希值找到对应索引，然后根据该位置的结构查找：若是链表则遍历比较 key，若是红黑树则通过树的查找方式，找到匹配的 key 后返回对应的 value，若未找到则返回 null。

整个过程通过哈希算法快速定位，结合链表和红黑树处理冲突，平衡了时间和空间效率，使得平均情况下 get 和 put 操作的时间复杂度接近 $O(1)$ 。

HashMap 计算元素索引位置大概是怎么算的？

首先，获取键的哈希值。HashMap 会调用 key 的 hashCode 方法得到一个整数哈希值，然后通过扰动函数对这个哈希值进行处理，目的是让哈希值的高位也参与到后续计算中，减少哈希冲突的概率。

然后，计算索引位置。将处理后的哈希值与 HashMap 数组的长度减一进行按位与运算，得到的结果就是元素在数组中的索引位置。这是因为 HashMap 的数组长度通常是 2 的幂次方，此时 length-1 的二进制表示全为 1，按位与运算就相当于对 length 取模，既能保证索引值在数组范围内，又比取模运算更高效。

接口和抽象类区别？

接口是一种完全抽象的规范，它只定义方法签名，不包含任何具体实现，也不能有实例变量。一个类可以实现多个接口，实现类必须重写接口中的所有方法。接口主要用于定义类之间的通信契约，强调能做什么。

抽象类则是半抽象的，它可以包含抽象方法，也能有具体方法和成员变量。一个类只能继承一个抽象类，继承后需要实现所有抽象方法。抽象类更侧重于定义一类事物的共性特征，强调是什么。

String a = "abc" 和 new String("abc") 区别？

String a = "abc" 会先在字符串常量池中查找是否存在 "abc"。如果存在，直接让引用 a 指向这个常量；如果不存在，会在常量池中新建 "abc"，再让 a 指向它。这种方式不会在堆内存中创建新对象，能节省内存空间。

而 new String("abc") 则一定会在堆内存中创建一个新的字符串对象。同时，它也会检查字符串常量池，如果常量池中没有 "abc"，会先在常量池中创建这个常量，然后让堆中的对象指向常量池中的值。所以这种方式至少会创建一个堆对象，可能还会在常量池创建一个常量，相比前者更消耗内存。

另外，从内存地址比较来看，使用 == 比较时，前者如果是相同的字符串常量，会返回 true；而后者即使内容相同，由于是不同的堆对象，== 比较会返回 false，需要用 equals() 方法

才能比较内容是否相等。

LinkedHashMap 和 TreeMap 底层分别怎么实现对应功能？

LinkedHashMap 底层是通过**哈希表结合双向链表**实现的。哈希表负责保证键的快速查找，其实现与 HashMap 类似，通过键的哈希值确定存储位置，解决哈希冲突时采用**链地址法**，当链表长度超过阈值会转为红黑树。而额外维护的双向链表则记录了键值对的插入顺序或访问顺序，这使得 LinkedHashMap 可以按照**插入顺序**或**最近最少使用**的访问顺序进行迭代，同时保持了 HashMap 般的 $O(1)$ 平均查找、插入和删除效率。

TreeMap 底层则是基于**红黑树**实现的。红黑树的特性保证了键值对会按照键的自然顺序或**自定义比较器**的顺序进行排序和存储。在红黑树中，每个节点的左子树所有节点的键都小于该节点的键，右子树所有节点的键都大于该节点的键，这种结构使得 TreeMap 可以在 $O(\log n)$ 的时间复杂度内完成**插入、删除和查找**操作，同时支持**键的范围查询**和获取最大/最小键等排序相关的操作。

ArrayList 和 LinkedList 底层在实现上有什么区别？

ArrayList 底层基于动态数组实现，它会在内存中开辟一块连续的存储空间来存储元素。这种连续存储的特性使得它在随机访问（通过索引获取元素）时效率很高，时间复杂度为 $O(1)$ ，但在执行插入或删除操作时，尤其是在中间位置，需要移动大量元素来维护连续性，时间复杂度为 $O(n)$ 。

而 LinkedList 底层采用双向链表结构，每个元素通过节点的形式存储，节点之间通过指针相互连接，不需要连续的内存空间。这种结构导致它的随机访问效率较低，需要从表头或表尾开始遍历，时间复杂度为 $O(n)$ ，但插入和删除操作仅需修改相关节点的指针指向，时间复杂度为 $O(1)$ 。

此外，ArrayList 会有容量的概念，当元素数量达到容量上限时需要进行扩容，通常是将容量翻倍并复制原有元素，这可能带来额外的性能开销。而 LinkedList 则不需要考虑容量问题，元素的增减更灵活。

ArrayList 和 LinkedList 在遍历场景下有区别吗？

ArrayList 基于数组实现，底层是连续的内存空间，遍历元素时通过索引直接访问，遵循内存**局部性原理**，CPU 缓存利用率高，因此遍历效率更高，时间复杂度接近 $O(n)$ 。

而 LinkedList 基于双向链表实现，元素在内存中分散存储，遍历需要通过指针逐个访问下一个节点，**无法有效利用 CPU 缓存**，且**每次访问都要进行指针跳转**，额外开销更大，实际遍历效率通常低于 ArrayList。

因此，遍历场景下优先选择 ArrayList，尤其是需要频繁遍历或随机访问元素时；而 LinkedList 更适合频繁插入 / 删除的场景，遍历并非其优势。

synchronized 关键字放在 static 方法上和放在普通方法上有什么区别？

锁的对象不同。

当 synchronized 修饰**普通方法**时，锁的是当前**实例对象**，也就是说，**多个线程访问同一个实例的同步方法时会互斥**，但**不同实例之间的同步方法可以并行执行**。

而当 synchronized 修饰 static 方法时，锁的是当前类的 Class 对象，这个锁是**全局唯一**的，无论创建多少个类的实例，所有线程访问该**静态同步方法**时都会**共享这把锁**，从而实现**跨实例的互斥**。

简单来说，普通方法的 synchronized 锁是**实例级别**的，控制的是同一对象的**方法调用顺序**；

静态方法的 `synchronized` 锁是类级别的，控制的是所有对象对该方法的调用顺序。

HashSet 底层用 HashMap 实现，具体怎么实现的？

HashSet 利用 HashMap 的 key 来存储元素，而 value 则使用一个固定的空对象来占位。具体来说，当向 HashSet 中添加元素时，实际上是将该元素作为 key 存入了底层的 HashMap 中，同时关联一个固定的 value 对象。由于 HashMap 的 key 不允许重复，这就保证了 HashSet 中不会有重复的元素。

在查找元素时，HashSet 会调用底层 HashMap 的 `containsKey` 方法，通过判断 key 是否存在来确定元素是否在集合中。删除元素则是调用 HashMap 的 `remove` 方法，移除对应的 key-value 对。

这种实现方式让 HashSet 天然具备了 HashMap 的特性，比如元素无序、查找效率高等。但也因为只用到了 HashMap 的 key 部分，所以 HashSet 不关注 value，仅用于维护一个不重复的元素集合。

内部类和静态内部类有什么区别，使用场景上有什么不一样？

主要区别体现在与外部类的关联方式和使用限制上。

非静态的内部类会隐式持有外部类的实例引用，因此可以直接访问外部类的所有成员，包括私有成员。但它必须依赖外部类实例才能创建，不能在静态方法中直接实例化，也不能包含静态成员。

静态内部类则不持有外部类的实例引用，它与外部类的关系更像是独立的类，只是定义在外部类内部。它只能访问外部类的静态成员，创建实例时无需依赖外部类实例。

使用场景上，内部类适合与外部类实例紧密关联的场景，比如作为外部类的辅助组件，需要频繁访问外部类实例状态，例如集合类中的迭代器实现。而静态内部类更适合作为外部类的附属工具类，不依赖外部类实例，仅需要与外部类存在逻辑上的关联，比如建造者模式中的建造者类，或者一些仅为外部类服务的工具方法封装。

你提到的“先后”感，其实源于初始化时机：

静态内部类：由于它不持有外部类的引用，它可以在外部类还没实例化的情况下就被初始化。

非静态内部类：它的初始化强制要求一个已经存在的外部类实例。

编译器层面的小动作：

编译器会给非静态内部类偷偷加一个构造参数，用来接收外部类的 `this`。而静态内部类则没有这个待遇。所以，静态内部类在逻辑上更像是一个“披着内部类外衣的顶级类”。

并发和并行的关系？

并发指的是系统能够同时管理多个任务的执行状态，这些任务可能在同一个 CPU 核心上交替执行，从用户视角看它们像在同时运行，但本质是 CPU 在快速切换处理；而并行指的是多个任务在物理上的多个 CPU 核心同时执行。且并行依赖并发的任务调度基础，只有系统先能管理多个任务，才有可能将这些任务分配到不同核心并行执行。另一方面，并行是提升并发效率的关键手段，如果只有单核心，无论并发管理多高效，任务都只能串行交替，效率有限。而多核心的并行执行，能让多个并发任务真正同时推进，大幅减少整体任务的完成时间。

线程的生命周期？

首先是新建状态，当我们创建一个线程对象时，线程就处于新建状态，此时它仅在内存中存

在，还**未分配系统资源**，也**没有开始执行**。

接着是**就绪状态**，当调用线程的 `start` 方法后，线程进入就绪状态。这时线程已经获取了除 CPU 之外的所有必要资源，等待操作系统的**调度**，一旦**得到 CPU 时间片**，就会进入运行状态。

然后是**运行状态**，处于就绪状态的线程**获得 CPU 资源后**，便会执行 `run` 方法中的代码，此时线程处于运行状态。在这个阶段，线程可能会因为**时间片用完而回到就绪状态**，等待下一次调度。

当线程运行过程中遇到 `sleep` 方法、等待同步锁、调用 `wait` 方法等情况时，会进入**阻塞状态**。此时**线程暂时停止执行**，**释放 CPU 资源**，但会保留自己的状态信息。当阻塞原因消除后，线程会重新回到就绪状态，等待再次获得 CPU 资源。

最后是**终止状态**，当线程的 `run` 方法执行完毕，或者**因异常导致线程终止**，线程就会进入终止状态。此时**线程的生命周期结束**，不再参与调度，所占用的**资源也会被回收**。

说说线程的上下文切换？

线程的**上下文切换**是指 CPU 从一个线程切换到另一个线程执行时，**保存当前线程的状态并加载新线程状态的过程**。

当多个线程共享 CPU 资源时，**操作系统**需要通过**调度算法**决定哪个线程获得执行权。此时，CPU 需要先保存当前**线程的执行上下文**，包括**程序计数器、寄存器值、栈指针**等关键信息，这些信息会被存储在**内存**中。然后，加载下一个要执行线程的上下文信息，**恢复其寄存器状态和程序计数器**，让 CPU 可以从该线程**上次暂停的位置继续执行**。

上下文切换会带来一定的**性能开销**，因为保存和加载状态需要消耗 CPU 时间，频繁的切换可能导致系统**效率下降**。特别是在线程数量过多时，大量的上下文切换会占用大量 CPU 资源，反而影响程序的整体执行效率。

讲讲 aop？你是怎么用的？

AOP 是一种编程范式，通过将横切关注点从**业务逻辑**中分离出来，实现代码的**模块化和解耦**。在传统的面向对象编程中，**横切关注点**会分散在各个业务方法中，导致代码冗余且难以维护。而 **AOP 通过定义切面来封装这些横切逻辑**，然后在程序**运行时动态将切面代码织入**到指定的业务方法中，实现了业务逻辑与横切逻辑的分离。

使用时是通过**注解或配置定义切入点**和**通知**。比如使用 `@Aspect` 定义切面，通过 `@Before`、`@After`、`@Around` 等注解声明前置、后置、环绕通知，指定在哪些方法执行前后插入**日志记录或事务控制**等逻辑。

多态是什么？怎么实现？重写重载又是怎么实现？

多态是面向对象编程中的重要特性，指**同一操作作用于不同对象时，会产生不同的执行结果**。实现多态主要通过**继承和接口**实现。在继承体系中，**子类重写父类的方法**，当用父类引用**指向子类对象时**，调用该方法会执行**子类的实现**；通过接口时，不同类实现同一接口的方法，**接口引用指向不同实现类对象**，调用方法会执行**对应类的实现**。

重写是实现多态的基础，指子类重新实现父类中已有的方法，要求**方法名、参数列表、返回值类型完全一致**，**访问权限不能严于父类**，**抛出异常不能宽于父类**。运行时，**JVM 会根据对象实际类型调用对应重写方法**，这就是**动态绑定**。

重载是在**同一类**中，多个方法具有相同名称但**参数列表**不同，与**返回值和访问修饰符**无关。重载是**编译时多态**，编译器根据**调用时的参数类型匹配**对应的方法。

浅拷贝和深拷贝说一下，深拷贝怎么实现

浅拷贝和深拷贝都是复制对象的方式，核心区别在于对嵌套对象的处理。

浅拷贝只会复制对象本身，对于对象中的嵌套对象，新对象和原对象会共享同一引用，引用类型（Object, Array）在拷贝时只会复制一份内存地址（指针）。这意味着修改新对象里的嵌套对象时，原对象也会受到影响。常见的浅拷贝方式有对象的扩展运算符、Object 类的 assign 方法，以及数组的 slice 等方法。

深拷贝则是完全复制整个对象结构，包括所有嵌套对象。新对象和原对象是完全独立的，修改任何一方都不会影响另一方。

实现深拷贝可以通过递归实现，遍历对象的每个属性，如果是基本类型就直接复制，如果是引用类型就递归调用拷贝方法，这样能处理各种数据类型，保证拷贝的彻底性。

JDK 和 JRE 和 JVM 的关系？

JVM 是执行 Java 字节码的虚拟计算机，负责将字节码翻译成机器码并运行，是 Java 跨平台特性的核心。

JRE 包含了 JVM，以及 Java 程序运行所需的核心类库和其他支持文件。只要安装了 JRE，就能运行编译好的 Java 程序。

JDK 则是面向开发者的，不仅包含了完整的 JRE，还提供了一系列开发工具，比如编译器、调试工具等。开发者必须安装 JDK 才能进行 Java 程序的开发、编译工作。

java 中静态代码块以及构造函数的区别？

静态代码块用 static 修饰，属于类级别的代码块，它在类加载到内存时就会执行，而且只会执行一次，无论后续创建多少个该类的实例，静态代码块都不会再运行。它通常用于初始化类的静态成员变量或执行类级别的初始化操作。

构造函数则是在每次创建类的实例时被调用，每创建一个实例就会执行一次对应的构造函数。它的主要作用是初始化对象的实例成员变量，为对象的创建做准备。

构造代码块和构造函数的区别？

构造函数是类中与类名相同的特殊方法，当通过 new 关键字创建对象时会被自动调用，主要用于对当前对象进行初始化，比如为成员变量赋值等。每个类可以有多个重载的构造函数，通过不同的参数列表满足不同的初始化需求，且构造函数没有返回值。

构造代码块则是直接定义在类中、用大括号包裹的代码片段，它不属于任何方法。当创建类的对象时，构造代码块会在所有构造函数之前执行，并且每次创建对象都会执行一次。构造代码块通常用于提取多个构造函数中的公共初始化逻辑，避免代码重复，比如对某些成员变量进行统一的初始化操作。

父类和子类中静态代码块、构造代码块以及构造方法，在执行子类是如何执行的？

当实例化子类对象时，父类和子类的静态代码块、构造代码块及构造方法的执行顺序是固定的，遵循“先静态后非静态，先父类后子类”的原则，具体可以分为以下几个步骤。

首先执行父类的静态代码块，这是因为静态代码块属于类级别的初始化，在类加载时执行，且只执行一次。接着执行子类的静态代码块，同样在子类加载时执行，且仅执行一次。

静态内容执行完毕后，进入对象实例化阶段。此时先执行父类的构造代码块，它在每次创建对象时都会执行，且位于父类构造方法之前。然后执行父类的构造方法，完成父类对象的初始化。

父类初始化完成后，再执行子类的构造代码块，最后执行子类的构造方法，完成子类对象的

实例化。

java 中内部类和静态内部类如何用？

内部类是依附于外部类实例的，可以直接访问外部类的所有成员。创建内部类实例时，必须先有外部类实例，通常通过 “外部类实例.new 内部类”的方式创建（读法外部类实例点内部类，下边也是类似）。静态内部类则不依赖外部类实例，相当于一个独立的类，只是定义在外部类的命名空间里。它只能访问外部类的静态成员，创建实例时无需外部类对象，直接通过 “new 外部类.静态内部类” 即可。

MySQL

请讲讲 mysql 的 sql 优化技巧有哪些？

避免使用 `select *`，`\*` 会取出所有字段。如果表里有很大的 `TEXT` 或 `BLOB` 字段，会产生巨大的网络传输压力。使用具体的列名，更有可能触发**覆盖索引（Covering Index）**。即直接从索引树里就能拿到数据，不需要再回表查主键索引，速度快几个数量级。用 `union all` 代替 `union`，`union` 会对合并后的结果集进行**去重**（类似于 `DISTINCT`），这涉及排序和计算。

小表驱动大表，在使用 `JOIN` 时，MySQL 会选择一个表作为“驱动表”，尽量让数据量更小的表放在左边。

用**连接查询代替子查询**，子查询在执行时，MySQL 有时会创建**临时表**。查询完毕后还要销毁临时表，这一来一回的 IO 消耗很大。

join 的表不宜过多，

添加索引和索引优化，- `WHERE` 子句中频繁出现的字段。- `ORDER BY`、`GROUP BY` 涉及的字段。- `DISTINCT` 字段。

说说 mysql 的回表查询？

MySQL 的回表查询是一种在查询过程中需要从索引和数据文件中多次获取数据的操作，当查询语句使用了**覆盖索引**时，MySQL 可以直接从索引中获取数据，而无需访问数据文件。但如果查询的列不都包含在索引中，就需要先通过**索引找到对应的主键值**，然后再根据主键值去**数据文件**中查找完整的行数据，这个过程就叫做回表查询。

回表查询会增加查询的成本，因为它需要额外的 I/O 操作来从数据文件中获取数据。如果回表次数过多，可能会导致查询性能下降。特别是在处理大量数据时，回表操作可能会成为性能瓶颈。不过，如果索引设计合理，回表查询的次数可以得到有效控制，从而保证查询性能。例如，尽量使用覆盖索引，避免不必要的回表查询；或者根据查询条件和业务需求，合理设计**联合索引**，将经常一起查询的列包含在索引中，减少回表的可能性。

什么是覆盖索引？

覆盖索引是一种包含了查询所需所有列的索引，当查询语句使用这类索引时，MySQL 可以直接从索引中获取到所有需要的数据，无需再去访问数据文件，所以避免了回表查询。

比如，如果查询的列是 id、name，而刚好有一个联合索引包含了这两个列，那么查询时直接通过这个索引就能拿到结果，不用再根据主键去数据文件里找其他字段。

使用覆盖索引能显著提升查询性能，因为它减少了 I/O 操作——不需要从索引跳转到数据文件，直接从索引就能获取全部信息。所以在实际开发中，合理设计包含查询所需字段的索引，利用覆盖索引可以有效优化查询效率。

Select for update 是什么含义？

SELECT FOR UPDATE 是数据库中的一种行级锁定语句，用于在查询数据时对所选中的行施加排他锁。

当执行该语句时，数据库会锁定符合查询条件的行，防止其他事务对这些行进行修改或添加排他锁，直到当前事务提交或回滚后，锁才会释放。

这种机制主要用于解决并发场景下的数据一致性问题。例如，在秒杀、库存扣减等业务中，通过 SELECT FOR UPDATE 可以确保读取到的数据状态在事务处理期间不会被其他事务篡改，避免出现超卖等异常情况。

SELECT FOR UPDATE 通常要在事务环境中使用，**如果不开启事务**，SQL 执行完的一瞬间锁就自动释放了，那“占坑”就毫无意义了。且只对使用索引的查询条件有效，这种情况下才是行锁，否则对没索引的字段可能会升级为表锁，影响并发性能。

慢查询日志了解吗？

慢查询日志有所了解，是 MySQL 中用于定位性能瓶颈、优化查询效率的重要工具，核心作用是记录下执行时间超过指定阈值的 SQL 语句，方便后续分析和优化。

慢查询日志的工作机制是 MySQL 会预先设置一个慢查询时间阈值，当某条 SQL 语句的实际执行时间超过这个阈值时，就会被记录到慢查询日志文件中。

慢查询日志的核心价值在于定位低效 SQL。比如线上业务出现查询卡顿、数据库 CPU 或 IO 使用率过高时，就可以查看慢查询日志，找出那些执行时间长、可能存在索引缺失、关联查询逻辑不合理或全表扫描的 SQL。

Sql 语句的内部执行流程？

第一步是词法与语法解析。数据库首先会把输入的 SQL 字符串拆成一个个“词法单元”，接着会用语法规则校验这些单元的组合是否合法，一旦发现语法错误，就会直接返回错误信息，不会进入后续流程。

第二步是语义分析。这一步要解决语法对但逻辑不对的问题，确保后续操作的对象是“真实且可访问”的。

第三步是**生成执行计划**。这一步是性能优化的核心。数据库会根据**表的索引情况、数据量大小、统计信息**，生成多种“执行方案”，再通过**内部的成本模型**选择最优的方案。第四步是**执行计划的执行**。数据库会按照最优执行计划，调用**存储引擎**去实际操作数据。第五步是**结果处理**。当存储引擎返回数据后，数据库会对结果进行后续处理，比如进行排序、截取前 10 条数据、分组，或者对**查询结果去重**。

第六步是**结果返回**。最后，数据库会把处理后的最终结果，以**规定的格式**返回给客户端，完成整个 SQL 的执行流程。

B+树、B树和二叉树的区别？

二叉树是最基础的树形结构，每个节点最多只有两个子节点。有明显问题，如果插入数据是有序的，会退化成链表，此时查询效率会从理想的 $O(\log n)$ 骤降到 $O(n)$ ，而且节点只存单个数据，**层高会随着数据量增长快速增加**，频繁的磁盘 I/O 会严重影响性能，所以很少直接用于**数据库、文件系统**这类需要高效处理大量数据的场景。

B树可以理解为**多叉平衡树**，解决了二叉树层高过高的问题，**B树的每个节点**（包括根和枝干）都存储了完整的**行数据**，每个节点占用的空间很大，但相比同层高的 **B+树**，**索引覆盖的范围更小**，而且**B树的叶子节点之间没有关联**，如果要做**范围查询**，只能从根节点开始逐个遍历符合条件的节点，效率较低。

而**B+树**是在 **B树**基础上针对数据库、文件系统优化的结构，数据下沉和叶子节点的横向相连，这样所有查询的路径长度一致，性能更稳定。底层的叶子节点是有序排列的，且用双向链表互相拉着手。做范围查询很好用，比如查“**10岁到20岁**”，先定位到 **10岁**，然后顺着链表往后一拉，全部数据就出来了，根本不需要回过头去翻根节点

普通二叉树因易退化、层高过高，几乎不用在大规模数据存储场景，**B树适合少量数据的快速单点查询，但范围查询弱**，而 **B+树**层高更低、范围查询高效、性能稳定。

为什么 mysql 索引要用 B+树？

B+树的节点可以**存储多个键值对和指针**，这样可以将更多的信息存储在一个节点中，减少磁盘 I/O 次数。每次读取一个节点，就能获取到多个键值对的信息，从而提高查询效率。

同时 **B+树的叶子节点**是按照键值的**顺序**排列的，并且通过链表相连。这使得范围查询变得非常高效，只需要从链表的一端开始，按照顺序遍历即可获取到指定范围内的所有数据。

以及，**B+树**是一种平衡树，查询性能稳定。

说说主键索引和非主键索引最主要的区别是什么？

主键索引的**叶子节点直接存储整行数据**，查询时只要旦找到对应的索引项，就能获取完整的行记录，不需要额外操作。

而非主键索引的叶子节点只存储主键值，当通过非主键索引查询到数据时，如果需要获取除索引列和主键之外的其他字段，就必须用查到的主键值去主键索引中再次查询，这个过程就是回表。

这种差异导致主键索引查询效率通常更高，而非主键索引可能因回表操作增加 I/O 开销，影响性能。

说说 mysql 的 explain 关键词？

MySQL 的 explain 关键词是用于分析 SQL 查询语句执行计划的重要工具，它能帮助我们了解 MySQL 数据库是如何处理一条 SQL 语句的，包括索引的使用情况、表的连接方式、数据扫描范围等关键信息，从而为优化查询性能提供依据。

使用时，只需在 SQL 语句前加上 explain 关键字，执行后就会得到一张包含多个字段的結果表。这些字段各有特定含义，比如 type 字段表示连接类型，从好到差有 system、const、eq_ref、ref、range、index、ALL 等，其中 ALL 意味着全表扫描，性能通常较差；key 字段会显示实际使用的索引，如果为 NULL 则表示没有使用索引。rows 字段则是 MySQL 预估的需要扫描的行数，数值越小说明查询效率可能越高。

通过分析 explain 的结果，可以判断查询是否合理使用了索引，是否存在全表扫描等低效操作，进而针对性地优化索引设计或调整 SQL 语句。比如当发现 type 为 ALL 时，可能需要为查询条件添加合适的索引。当 key 与预期使用的索引不符时，可能需要检查索引是否失效或调整查询条件，让优化器选择更优的执行计划。

mysql 索引的数据结构是怎样的？

MySQL 索引的数据结构主要采用 B+ 树，B+树是一种专为数据库系统优化的多路平衡搜索树，每个节点可以有多个子节点，所有数据都存储在叶子节点，非叶子节点仅存储索引键和指针，叶子节点之间通过指针相连，形成有序链表。

B+树的平衡特性保证了查询效率的稳定性，多路设计减少了树的高度，降低磁盘 I/O 次数，叶子节点的有序链表支持高效的范围查询。

mysql 中 mvcc 是什么？

Mvcc 是多版本并发控制，一种数据库并发控制方法，通过维护数据的多个版本来实现读写操作的并行执行。它允许读操作不加锁，避免了读写之间的阻塞，从而提高了系统的并发性能。多版本并发控制作为一种有效的并发控制机制，被广泛应用于现代关系型数据库系统中，如 InnoDB 存储引擎。

MVCC 的主要目的是实现高并发下的数据一致性与隔离性。它通过为每个事务提供数据的一个“快照”，确保事务在其执行期间看到的数据是一致的，从而避免了幻读、不可重复读等现象。此外，MVCC 还减少了锁的使用，降低了锁竞争，提高了系统的吞吐量。

MySQL 如何实现事务？

MySQL 的事务机制主要通过 InnoDB 存储引擎实现，其核心是确保数据操作满足 ACID 特性。

原子性的实现依赖于 undo 日志，当事务需要回滚时，undo 日志会记录数据修改前的状态，从而将数据恢复到初始状态；

一致性则通过原子性、隔离性和持久性共同保障，同时结合数据库的约束条件来确保数据完整性。

隔离性的实现较为复杂，InnoDB 同时采用锁机制和多版本并发控制 MVCC，锁机制包括共享锁、排他锁以及间隙锁，而 MVCC 则通过为每行数据维护版本号和 undo 日志链，让不同事务能读取到一致的数据视图。

持久性的保障来自 redo 日志，采用预写式日志策略，事务提交时先将操作记录写入 redo 日志，确保即使数据库崩溃也能通过日志恢复数据。

什么是间隙锁？

间隙锁是 MySQL 中 InnoDB 存储引擎在可重复读隔离级别下，为了防止幻读而引入的一种锁机制，它锁定的不是具体的行记录，而是两条记录之间的间隙。比如表中有 id 为 10、20、30 的三条记录，当执行 `select * from table where id between 10 and 30 for update` 时，InnoDB 不仅会给 id=10、20、30 这三条行记录加行锁，还会给 10 之前的间隙、10-20 之间的间隙、20-30 之间的间隙、30 之后的间隙这四个区间加间隙锁。这样一来，其他事务就无法在这些间隙中插入新的记录比如插入 id=15、25，从而避免了当前事务再次查询时，因为有新记录插入而出现“幻读”的情况。

不过间隙锁也会带来一些影响，它可能会增加锁等待的概率，甚至引发死锁。因为间隙锁与间隙锁之间是不冲突的，比如两个事务分别对不同的间隙加锁后，又尝试插入对方锁定的间隙范围，就可能互相等待对方释放锁。

MySQL 超大分页如何处理？

可以利用覆盖索引，确保查询的字段全部包含在索引中，避免回表查询。

可以使用书签分页，通过记录上次查询的最后一条记录的主键或索引值，直接定位到下一页的起始位置，避免扫描大量无关数据，适用于 API 接口的分页场景，如社交媒体的滚动加载。可以提前分表存储，将数据按特定规则如时间预先分表存储，查询时直接定位到对应表，减少查询时的表数据总量，适用于数据有明显时间或范围划分的场景，如日志系统。

以及对于需要遍历全量数据如数据导出场景，可以使用游标逐批处理。还有要避免使用 limit 和 offset，有可能导致全表扫描和占据过多临时内存与磁盘空间。

请讲讲 mysql 事务的四大特性是哪些？

原子性，指事务包含的所有操作要么全部成功，要么全部失败回滚，因此事务的操作如果成功就必须完全应用到数据库，如果操作失败则不能对数据库有任何影响。

一致性，指事务必须使数据库从一个一致性状态变换到另一个一致性状态，也就是说一个事务执行之前和执行之后都必须处于一致性状态。

隔离性，当多个用户并发访问数据库时，比如操作同一张表时，数据库为每一个用户开启的事务，不能被其他事务的操作所干扰，多个并发事务之间要相互隔离。

持久性，指一个事务一旦被提交了，那么对数据库中的数据的改变就是永久性的，即便

是在数据库系统遇到故障的情况下也不会丢失提交事务的操作。

什么是脏读、不可重复读、幻读？

脏读：事务 A 读取了事务 B 更新的数据，然后 B 回滚操作，那么 A 读取到的数据是脏数据。

不可重复读：事务 A 多次读取同一数据，事务 B 在事务 A 多次读取的过程中，对数据作了更新并提交，导致事务 A 多次读取同一数据时，结果不一致。

幻读：事务 A 对表中的数据进行了修改，涉及到表中的全部行。同时，事务 B 也修改这个表中的数据，向表中插入一行新数据。那么，事务 A 发现表中还有自己没有修改的行，就好象发生了幻觉一样。

请你讲讲 MySQL 的 update 语句？

UPDATE 语句用于修改表中已存在的数据记录。基本语法是通过 SET 子句指定要更新的列及其新值，再通过 WHERE 子句限定需要更新的行范围。

如果没有 WHERE 子句，UPDATE 会更新表中所有行，这在实际操作中需要特别谨慎，避免误操作导致全表数据被修改。

在执行 UPDATE 时，MySQL 会先根据 WHERE 条件找到符合条件的记录，然后对这些记录的指定列进行更新。这个过程中，数据库会自动处理事务，确保更新操作的原子性，要么全部成功，要么失败时回滚到修改前的状态。

对于有索引的列，更新操作可能会触发索引的维护，这会带来一定的性能开销。因此，在更新大量数据时，建议合理规划操作，比如分批次更新，避免长时间占用表资源导致锁表，影响其他业务操作。

另外，UPDATE 语句还支持使用 FROM 子句关联其他表的数据来进行更新，这在需要根据关联表的信息修改当前表数据时非常有用。同时，也可以结合 LIMIT 子句限制更新的行数，进一步提高操作的可控性。

最左前缀匹配原则是什么？

最左前缀匹配原则是 MySQL 中联合索引的一项重要规则，指的是在使用联合索引进行查询时，数据库会优先匹配索引中最左侧的字段，并且按照索引定义的字段顺序依次匹配。只要查询条件中包含联合索引最左侧的字段，就能触发索引的部分或全部使用。

比如，如果创建了一个联合索引 (a, b, c)，那么查询条件中包含 a 时，索引就可能被用到；如果同时包含 a 和 b，也能利用到索引的前两列；但如果查询条件中没有 a，哪怕包含 b 和 c，也无法使用这个联合索引。

这一原则的本质是联合索引的存储结构决定的，索引会按照字段顺序逐层排序，只有从最左侧开始匹配，才能有效定位到数据。合理利用这一原则设计联合索引，可以提升查询效率，比如将查询频率高、区分度高的字段放在联合索引的左侧，能让更多查询命中索引。反之，如果违背这一原则，可能导致索引失效，增加全表扫描的概率，影响查询性能。

联合索引遵循最左前缀匹配原则，原因跟索引结构有什么关系？

联合索引的底层结构是 B+ 树，而联合索引的 B+ 树节点里，存储的是多个索引字段的组合值+主键，并且整个 B+ 树是按照索引字段的定义顺序来构建和排序的。

这种按左到右字段顺序排序的结构，直接决定了查询时必须遵循最左前缀匹配。因为 B+ 树的查找是从根节点到叶子节点的有序定位过程，只有先匹配最左侧的第一个字段 a，才能确定后续字段 b、c 的查找范围。

(下方仅方便理解，不需要面试时回答)

举个例子，假设我们建了联合索引 (a,b,c)，它的 B+ 树叶子节点是按 “a 值→b 值→c 值” 的顺序有序排列的。如果查询时只条件匹配 b 和 c（比如 where b=2 and c=3），B+ 树根本无法定位 —— 因为叶子节点里，不同 a 值下都可能存在 b=2、c=3 的记录，没有 a 的过滤，数据库不知道该从哪个 “a 分组” 里找 b 和 c，只能全量扫描索引，这就失去了索引的意义。

但如果我们遵循最左前缀，比如只查 a (where a=1)，或者查 a 和 b (where a=1 and b=2)，B+ 树就能先通过最左侧的 a 定位到一个确定的区间，再在这个区间里按 b（或 b 和 c）进一步缩小范围，最终快速找到目标记录的主键，这正是索引提升查询效率的核心逻辑。

InnoDB 索引数据结构的高度一般是 3 或 4，为什么能做到高度这么低？

采用了 B+ 树索引结构，B+ 树的单个非叶节点能容纳非常多的索引条目，从而大幅减少树的层级。并且通过充分利用磁盘页空间、最大化单节点存储的索引条目数量，实现了用少量层级就能管理海量数据的效果。另外，InnoDB 的聚簇索引结构也间接帮了忙，聚簇索引的叶节点直接存储完整数据行，不需要额外关联其他数据文件，这让叶节点的数据密度更高，而非聚簇索引的叶节点存的是主键值，同样比存完整数据更节省空间，进一步保证了每个节点能容纳足够多的索引条目，避免树高因节点存储效率低而被迫增加。

讲讲 mysql 事务的四种隔离级别以及对应会遇到的问题？

读未提交，能读到未提交的数据。会出现脏读、不可重复读、幻读。

读已提交，读已提交的数据。会出现不可重复读和幻读。

可重复读，mysql 默认的事务隔离级别，查询的都是事务开始时的数据。只会出现幻读。

串行读，完全串行化读，每次都会锁表，读写互相阻塞。最高隔离级别，不会出现脏读，不可重复读，幻读。但会大大影响系统的性能，一般不用。

mysql 的索引是什么？有哪些索引？

普通索引

最基本的索引类型，它没有任何限制，其主要目的就是加快对数据的访问速度。

唯一索引

要求索引列中的值必须唯一，即不允许出现重复值，但可以有 NULL 值。这种索引常用于确保数据的唯一性，同时也能提高查询效率。

主键索引

一种特殊的唯一索引，一个表只能有一个主键索引，并且主键索引不允许有空值。它通常用于唯一标识表中的每一行数据，是表中记录的唯一标识符。

全文索引

主要用于在文本类型的列中进行全文搜索，例如 CHAR、VARCHAR 或 TEXT 类型的列。它可以对文本内容进行分词处理，然后根据关键词进行快速搜索。

空间索引

用于对空间数据类型的列进行索引。它可以加速空间数据的查询，例如查找某个区域内的地

理坐标点。

复合索引

指在多个列上创建的索引。使用复合索引时，MySQL 可以根据索引中的列顺序依次进行查找，因此在设计复合索引时，列的顺序非常重要。

以及并非一种单独索引类型而是一种数据存储方式的聚簇索引，特点是表数据会按照主键的顺序进行物理存储。由主键索引实现，因此每个表只能有一个聚簇索引。

说说索引怎样合理设置？

频繁用于查询条件、join 连接操作、排序、分组、唯一约束或主键的列应该设置索引，小表不应该设置索引，避免冗余索引，控制索引数量，对长字符串使用前缀索引。

讲一下索引的失效场景？

查询条件不满足最左前缀原则，在索引列上使用函数或表达式，使用 OR 连接条件时部分列无索引，复合索引中其中一列进行了范围查询，模糊查询以通配符开头，强制索引失效

mysql 有哪些存储引擎？对比特点是什么？

InnoDB，有事务支持，InnoDB 是 MySQL 默认的存储引擎，它支持事务处理，遵循原子性、一致性、隔离性、持久性原则。

支持外键约束，这有助于维护数据的参照完整性。当一个表中的数据依赖于另一个表中的数据时，外键约束可以确保数据的一致性。

采用行级锁，在并发访问时可以提高性能。当多个事务同时操作不同行的数据时，行级锁可以减少锁的竞争，提高并发处理能力。

使用聚簇索引，数据行按照主键的顺序存储在磁盘上。这使得基于主键的查询非常高效，因为可以直接定位到数据行所在的物理位置。

适用于对事务处理要求较高、数据一致性要求严格、并发访问频繁的应用。

另一个 MyISAM 不支持事务处理，这意味着它不能保证一组操作的原子性和一致性。在进行数据修改时，如果出现错误，无法回滚到操作前的状态。

也不支持外键约束，因此无法通过外键来维护数据的参照完整性。

采用表级锁，在并发访问时性能较差。当一个事务对表进行写操作时，会锁定整个表，其他事务必须等待该锁释放才能进行操作。

支持全文索引，对于文本搜索功能有较好的支持。

适用于对事务和外键要求不高，以读操作和插入操作为主的应用。

对 Mysql 的锁有了解吗？

按锁粒度分类有全局锁、表级锁、行级锁，全局锁对整个数据库实例加锁，表级锁有直接锁定整张表的表锁，表明某个事务正在锁定表的某一行或准备锁定的意向锁，行锁有允许事务读取一行数据的共享锁，允许事务更新或删除一行数据的排他锁。

按理念划分有假设冲突总会发生，直接加锁的悲观锁和假设数据在读取和更新期间不会发生冲突，因此不直接加锁，而是在更新时验证数据是否被其他事务修改过。如果发现冲突，则回滚重试的乐观锁。

你对哪些设计模式有所了解，讲讲这些设计模式？（设计模式只有这一个问题，因此放到 mysql 下）

工厂设计模式，提供了一种创建对象的方式，通过一个工厂类来负责对象的创建过程，而不

是在客户端代码中直接使用 `new` 关键字来实例化对象。这样做的好处是，当对象的创建逻辑发生变化时，只需要修改工厂类，而不会影响到使用这些对象的客户端代码。

优点

解耦对象的创建和使用，将对象的创建逻辑封装在工厂类中，客户端只需要使用对象，不需要关心对象的创建过程，降低了代码的耦合度。

提高可维护性和可扩展性，当对象的创建逻辑发生变化时，只需要修改工厂类，而不会影响到客户端代码。同时，添加新的产品时，只需要扩展工厂类或创建新的具体工厂类。

符合开闭原则，工厂方法模式和抽象工厂模式遵循开闭原则，对扩展开放，对修改关闭。

缺点

增加代码复杂度，随着工厂模式的使用，类的数量会增加，导致系统复杂度上升。

理解难度较大，对于初学者来说，理解工厂模式的概念和实现可能有一定的难度。

建造者模式，在软件开发中，有些对象的创建过程较为复杂，包含多个步骤，并且可能有多种不同的配置组合，而该模式将对象的创建步骤进行封装，通过一个指挥者来控制这些步骤的执行顺序，从而可以创建出不同表示的对象，而客户端只需要指定要创建的对象类型，无需关心具体的创建过程。

优点

封装性好，将对象的创建过程封装在具体建造者中，客户端无需了解对象的具体创建细节，降低了代码的耦合度。

可扩展性强，可以很方便地添加新的具体建造者来创建不同表示的对象，符合开闭原则。

构建过程可控：指挥者可以控制构建过程的顺序，确保对象的构建步骤按照正确的顺序执行。

缺点

代码复杂度增加，由于引入了多个类，代码量会增加，结构变得相对复杂。

适用范围有限，建造者模式适用于创建复杂对象，如果对象的构建过程简单，使用该模式会显得冗余。

责任链模式，多个处理者组成一条链，每个处理者都有机会处理请求。当客户端发出一个请求时，该请求会从链的头部开始传递，依次经过每个处理者。每个处理者在接收到请求后，会判断自己是否能够处理该请求，如果可以，则进行处理；如果不能，则将请求传递给链中的下一个处理者，直到请求被处理或者到达链的末尾。

优点

降低耦合度，请求发送者和接收者之间解耦，发送者不需要知道哪个处理者会处理请求，只需要将请求发送到链上即可。

可扩展性强，可以动态地增加或删除处理者，或者改变处理者的顺序，而不会影响到客户端代码。

灵活性高，可以根据实际需求灵活地组织处理者链，满足不同的业务逻辑。

缺点

请求可能得不到处理，如果处理者链配置不当，可能会导致请求在链中传递到末尾都没有被处理。

调试困难，由于请求的处理过程分散在多个处理者中，当出现问题时，调试和排查错误可能会比较困难。

性能问题，如果处理者链过长，请求的传递会消耗一定的时间和资源，影响系统性能。

讲讲 Mysql 主从同步？

工作流程主要分为三个关键步骤，

当主库执行 INSERT、UPDATE、DELETE 等写操作时，会先将这些操作的逻辑记录到二进制日志中，binlog 会按操作顺序生成，确保数据变更的时序性。

然后从库会启动一个 IO 线程，主动连接主库，请求获取主库新生成的 binlog 内容。主库则会启动一个 dump 线程，响应从库的请求，将未同步的 binlog 数据源源不断地发送给从库。从库接收到这些 binlog 后，不会直接执行，而是先写入到本地的中继日志中，避免因从库执行速度慢导致同步中断，也能保证 binlog 的完整性。

最后从库还会启动一个 SQL 线程，这个线程会实时读取中继日志的内容，按照 binlog 记录的逻辑，在从库上重复执行一遍相同的写操作。

这样主库只负责写操作，从库负责读操作，能分散数据库的读写压力，避免单库因读写并发过高导致性能瓶颈。且从库本身就是主库的一份实时备份，当主库因硬件故障、数据误删等问题不可用时，能快速将从库切换为主库，减少业务中断时间，提升系统可用性。

MySQL 索引建立过多的代价？

首先是存储资源额外消耗，除了数据的存储，还有备份、迁移时的开销都增大了。

然后会严重影响写入操作的性能。MySQL 执行 INSERT、UPDATE、DELETE 这类写入语句时，不仅要修改数据表本身的行数据，还需要同步更新所有相关的索引。写入操作的性能会随索引数量增加而线性下降。

最后会干扰查询优化器的判断，甚至导致低效查询。MySQL 查询优化器会根据索引情况选择最优执行计划，但当索引过多时，优化器需要遍历更多索引组合来评估成本，可能导致选择超时或选错索引，导致需要扫描大量无关数据，反而比无索引时的全表扫描更慢。

出现了什么问题会用到 undo log 和 redo log？（之前放错位置了，现在纠正，从提问文档放到八股）

redo log 主要用于应对“事务执行中数据库突然崩溃”的问题。当执行 INSERT、UPDATE、DELETE 等写操作时，InnoDB 并不会直接把修改立刻写入磁盘数据文件，因为磁盘随机写的效率很低，频繁直接写会严重影响性能。这时候 redo log 就会先记录下“某数据页被修改成了什么样子”，而且 redo log 的写入是顺序写，写完 redo log 后，事务就可以先标记为“已提交”，后续 InnoDB 会通过后台线程，再把 redo log 里的修改异步刷到磁盘数据文件中。

如果此时数据库突然断电或崩溃，之前没来得及刷到数据文件的修改，会在数据库重启后，通过读取 redo log 重新执行一遍，确保已经提交的事务修改不会丢失。

undo log 主要用于解决事务需要回滚和 MVCC 读的问题。一方面，当事务执行过程中需要回滚，undo log 就会发挥作用，因为记录了事务修改前的数据状态，回滚时，InnoDB 会根据 undo log 的记录，把数据恢复到修改前的状态。

什么是索引下推？

索引下推是 MySQL 中的一种索引优化技术，主要用于优化查询过程中对索引的利用效率。当 MySQL 使用二级索引查询数据时，如果查询条件中包含索引列之外的过滤条件，在没有 ICP 的情况下，数据库会先通过索引找到所有符合索引条件的记录的主键，然后回表到聚簇索引中获取完整数据，再对这些数据应用剩余的过滤条件。

而开启索引下推后，数据库会在遍历二级索引的过程中，就对索引中包含的字段先进行过滤，直接排除掉不满足条件的记录，只将符合条件的记录进行回表操作。

这样做的好处是减少了回表的次数，降低了 I/O 操作开销，尤其在二级索引包含较多记录

但实际符合完整条件的记录较少时，能显著提升查询性能。

什么情况下方法产生的异常会往上抛而不是自己处理？

当方法产生的异常不需要或不适合在当前方法内处理时，就会选择往上抛而不是自己处理。具体来说，常见情况包括：如果当前方法无法合理处理该异常，比如缺乏必要的上下文或资源去解决问题，就会将异常抛给调用者，让上层根据具体场景处理；另外，当需要让多个调用层级共同参与异常处理时，当前方法也会选择抛出异常，由更上层的代码统一决策处理方式；还有一种情况是，某些异常属于业务流程中的预期情况，当前方法无需干预，只需传递给上层让其根据业务逻辑判断如何处理。这种向上抛异常的方式，能让异常在合适的层级得到妥善处理，避免在不恰当的地方强行处理导致逻辑混乱，也符合异常处理的分层职责原则。

Throw 和 throws 的区别？

throw 是一个语句，用于在方法内部主动抛出具体的异常对象。比如当检测到不合理的参数时，可以用 throw 抛出一个 IllegalArgumentException，它的作用是明确触发某个异常。而 throws 是方法声明的一部分，用于声明该方法可能会抛出的异常类型。它的作用是告知调用者这个方法存在哪些潜在的异常，需要调用者进行处理。比如一个方法可能会因 IO 操作出错而抛出 IOException，就可以在方法声明后加上 throws IOException。

Redis

讲讲 redis 的分布式锁？

Redis 分布式锁的核心原理基于 Redis 的原子操作。Redis 是单线程执行命令的，这保证了单个命令执行的原子性。当多个客户端尝试获取同一把锁时，通过在 Redis 中设置一个具有特定键名的键值对来表示锁的状态。如果设置成功，则表示获取到了锁；如果设置失败，则表示锁已被其他客户端持有。

常见实现方式有

使用 SETNX (Set if Not Exists) 命令，SETNX 是 Redis 的一个原子命令，用于在键不存在时设置键的值。如果键已经存在，SETNX 命令将不做任何操作并返回 0；如果键不存在，则设置键的值并返回 1。

使用 SET 命令，Redis 2.6.12 版本及以后，SET 命令支持了 NX 和 EX 选项，可以原子性地设置键的值并设置过期时间。这样即使持有锁的客户端出现异常，锁也会在过期时间后自动释放。

存在的问题有

锁过期问题

如果业务逻辑的执行时间超过了锁的过期时间，可能会导致多个客户端同时持有锁，从而引发并发问题。

解决方案是可以使用锁续约机制，在业务逻辑执行过程中，启动一个定时任务，定期检查锁是否还存在，如果存在则延长锁的过期时间。

误释放锁问题

如果客户端 A 获取到锁后，由于某些原因执行时间过长，锁过期自动释放，此时客户端 B 获取到了锁。而客户端 A 在执行完业务逻辑后释放锁，会错误地释放客户端 B 的锁。

解决方案是在释放锁时，需要先检查锁的值是否是自己设置的那个值，如果是则释放，否则不释放。可以使用 Lua 脚本来保证检查和删除操作的原子性。

集群环境下的锁问题

在 Redis 集群环境中，由于数据可能分布在不同的节点上，使用单个 Redis 节点实现的分布式锁可能会出现锁丢失问题，如主从切换时锁丢失。

解决方案是可以使用 Redlock 算法，该算法需要在多个独立的 Redis 节点上获取锁，只有当在大多数节点上都成功获取到锁时，才认为获取锁成功。不过 Redlock 算法也存在一些争议，使用时需要根据具体场景权衡。

本地缓存和 redis 缓存分别有什么好处？

本地缓存是应用程序进程内的缓存，它的好处主要在于访问速度极快，因为数据直接存储在内存中，不需要经过网络传输，能最大程度减少数据获取的延迟，适合存储频繁访问且变动不频繁的数据，比如系统配置、常量等。不过它的缓存范围仅限于当前应用实例，多个实例间无法共享缓存数据，且受限于应用所在服务器的内存大小。

Redis 缓存是独立的分布式缓存服务，它的优势在于支持多应用实例共享缓存数据，能有效减少分布式系统中数据不一致的问题，比如在集群部署的应用中，不同节点可以通过 Redis 获取相同的缓存数据。同时，Redis 提供了丰富的数据结构和持久化机制，能应对更复杂的缓存场景，比如实现分布式锁、计数器等，而且它的内存管理更灵活，可独立扩展内存容量，适合存储需要跨实例共享或有较高持久化需求的数据。但相比本地缓存，Redis 需要通过网络通信，存在一定的网络延迟。

Redis 中分布式锁如何进行加锁和解锁，特殊键值对有什么要求吗？

Redis 实现分布式锁的核心思路是利用其单线程特性和原子操作，通过设置特定键值对来标识锁的占用状态。

加锁时，通常使用 SET 命令并带上 NX 和 EX/PX 参数，NX 确保只有一个客户端能成功设置键，实现互斥。Ex/pX 则是防止客户端异常崩溃后锁无法释放。

解锁需要两步操作，先判断当前锁的 value 是否与自己持有的一致，一致才删除。为保证原子性，通常用 Lua 脚本实现，例如先通过 GET lock_key 获取值，对比一致后执行 DEL lock_key。直接使用 DEL 命令可能删除其他客户端持有的锁，而 Lua 脚本能确保判断和删除的原子性。特殊键值对的要求主要有三点，键名需全局唯一，避免不同业务锁冲突。value 需具备唯一性，用于验证锁的归属。必须设置合理的过期时间，防止死锁，同时过期时间要大于业务执行时间，避免锁提前释放导致并发问题。

Redis 线程模型了解吗？

Redis 的线程模型是单线程加多路 I/O 复用的模式。Redis 服务器在处理客户端请求时，核心的网络事件处理和数据操作由一个主线程完成，这避免了多线程带来的线程安全问题和上下文切换开销。

为了高效处理大量并发连接，Redis 采用 I/O 多路复用技术，通过 select、epoll 等机制同时监听多个套接字的读写事件。当有事件就绪时，主线程会按照一定顺序依次处理这些事件，执行相应的命令操作。

这种模型的优势在于，对于内存中的数据操作，单线程可以高效处理，而 I/O 多路复用机制又能让 Redis 同时应对大量客户端连接，不会因为 I/O 阻塞而影响整体性能。不过，这也意味着如果有某个命令执行时间过长，会阻塞整个主线程，影响其他请求的处理。因此，在使用 Redis 时，应避免执行耗时过长的命令，确保主线程能快速处理各类请求。

Redis 的哈希怎么扩容？

Redis 底层使用哈希表存储数据，为了平衡性能，会维护两个哈希表。当哈希表的负载因子超过设定阈值时会触发扩容。

扩容时，Redis 会为表 2 分配更大的空间，然后将表 1 中的数据逐步迁移到表 2 中。这个迁移过程不是一次性完成的，而是在后续的读写操作中分批进行，避免了一次性迁移带来的性能波动。

在迁移期间，新的写入操作会直接在表 2 上进行，而查询、删除等操作则会同时在这两个哈希表上执行，确保数据访问的正确性。当所有数据迁移完成后，表 1 会被清空，表 2 成为新的表 1，为下一次扩容或缩容做准备。

什么是 RedLock？

RedLock 是 Redis 官方提出的一种用于解决分布式锁单点故障问题的算法，主要目标是在多个 Redis 节点环境下，实现更可靠的分布式锁，避免因单个 Redis 节点宕机导致锁失效或出现锁竞争、锁丢失等问题。

传统的单节点 Redis 分布式锁，依赖单个主节点存储锁信息，但如果这个主节点突然宕机，且锁信息还没同步到从节点，就可能出现多个客户端同时获取到无效锁的情况，引发业务数据不一致。而 RedLock 正是通过多节点投票的机制，解决了这种单点风险。

核心逻辑可以拆解为三个关键步骤。

首先是获取锁的投票阶段，客户端会向预先配置的 N 个独立 Redis 主节点同时发起“获取锁”的请求。这里的关键是，客户端会为每个节点的请求设置一个较短的超时时间，避免因某个节点宕机导致客户端一直阻塞。

然后判断锁是否获取成功，当客户端收到超过半数节点的锁获取成功响应后，才认为整体锁获取成功。比如 N=5 时，至少需要 3 个节点返回成功。同时，客户端会计算一个实际锁有效时间，确保后续业务操作能在这个有效时间内完成。

最后是释放锁的阶段，客户端完成业务操作后，会向所有 N 个节点发起释放锁的请求，无论之前某个节点是否成功授予锁，都要释放，避免残留的锁影响后续请求。

Redis 中什么是跳表，做了什么优化？

Redis 中的跳表是一种有序数据结构，主要用于实现 ZSet 这种核心数据类型。它的本质是在普通有序链表的基础上，通过建立多层“索引”来优化查询效率。普通有序链表要找一个元素，得从表头逐个遍历，时间复杂度是 $O(n)$ ，而跳表通过多层索引的“跳跃式”查找，能把查询时间复杂度降到 $O(\log n)$ ，和平衡二叉树类似，但实现起来更简单、维护成本更低。优化有几点，第一是动态层数设计，避免索引层级固定导致的效率问题。跳表每个节点的层数不是固定的，而是通过“随机算法”决定的。这种随机层数能保证索引的“稀疏度”合理，上层索引节点少、跳跃跨度大，下层索引节点多、定位更精确，整体形成类似“金字塔”的结构，既不会因为索引太密浪费空间，也不会因为索引太疏影响查询效率。

第二是支持范围查询和有序操作，这是跳表相比哈希表的核心优势。因为跳表本身是有序的，跳表可以直接通过索引定位到范围的起始节点，然后顺着底层链表依次遍历获取元素，不用像哈希表那样先拿到所有元素再排序，时间复杂度能稳定在 $O(\log n + k)$ ，非常适合 ZSet 的业务场景。

第三是插入和删除的高效维护，相比平衡二叉树更简单。平衡二叉树在插入删除时需要通过旋转来维持平衡，逻辑复杂且容易出错；而跳表插入删除时，只需要先通过索引找到待操作节点的位置，然后修改对应层级的指针，不需要复杂的平衡调整。虽然理论上时间复杂度和

红黑树都是 $O(\log n)$ ，但跳表的实现代码更简洁，在 Redis 这种需要兼顾性能和代码可维护性的场景下，是更优的选择。

第四是节点存储额外信息，适配 ZSet 需求。Redis 跳表的每个节点不仅存储元素值，还存储对应的分数，并且支持相同分数元素按字典序排序，在节点中会额外记录字典序相关的信息，确保即使分数相同，元素也能保持有序。

Zset 是如何实现的？

靠哈希表和跳跃表两种结构实现。

从功能上看，Zset 既要存储元素的值和对应的分数，又要支持高效的按分数范围查询、按排名操作等功能。其底层实现中，哈希表用于存储元素值与分数的映射关系，这样可以实现 $O(1)$ 时间复杂度的元素分数查询、更新等操作。

而跳跃表则负责维护元素的有序性，通过多层索引结构，能够在平均 $O(\log N)$ 的时间复杂度内完成元素的插入、删除以及范围查询等操作。跳跃表中的每个节点都包含元素的分数和值，并且按照分数进行排序。

这种组合实现让 Zset 既具备了哈希表的快速查询特性，又拥有了跳跃表的高效有序操作能力，从而满足了各种场景下对有序集合的操作需求，比如排行榜、带权重的任务调度等场景。

redis 有哪些数据结构？

字符串，Redis 中最基本的数据结构，一个键对应一个字符串值，它可以存储多种类型的数据，如文本、整数、二进制数据等，并且最大能存储 512MB 的数据。

哈希，一个键值对的集合，类似于 Python 中的字典。它适合存储对象，其中每个字段都有一个关联的值，并且可以独立地对这些字段进行操作。

列表，一个有序的字符串元素集合，它基于双向链表实现，这意味着可以在列表的两端快速地进行插入和删除操作。

集合，一个无序且唯一的字符串元素集合，不允许有重复的成员。它基于哈希表实现，所以添加、删除和查找元素的时间复杂度都是 $O(1)$ 。

有序集合，一种特殊的集合，它在集合的基础上为每个元素关联了一个分数，元素按照分数从小到大排序。如果分数相同，则按照元素的字典序排序。它使用跳跃表和哈希表来实现，保证了插入、删除和查找操作的时间复杂度为 $O(\log N)$ 。

位图，不是一种实际的数据类型，而是基于字符串类型实现的一种位操作方式。它可以将字符串看作是一个由二进制位组成的数组，每个位可以存储 0 或 1。（用户签到）

HyperLogLog，一种用于估计集合中唯一元素数量的概率性数据结构。它只需要使用固定大小的内存，就可以对非常大的数据集进行基数估计，误差率在 0.81% 左右。

地理空间索引 Geo，用于存储地理位置信息，并支持对这些位置进行各种地理空间查询，如范围查询、距离计算等。它基于有序集合实现，将地理位置的经纬度编码为一个有序集合的分数。

redis 速度快的原因？////////////////////

Redis 将所有数据都存储在内存中，内存的读写速度远远快于硬盘等传统存储介质。

采用了多种高效的数据结构，如字符串、哈希、列表、集合、有序集合等，能够快速地进行数据的插入、删除、查找和更新操作。

且使用单线程来处理所有的客户端请求，避免了多线程环境下的线程切换和锁竞争开销。

同时采用了非阻塞的 I/O 模型，使用事件驱动机制来处理多个客户端的连接和请求。它通过多路复用技术可以同时监听多个套接字的事件，当有事件发生时才进行处理，能够同时处理大量的客户端请求。

以及网络通信架构经过了精心优化，它采用了简洁的请求 - 响应协议，减少了网络传输的开销和复杂性。

同时，Redis 还支持管道功能，客户端可以将多个命令一次性发送给服务器，服务器依次执行这些命令并将结果一次性返回给客户端，这样可以减少网络往返次数，提高数据传输效率。

讲讲 redis 的 IO 多路复用是怎样的？

Redis 使用 I/O 多路复用函数来监控多个文件描述符的状态变化，这些文件描述符代表了与客户端的连接、网络套接字或其他 I/O 资源。

服务器进程运行在一个事件循环中，在每次循环迭代时，会调用 I/O 多路复用函数，该函数会阻塞等待，直到有一个或多个文件描述符上有事件发生。当有事件发生时，函数返回，Redis 会根据发生的事件类型来执行相应的处理逻辑，例如读取客户端发送的命令、向客户端发送响应数据等。

通过 IO 多路复用，Redis 可以用一个线程处理多个客户端连接的 I/O 操作，而无需为每个连接创建一个单独的线程。这大大提高了服务器的并发处理能力，能够同时处理大量的客户端请求，节省了大量的系统资源，如内存和 CPU 时间。

以及由于 Redis 能够及时响应发生在各个文件描述符上的事件，不会因为某个连接的阻塞而影响其他连接的处理，所以可以实现低延迟的 I/O 操作。

讲讲 redis 的事务？

Redis 事务允许用户在一个步骤中执行多个命令。通过将多个命令组合成一个事务，用户可以确保这些命令作为一个不可分割的单元被执行，避免了在执行过程中被其他客户端的操作干扰，从而实现对数据的一致性操作。

主要通过几个命令来实现

MULTI，用于开启一个事务，之后输入的命令不会立即执行，而是会被放入一个队列中。

EXEC，执行事务队列中的所有命令。当执行 EXEC 命令时，Redis 会按照命令入队的顺序依次执行队列中的所有命令，并将执行结果返回给客户端。

DISCARD，取消当前事务，清空事务队列。

WATCH，用于监视一个或多个键。如果在事务执行之前，被监视的键被其他客户端修改了，那么整个事务将会被取消，EXEC 命令将返回 nil。

有几个特性

原子性，在 Redis 事务中，要么所有命令都执行，要么所有命令都不执行。当执行 EXEC 命令时，Redis 会将事务队列中的所有命令一次性执行，在执行过程中不会被其他客户端的命令打断。

顺序性，事务队列中的命令会按照入队的顺序依次执行，保证了操作的顺序性。

局限性在于

不支持回滚，与传统数据库事务不同，Redis 事务不支持回滚。如果事务中的某个命令执行失败，Redis 不会回滚之前已经执行的命令，也不会停止执行后续的命令。

以及在 Redis 集群环境下，由于数据分布在不同的节点上，事务的原子性可能无法得到保证。

讲讲 redis 的持久化策略？

Redis 的持久化策略有 RDB 和 AOF 以及混合持久化。

其中 RDB 持久化是将 Redis 在某个时间点上的内存数据快照保存到磁盘上的一个二进制文件。可以通过手动执行 SAVE 或 BGSAVE 命令，或者配置定期执行快照操作来触发 RDB 持久化。

而 AOF 持久化是将 Redis 执行的所有写命令追加到一个日志文件中。当 Redis 重启时，会重新执行 AOF 文件中的所有命令来恢复数据。随着 Redis 不断执行写命令，AOF 文件会越来越大，为了避免 AOF 文件过大，Redis 提供了 AOF 重写机制。AOF 重写会创建一个新的 AOF 文件，将当前内存中的数据以最小的命令集合重新写入新文件，从而减小 AOF 文件的大小。可以通过手动执行 BGREWRITEAOF 命令或配置自动重写来触发 AOF 重写。

在 Redis 4.0 中引入了混合持久化机制，结合了 RDB 和 AOF 的优点。在进行 AOF 重写时，Redis 会将当前内存中的数据以 RDB 格式写入 AOF 文件的开头，然后再将重写后的增量写命令以 AOF 格式追加到文件末尾。

其中 RDB 文件体积小，恢复速度快，对性能影响小，但数据安全性低，而 AOF 数据安全性高，日志文件可读性强，但文件体积大且恢复速度慢，混合持久化先加载 RDB 部分，再执行 AOF 部分的命令，既保证了数据恢复速度，又提高了数据安全性。

这是 Redis 4.0 后的神作，也是现在的主流配置。

做法：在 AOF 重写时，先把内存里的数据以 RDB 的格式（二进制）写在文件开头，后续产生的写命令再以 AOF 的格式（文本）追加在后面。

恢复过程：

先加载开头的 RDB 部分（飞快）。

再重放结尾的一点点 AOF 命令（补齐增量数据）。

结果：既拥有了 RDB 的加载速度，又拥有了 AOF 的数据安全性

AOF 重写 (Rewrite)：如果一个计数器加了 100 万次，账本里就有 100 万行 INCR。重写机制会扫描内存，直接在冷启动文件里写一句 SET counter 1000000。
瘦身成功！

请说说 redis 的主从复制原理？

主从复制的工作流程先是从节点启动，当从节点启动时，它会根据配置信息向主节点发送连接请求，请求建立复制连接。

如果主节点设置了密码，从节点在连接时需要提供正确的密码进行身份验证。

请求通过后从节点与主节点之间建立一个 TCP 连接，用于后续的数据同步和命令传输。

然后从节点向主节点发送 PSYNC /pi: sIrk/命令，请求进行数据同步。该命令会携带从节点当前的复制偏移量和主节点的运行 ID。

主节点接收到 PSYNC 命令后，会根据从节点提供的运行 ID 和复制偏移量判断是否可以增量同步。如果从节点提供的运行 ID 和主节点运行 ID 一致且复制偏移差距较大则主节点会开始执行全量同步流程。

主节点执行 BGSAVE 命令，在后台生成一个 RDB 文件，该文件包含了主节点当前的所有数据。

然后将生成的 RDB 文件发送给从节点。在发送 RDB 文件的过程中，主节点会将新接收到的写命令缓存在内存中。

从节点接收到 RDB 文件后，会先清空自己的内存数据，然后将 RDB 文件中的数据加载到内存中。

主节点在 RDB 文件发送完成后，会将缓存的写命令发送给从节点，从节点执行这些写命令，以保证数据的一致性。

如果从节点提供的运行 ID 和主节点运行 ID 一致且复制偏移差距较小则主节点会开始执行增量同步流程。

主节点将从节点复制偏移量之后的所有写命令发送给从节点，从节点执行这些写命令，从而实现数据的同步。

在完成同步阶段后，主从节点进入命令传播阶段。在这个阶段，主节点会将新接收到的写命令实时地发送给从节点，从节点接收到这些命令后会执行，以保证主从节点的数据一致性。

同时为了确保主从节点之间的连接正常，Redis 在主从复制过程中引入了心跳机制。

从节点每隔一段时间向主节点发送带有从节点当前偏移量的命令，主节点可以根据这个偏移量判断从节点是否正常接收数据，以及主从节点之间的数据差距。

主节点也会定期向从节点发送 PING 命令，检查从节点连接状态。

你对 redis 的缓存淘汰策略有哪些了解？

有以下几个，从设置了过期时间的键中，移除最近最少使用的键。Redis 会维护一个近似的 LRU 算法，通过对每个键记录一个时间戳，在需要淘汰数据时，优先选择时间戳最旧的键进行删除。

从所有键中，移除最近最少使用的键。无论键是否设置了过期时间，只要它是最久未被使用的，就可能被淘汰。

从设置了过期时间的键中，移除最不经常使用的键。

从所有键中，移除最不经常使用的键。不考虑键是否设置了过期时间，只根据访问频率来决定淘汰哪些数据。

从设置了过期时间的键中，随机移除一个键。

从所有键中，随机移除一个键。无论键是否设置了过期时间，都有相同的概率被随机选中并删除。

从设置了过期时间的键中，移除剩余生存时间最短的键。也就是优先删除那些即将过期的键。以及当内存达到限制时，不进行任何淘汰操作。如果此时有新的数据写入，Redis 会返回错

误，阻止新数据的写入。

redis 的哨兵机制是怎样的？

会先启动哨兵进程，需要在配置文件中指定要监控的 Redis 主节点信息，包括主节点的 IP 地址、端口号以及监控的名称等。启动多个哨兵进程，它们会自动发现彼此并建立通信连接，形成一个哨兵集群。

然后发现主从节点，哨兵进程启动后，会向配置的主节点发送 INFO 命令，获取主节点的信息，同时也会获取该主节点下所有从节点的信息，从而发现整个主从集群的拓扑结构。

有心跳检测，哨兵会定期向主从节点发送 PING 命令，根据节点的响应情况来判断节点是否正常。默认情况下，每隔 1 秒就会向所有被监控的节点发送 PING 命令。

如果某个节点在一定时间内没有响应 PING 命令，哨兵会将该节点标记为“主观下线”。这只是单个哨兵的判断，可能存在误判。

当一个哨兵将主节点标记为“主观下线”后，它会向其他哨兵发送询问，了解它们对该主节点的状态判断。如果超过一定数量的哨兵都认为该主节点“主观下线”，那么该主节点就会被标记为“客观下线”。这表明主节点确实出现了故障。

当主节点被标记为“客观下线”后，哨兵集群需要选举出一个领导者哨兵，由它来负责后续的故障转移操作。选举过程基于 Raft 算法，通过投票机制选出一个领导者。

领导者哨兵会从从节点中选择一个合适的节点作为新的主节点。选择的规则通常会考虑从节点的优先级、复制偏移量等因素。

然后向选中的从节点发送 SLAVEOF NO ONE 命令，将其提升为新的主节点。

再向其他从节点发送 SLAVEOF 命令，让它们将新的主节点作为复制源，建立新的复制关系。通过发布 - 订阅机制通知客户端主节点已经发生变更，客户端可以根据新的主节点信息进行连接。

说说 redis 的缓存穿透、缓存击穿、缓存雪崩是怎样的？

缓存穿透是指客户端请求的数据在缓存中不存在，并且在数据库中也不存在，导致请求直接穿透缓存到达数据库，给数据库带来压力。

可以使用布隆过滤器，它能快速判断一个元素是否在集合中。在缓存之前先使用布隆过滤器过滤掉不存在的键，避免无效请求到达数据库。另外，也可以对查询结果为空的情况进行缓存，设置一个较短的过期时间，这样下次相同请求就可以直接从缓存中获取空结果，而不会穿透到数据库。

缓存击穿是指缓存中某个热点数据过期的瞬间，大量并发请求同时访问该数据，由于缓存中没有该数据，这些请求会同时穿透到数据库，导致数据库压力瞬间增大。

、可以采用互斥锁的方式，在缓存数据过期时，只允许一个请求去数据库查询数据并更新缓存，其他请求则等待该请求完成后从缓存中获取数据。另外，也可以考虑延长热点数据的过期时间，或者采用“永不过期”的策略，通过定期异步更新数据来保证数据的新鲜度。

缓存雪崩是指在同一时刻，大量的缓存数据同时过期或者 Redis 服务器出现故障，导致大量请求直接涌向数据库，使数据库压力剧增，甚至可能导致数据库崩溃。

对于过期时间集中的问题，可以将缓存数据的过期时间设置为随机值，避免大量数据同时过期。同时，可以采用多级缓存架构，如在应用服务器本地设置一级缓存，再加上 Redis 作为二级缓存，当 Redis 中的数据失效时，先从本地缓存中获取数据，减轻数据库的压力。另外，为了应对 Redis 服务器故障，可以搭建 Redis 集群，实现数据的冗余和高可用性。

有看过 redis 的底层源码吗？

看过哈希类型键值对的实现，采用了类似字典的哈希表结构，有哈希表节点，用于存放键值对，其中包含键、值以及指向下一个节点的指针，

有哈希表，一个二维数组，由多个哈希表节点组成，还包含了大小、掩码和当前节点数量等元数据，

有字典，对两个哈希表进行管理，在再哈希期间会交替使用这两个哈希表。

当出现哈希冲突时，运用链地址法来处理哈希冲突，也就是当多个键映射到相同槽位时，会通过链表将这些节点串联起来。为了避免链表过长影响查询效率，Redis 在必要时会进行再哈希操作，以此保证哈希表的负载因子处于合理范围。

同时当哈希表需要扩容或者缩容时，不会一次性完成所有键的再哈希，而是采用渐进式的方式，在每次对字典进行增删改查操作时，顺带迁移少量的键到新的哈希表，在再哈希过程中，新旧两个哈希表会同时存在，查询操作会先在旧表中进行查找，如果找不到再到新表中查找。还会在空闲时执行后台再哈希任务，加快迁移的速度。

JUC

讲讲线程池参数如何设置？

线程池有多个参数的设置需要考虑。

核心线程数，设置时要考虑任务的性质和预期的并发量。如果任务是 CPU 密集型的，通常将核心线程数设置为等于或略大于 CPU 核心数，以充分利用 CPU 资源，避免过多线程竞争导致上下文切换开销增加。如果任务是 I/O 密集型的，由于线程在等待 I/O 操作完成时会处于空闲状态，此时可以设置核心线程数为 CPU 核心数的数倍，以确保有足够的线程来处理 I/O 任务，提高系统的吞吐量。

最大线程数，它应该大于或等于核心线程数。当任务队列已满且所有核心线程都在忙碌时，线程池会创建新的线程来处理任务，直到达到最大线程数。最大线程数的设置要考虑系统的资源限制，包括内存、CPU 处理能力等。如果设置过大，可能会导致系统资源耗尽，影响系统的稳定性；如果设置过小，可能无法充分利用系统资源来处理高并发任务。

任务队列，有多种类型的队列可供选择，如无界队列和有界队列。无界队列可以无限容纳任务，但可能会导致内存溢出，适用于任务处理速度较快，不会出现大量任务堆积的情况。有界队列则需要设置一个固定的容量，当队列已满时，新的任务会根据线程池的拒绝策略进行处理。设置队列大小时，要根据任务的产生速度和处理速度来权衡，以避免队列过长导致任务延迟过高，或者队列过小导致任务被拒绝。

线程存活时间，多余的线程在空闲一段时间后会销毁，这个空闲时间就是线程存活时间。设置合理的线程存活时间可以避免资源浪费，同时又能保证在有新任务到来时可以快速响应。如果存活时间设置过短，可能会导致线程频繁创建和销毁，增加系统开销；如果设置过长，可能会导致线程长时间闲置，占用系统资源。

拒绝策略，常见的拒绝策略有直接拒绝、丢弃最老的任务、丢弃当前任务和由调用者线程处理等。选择合适的拒绝策略要根据业务需求来决定。例如，对于一些对实时性要求不高的任务，可以选择丢弃最老的任务；对于一些不能丢失的任务，可以选择由调用者线程处理，让调用者自己决定如何处理任务。

线程工厂，用于创建新的线程，它可以统一设置线程的名称、优先级、是否为守护线程等属性。合理设置线程工厂有助于线程的管理和问题排查。例如，通过为不同业务的线程池设置不同的线程名称前缀，可以在日志中快速区分不同来源的线程，便于定位问题。

时间单位，配合线程存活时间使用，指定线程存活时间的单位。

线程池任务怎么知道完成了？

判断线程池任务是否完成，主要有几种常用方式。首先，可以通过线程池的 `isTerminated` 方法，不过这需要先调用 `shutdown` 方法关闭线程池，当所有任务执行完且线程池终止后，该方法会返回 `true`。

其次，对于提交的单个任务，若使用 `submit` 方法提交，会返回一个 `Future` 对象，通过调用 `Future` 的 `isDone` 方法可以判断任务是否完成，`get` 方法则会阻塞等待任务完成并获取结果，也能间接知道任务已结束。

另外，如果是多个任务，可以用 `CountDownLatch` 来协调，初始化时设置与任务数量相同的计数器，每个任务执行完调用 `countDown`，主线程通过 `await` 阻塞等待计数器归零，以此判断所有任务完成。

还有 `ExecutorService` 的 `invokeAll` 方法，它会等待所有提交的任务完成后才返回，返回的 `Future` 列表可用于获取结果，这也能表明所有任务已执行完毕。

线程池中线程的回收细节？

首先，线程池会区分核心线程和非核心线程。核心线程是线程池长期保留的线程，而非核心线程则是在任务量激增时临时创建的。默认情况下，核心线程即使处于空闲状态也不会被回收，而非核心线程在完成的任务后，若空闲时间超过设定的时间，就会被回收。不过现在很多线程池实现支持通过设置，让核心线程也遵循空闲回收机制。

其次，线程的回收过程是被动触发的。当线程执行完一个任务后，会进入循环等待状态，不断从任务队列中获取新任务。如果队列中没有任务，线程会阻塞等待。若等待超时仍没有新任务，非核心线程就会主动退出，被系统回收。核心线程则会继续阻塞等待，直到有新任务到来。

另外，线程池的状态会影响回收行为。当线程池处于 SHUTDOWN 或 TERMINATED 状态时，无论核心线程还是非核心线程，在完成当前任务后都会陆续退出，不会再接收新任务，最终所有线程都会被回收，线程池彻底关闭。这种回收机制的好处是平衡了资源消耗和响应速度：核心线程保留避免了频繁创建线程的开销，非核心线程超时回收则防止资源浪费，让线程池能根据任务量动态调整线程数量。

无锁队列一般怎么做的？

队列内部维护头指针和尾指针，分别指向队首和队尾。当进行入队操作时，线程通过原子操作获取当前尾指针位置，计算新元素的插入位置，再用 CAS 操作尝试更新尾指针。如果 CAS 成功，说明入队成功，否则重试。出队操作类似，通过原子操作获取头指针，确认队列非空后，用 CAS 更新头指针完成元素取出。

线程池创建新线程的时机和规则？

线程池创建新线程的时机和规则主要取决于其核心参数与任务队列的状态。

当有新任务提交时，线程池会先判断当前运行的线程数是否小于核心线程数，如果是，就会直接创建新线程来处理任务，无论是否有空闲线程。

若当前线程数已达到核心线程数，线程池会将任务放入等待队列。当队列未满时，任务会在队列中等待，由空闲的核心线程处理；而当队列已满且当前线程数小于最大线程数时，线程池会创建非核心线程来处理新任务。

只有当队列已满且线程数已达到最大线程数时，新任务才会被拒绝，触发拒绝策略。

这种机制通过核心线程保证基本处理能力，结合队列缓冲和非核心线程应对流量波动，既能避免频繁创建销毁线程的开销，又能在高负载时通过扩展线程数提升处理能力，是一种平衡资源消耗与处理效率的设计。

线程池有哪几种？

固定大小线程池，线程池中的线程数量固定不变，由创建线程池时指定。它适用于处理具有稳定并发量的任务，能有效控制线程数量，避免线程频繁创建和销毁带来的开销。例如，在一个服务器中处理固定数量的并发请求，就可以使用固定大小线程池。

可缓存线程池，线程池中的线程数量会根据**任务数量动态调整**。如果有空闲线程，会优先使用空闲线程处理任务；如果没有空闲线程，会创建新的线程来处理任务。当线程空闲一段时间后，会被销毁。这种线程池适用于处理**突发的、并发量变化较大的任务**，能灵活地适应任务量的变化。例如，在处理用户的搜索请求时，搜索量可能会突然增加或减少，可缓存线程池就可以很好地应对这种情况。

单例线程池，线程池中只有**一个线程**，所有任务按照**提交顺序依次执行**。它保证了任务的顺序执行，适用于需要**按顺序处理任务**，且不希望有并发执行的场景。比如，在处理一些对**数据一致性要求很高的任务时**，如**银行转账**操作，使用单线程线程池可以确保每个转账任务依次执行，避免数据不一致的问题。

定时线程池，用于**定时或周期性执行任务**。可以指定任务在特定的延迟后执行，或者按照一定的时间间隔周期性地执行。例如，在一个定时任务系统中，需要每天凌晨执行**数据备份任务**，就可以使用**定时线程池**来实现。

分治并行线程池，是 **Java 7** 引入的一种专门用于处理**能够被分解成多个子任务，且子任务之间可以并行执行的线程池**，它基于**分治思想**和**工作窃取算法**来提高并行处理效率。核心特点在于其工作窃取算法。每个工作线程都有自己的**双端队列来存储任务**。当一个线程完成了自己队列中的任务后，它会随机选择其他线程的队列，从**队尾窃取一个任务来执行**。这样可以保证线程资源得到充分利用，避免有的线程忙碌而有的线程闲置的情况，从而提高整体的并行处理性能。适用于能够进行**任务分解**和**结果合并**的场景。除了上述的**数组求和**，还常用于对大规模**数据集**进行操作，如**排序、搜索**等。在处理**树结构数据**时，例如遍历树并计算节点属性，也可以将树的不同分支作为子任务，通过分治并行线程池并行处理，以加快处理速度。此外，**在一些需要高效利用多核处理器的计算密集型任务中**，也能发挥很好的作用，充分利用多核优势，提升计算效率。

新的任务被提交到线程池会做什么？

线程池首先会检查自身的状态，判断线程池是否正在运行、是否已关闭或处于其他特殊状态。如果线程池已关闭，那么新任务可能会根据**具体的策略被拒绝执行**，并返回相应的**错误信息**。

然后查看当前已创建的核心线程数是否达到了设定的核心线程数上限。如果核心线程尚未满，线程池会创建一个新的核心线程来执行新提交的任务。

如果核心线程已满，线程池会将任务**放入任务队列中**。任务队列有多种类型，如**无界队列、有界队列**等。如果任务队列是有界队列，且队列已经满了，但当前线程池的线程数没到最大线程数会临时创建线程来处理任务，若当前线程池的线程数达到了最大线程数，那么会根据线程池的拒绝策略来处理新任务。

如果任务队列已满，线程池会检查当前线程数是否达到了最大线程数限制。如果未达到最大线程数，线程池会创建非核心线程来执行新任务。

无论是核心线程还是非核心线程，在获取到任务后，都会执行任务中的具体逻辑。线程会从任务队列中取出任务，然后调用任务的执行方法来完成任务的处理。

任务执行完成后，线程会进行一些清理工作，如释放任务执行过程中占用的资源等。然后线程会回到线程池中，等待下一个任务的分配。如果线程池中的线程长时间没有任务可执行，超过了一定的空闲时间，非核心线程可能会被销毁，以节省系统资源。不同的线程池实现可能会在具体细节上有所差异，但总体的处理流程大致是相似的。线程池通过这种方式来管理线程和任务，实现任务的高效执行和资源的合理利用，提高系统的性能和稳定性。

任务被放在队列里面，有些线程被释放了，队列的任务怎么被线程消费呢？

线程池会定期检查是否有空闲线程。即使有部分线程被释放，但只要线程池未被完全销毁且还有核心线程存在，这些核心线程在完成当前任务后会尝试从队列中获取新的任务进行执行。

同时当线程池中的现有线程无法满足任务处理需求，即所有线程都在忙碌且任务队列不为空时，线程池会根据配置的策略来决定是否创建新的线程。

如果线程池允许创建新线程，那么会创建新的线程来从队列中获取任务并执行。新线程创建后会立即投入到任务队列中获取任务进行处理，以尽快处理积压在队列中的任务。

线程池怎么保证一个任务不会被多个线程重复消费呢？

Java 线程池在设计上就保证了任务调度的原子性。线程池会依次从任务队列里获取任务，且每次只会有一个线程能成功获取任务。一旦某个线程获取到任务，该任务就会从队列中移除，其他线程也就无法再次获取这个任务。

实现则是利用任务队列所具备的线程安全的特性。当一个线程尝试从队列中取出任务时，队列内部的同步机制会保证同一时间只有一个线程能够进行取出操作。像 `LinkedBlockingQueue` 和 `ArrayBlockingQueue` 这些常见的队列，都通过锁机制或者无锁算法来保证操作的原子性，从而避免多个线程同时取出同一个任务。

假如我有这么个场景，我需要开线程，然后并发的去请求下游，然后我的主线程需要去等待开出来的线程的执行结果，然后我只能从这往下走，这个怎么实现？

可以使用 `CompletableFuture`，通过 `CompletableFuture` 类的 `supplyAsync` 方法提交异步任务，然后用 `CompletableFuture` 类的 `allOf` 方法等待所有任务完成，最后获取结果。

在面试中，面试官可能会追问你以下几个细节：

1. ****异常处理怎么办？****

如果“烤串”任务报错了，``allOf`` 会立刻感知到异常。你可以用 ``exceptionally()`` 给每个任务设置一个“保底结果”（比如烤串没了，就返回个面包）。

2. ****为什么不用 ``get()`` 用 ``join()``？****

``get()`` 会强制你写 ``try-catch`` 块（处理 ``InterruptedException``），代码非常难看；``join()`` 抛出的是运行时异常，代码更简洁。

3. ****超时控制**：**

如果麻辣烫老板睡着了，你总不能等一辈子吧？通常会在 ``join()`` 之前给整个大任务加一个超时时间：``allFinished.get(5, TimeUnit.SECONDS)``。

看过线程池的源码吗？

有看过核心实现类 `ThreadPoolExecutor` / `ɪgˈzɛkjətə(r)` /，有几个关键属性，

线程池状态，用于表示线程池当前的运行状态，如运行、关闭、终止等。不同状态决定了线程池对新任务的处理方式。

线程数量，记录线程池里当前线程的数量，包含核心线程和非核心线程。

任务队列，用于存放提交但还未处理的任务。

线程工厂，负责创建新的线程。

拒绝策略，当线程池满且任务队列也满时，决定如何处理新提交的任务。

其中线程池状态和线程数量由一个 `AtomicInteger` 变量来控制。这个变量的高几位表示线程池状态，低几位表示线程数量。通过这种方式，利用一个变量就可以原子性地操作线程池状态和线程数量，避免了并发问题。

以及线程池中的非核心线程在空闲一段时间后会被回收。当线程从任务队列中获取任务时，如果在指定的时间内没有获取到任务，线程会自动退出，从而实现线程的回收。

线程池提交的方式？

有 `execute` 方式，是 `Executor` 接口中的方法，`ThreadPoolExecutor` 实现了该接口，可用于向线程池提交任务。接收一个 `Runnable` 类型的参数，也就是一个实现了 `run` 方法的任务对象，该方法定义了任务要执行的具体操作。

这种方式提交的 **任务没有返回值**，同时当任务执行过程中出现异常时，如果没有在任务内部进行捕获处理，那么这个异常会导致执行该任务的线程终止，但不会影响线程池中的其他线程继续执行其他任务。

以及 `submit` 方式，`ExecutorService` 接口中的方法，`ThreadPoolExecutor` 也实现了该接口。`submit` 方法有多种重载形式，可以接收 `Runnable` 或 `Callable` 类型的参数。

当提交 `Callable` 任务时，该方法会返回一个 `Future` 对象，通过这个 `Future` 对象可以在任务执行完成后获取任务的执行结果。如果提交的是 `Runnable` 任务，也会返回一个 `Future` 对象，不过调用其 `get` 方法时返回的是 `null`，因为 `Runnable` 任务没有返回值。

通过 `Future` 对象的 `get` 方法获取结果时，如果任务执行过程中抛出了异常，`get` 方法会将这个异常重新抛出，需要在调用 `get` 方法的地方进行捕获和处理。

任务被放在队列里面，有些线程被释放了，队列的任务怎么被线程消费呢？

线程池会定期检查是否有空闲线程。即使有部分线程被释放，但只要线程池未被完全销毁且还有核心线程存在，这些核心线程在完成当前任务后会尝试从队列中获取新的任务进行执行。

同时当线程池中的现有线程无法满足任务处理需求，即所有线程都在忙碌且任务队列不为空时，线程池会根据配置的策略来决定是否创建新的线程。

如果线程池允许创建新线程，那么会创建新的线程来从队列中获取任务并执行。新线程创建后会立即投入到任务队列中获取任务进行处理，以尽快处理积压在队列中的任务。

怎么接收线程池的返回值？

可以使用 `submit` 方法提交 `Callable` 任务，`submit` 方法接收一个 `Callable` 类型的参数，并返回一个 `Future` 对象，借助这个 `Future` 对象就能获取任务的返回值。

也可以使用 `ExecutorService` 接口提供的 `invokeAll` 和 `invokeAny` 方法，其中 `invokeAll` 方法接收一个 `Callable` 任务列表，返回一个包含 `Future` 对象的列表，每个 `Future` 对象对应一个任务的执行结果。该方法会等待所有任务完成后才返回。

`invokeAny` 方法则是接收一个 `Callable` 任务列表，返回其中一个已经完成的任务的结果。一旦有一个任务完成，该方法就会立即返回，其他未完成的任务可能会被取消。

如何查看线程池的状态？

可以利用 `ThreadPoolExecutor` 类的方法 `getPoolSize`，这个方法会返回线程池当前的线程数量，它包含正在执行任务的线程以及空闲的线程。

`getActiveCount`，返回正在执行任务的线程数量。

`getQueue().size()`，返回任务队列中等待执行的任务数量。

`isShutdown`，用于判断线程池是否已经调用了 `shutdown` 方法。调用 `shutdown` 方法后，线程池将不再接受新的任务，但会继续执行已经提交的任务。

`isTerminated`，判断线程池是否已经完全终止。当所有任务都执行完毕，并且所有线程都已经销毁时，线程池才会被认为是终止状态。

`getLargestPoolSize`，返回线程池在运行过程中达到的最大线程数。

除了使用 `ThreadPoolExecutor` 类的方法，还可以结合日志来查看线程池的状态。

比如在代码中添加日志记录，记录线程池的关键状态变化，例如线程池的启动、关闭、任务提交和完成等。这样可以帮助跟踪线程池的运行过程，及时发现问题。

提交到线程池的任务执行异常了，如何获取这个异常？

当使用 `execute` 方法提交任务时，为了获取异常信息，可采用自定义 `ThreadFactory`，能为每个线程设置 `UncaughtExceptionHandler`。当线程因未捕获的异常而终止时，`UncaughtExceptionHandler` 就会被调用，进而获取异常信息。

或设置默认的 `UncaughtExceptionHandler`，可以为线程池中的所有线程设置默认的 `UncaughtExceptionHandler`，这样当线程出现未捕获的异常时，就能捕获到异常信息。

使用 `submit` 方法提交任务时，该方法会返回一个 `Future` 对象。可以调用 `Future` 对象的 `get` 方法来获取任务的执行结果。如果任务执行过程中抛出异常，`get` 方法会将该异常封装在 `ExecutionException` 中抛出。通过捕获 `ExecutionException` 并调用其 `getCause` 方法，就能获取到实际的异常信息。

说说 `hashmap` 的底层原理？

JDK 1.7 及之前，`HashMap` 的底层数据结构是数组加链表，JDK 1.8 及之后是数组加链表/红黑树。

数组用于存储键值对的桶，每个桶可以存储一个键值对或一个链表

当链表长度超过一定阈值时，链表会转化为红黑树，以提高查找效率，这个阈值在 jdk1.8 中默认是 8。

以及当不同的键通过哈希函数计算得到相同的桶索引时，就会发生哈希冲突。

JDK 1.7 及之前采用头插法将新的键值对插入到链表头部。

JDK 1.8 及之后采用尾插法插入，并在链表长度超过阈值时转换为红黑树。当红黑树节点数量小于一定阈值时，又会退化为链表。

hashmap 的扩容机制是怎样的？

当 HashMap 中的键值对数量超过了负载因子与当前数组容量的乘积时，就会触发扩容。默认情况下，负载因子为 0.75，即当 $size > capacity * 0.75$ 时，进行扩容。先创建一个新的数组，其容量是原来数组容量的两倍。

再将原数组中的所有键值对重新分配到新的数组中。这是通过重新计算每个键的哈希值，并根据新的数组容量来确定其在新数组中的位置实现的。

在将键值对重新分配到新数组的过程中，需要重新计算键的哈希值和在新数组中的索引位置。由于数组容量发生了变化，原来的哈希值与数组长度的取模结果也会改变。

重新哈希并非简单地重新计算哈希码和取模，JDK 1.8 中对重新哈希的过程进行了优化。利用了扩容后数组长度是原来的 2 倍这一特性，通过位运算来高效地确定键值对在新数组中的位置，减少了计算量。

通过扩容机制，HashMap 能够在元素数量增加时，保持较低的哈希冲突概率，从而维持较好的性能。但扩容操作也会带来一定的性能开销，因为需要重新分配内存空间和重新哈希所有的键值对，所以在使用 HashMap 时，若能提前预估数据量大小，合理设置初始容量，可减少扩容次数，提高性能。

多线程情况下进行 hashmap 的 resize（扩容）会有什么问题？

多线程环境下对 HashMap 进行 resize 操作可能会导致死循环和数据丢失问题，这也是 HashMap 不适合多线程场景的重要原因。

HashMap 的 resize 过程会涉及到链表的重新哈希和迁移。在 JDK 1.7 及之前，HashMap 采用头插法迁移链表节点，当多个线程同时检测到需要扩容并同时执行 resize 时，线程间的操作可能相互干扰，导致链表形成环形结构。这会使得后续对 HashMap 的 get 操作陷入无限循环，引发程序卡死。

此外，多线程并发修改时，还可能出现节点丢失的情况。比如两个线程同时处理同一个链表，迁移节点时可能会覆盖对方的操作，导致部分节点没有被正确转移到新数组中，最终造成数据丢失。

JDK 1.8 虽然将头插法改为尾插法避免了死循环，但仍然没有解决多线程安全问题，并发修改时仍可能出现数据不一致。因此，多线程场景下应使用 ConcurrentHashMap 等线程安全的容器，而非 HashMap。

负载因子为什么要设置为 0.75？

HashMap 将默认负载因子设置为 0.75，主要是时间和空间成本的平衡结果。

负载因子是哈希表中已使用容量与总容量的比值。如果负载因子过高，意味着哈希表快满时才扩容，此时链表或红黑树会变得很长，查询效率会下降到接近 $O(n)$ 。而 0.75 的值能在一定程度上减少键的哈希碰撞，保证查询操作的平均时间复杂度接近 $O(1)$ 。

如果负载因子过低，哈希表会频繁扩容，虽然查询速度快，但会浪费大量内存空间。HashMap 的容量默认是 2 的幂，0.75 与 2 的幂相乘的结果通常是整数，便于计算扩容阈值，避免浮点运算带来的精度问题。

因此 0.75 既能保证哈希表的查询性能，又不会过度浪费内存，是时间效率和空间效率的最优平衡。

什么是红黑树？

红黑树是一种自平衡的二叉搜索树，它通过在每个节点上增加一个红色或黑色的颜色属性来维持树的平衡，从而保证插入、删除和查找操作的时间复杂度始终为 $O(\log n)$ 。

根节点必须是黑色，所有叶子节点都是黑色，如果一个节点是红色，则两个子节点必须是黑色。从任意节点到其所有叶子节点的路径中，黑色节点的数量相同。当插入或删除节点破坏上述规则时，红黑树会通过旋转调整节点的位置关系和改变节点的颜色来恢复平衡。

线程安全的 map 结构有哪些？

首先是 Java 中的 Hashtable，通过在所有方法上添加 synchronized 关键字实现线程安全，任何线程操作它时都需要获取整个对象的锁，虽然安全但并发效率较低。

其次是 Collections.synchronizedMap 方法返回的同步 Map，通过包装普通 Map 并使用同步代码块实现线程安全，本质上也是对整个 Map 加锁，与 Hashtable 类似，适合并发量不高的场景。

再就是 ConcurrentHashMap，juc 包中的高效线程安全 Map。在 JDK 1.7 中采用分段锁机制，将 Map 分成多个段，每个段独立加锁，多线程操作不同段时可以并行执行，大幅提升了并发性能；JDK 1.8 及以上则改用 CAS 操作和 synchronized 关键字优化，进一步提高了效率，是高并发场景下的首选。

说说 ConcurrentHashmap 的底层原理？

JDK 1.8 及之后 ConcurrentHashMap 的底层数据结构与 HashMap 类似，也是数组加链表 / 红黑树。它由多个 Node 数组组成，每个 Node 数组称为一个桶。

当桶中的元素以链表形式存储时，链表长度超过一定阈值时会转换为红黑树，以提高查找效率。

在 JDK 1.7 中，ConcurrentHashMap 采用了分段锁的机制来实现线程安全。它将整个哈希表分为多个段，每个段都是一个独立的哈希表，有自己的锁。

不同段之间的操作可以并发执行，只有在访问同一个段时才需要获取相应段的锁。这样可以提高并发度，允许多个线程同时访问不同段的数据。

JDK 1.8 对 ConcurrentHashMap 的实现进行了优化，不再使用分段锁，而是采用 CAS 和 synchronized 关键字相结合的方式保证线程安全。

在插入和更新操作时，首先使用 CAS 尝试直接在数组中插入或更新元素，如果 CAS 操作失败，说明有其他线程正在同时修改该位置的元素，此时再使用 synchronized 关键字对该桶进行加锁，进行后续的插入或更新操作。

对于读取操作，由于 Node 节点中的数据通过 `volatile` 关键字修饰，保证了数据的可见性，所以不需要加锁就能获取到最新的数据。

`ConcurrentHashMap` 的扩容机制与 `HashMap` 类似，但在多线程环境下更为复杂。当元素数量超过负载因子与当前数组容量的乘积时，会触发扩容。

扩容过程中，会将原数组中的元素迁移到新的更大的数组中。在多线程环境下，多个线程可以同时参与扩容操作，每个线程负责一部分桶的迁移工作。通过 `transfer` 方法来实现元素的迁移，该方法会对每个桶进行处理，将其中的元素重新计算哈希值并放入新数组的相应位置。

说说 `reentrantlock`?

`ReentrantLock` 是 `juc` 包中的一个可重入互斥锁，同一线程可以多次获取同一把锁而不会发生死锁。获取锁的次数会被计数，线程需要释放相同次数才能真正释放锁。需要手动调用 `lock` 方法获取锁，`unlock` 方法释放锁，有非公平锁和公平锁，默认是非公平锁，线程获取锁的顺序不确定，可能存在饥饿现象，但性能更好。

而公平锁需要在构造函数中传 `true`，线程按等待顺序获取锁，避免饥饿，但性能开销较大。允许线程在等待锁时响应中断，避免无限阻塞。

以及 `ReentrantLock` 可通过 `newCondition` 方法生成多个 `Condition` 对象，实现更精细的线程等待/唤醒控制。

为什么可重入锁是可重入的?

因为引入了持有线程标记和重入计数器这两个关键机制，打破了一次锁定只能对应一次释放的简单逻辑，允许同一线程多次获取同一把锁而不会引发死锁。

当线程第一次尝试获取可重入锁时，锁会先判断当前是否处于未被持有状态：如果是，就会将锁的持有线程标记为当前线程，并把重入计数器初始化为 1，此时线程成功获取锁。

而当同一线程再次请求这把锁时，锁会先检查持有线程是否就是当前线程，由于是同一线程，它不会像非可重入锁那样直接阻塞，而是会让重入计数器加 1，以此记录线程重复获取锁的次数。

对应的，在释放锁时，线程每执行一次释放操作，锁的重入计数器就会减 1。只有当计数器减到 0 时，才会清除持有线程标记，将锁恢复到未被持有状态，此时其他线程才能有机会获取这把锁。

`Synchronized` 和 `reentrantlock` 的区别?

Synchronized 锁操作是隐式的，自动获取释放，ReentrantLock 是显式的，需手动获取释放，Synchronized 仅支持非公平锁，reentrantlock 可选择公平和非公平锁，Synchronized 不支持中断响应，而 reentrantlock 支持，Synchronized 获取不到锁时会一直阻塞，ReentrantLock 可以设置超时时间，若超时还未获取锁就会返回 false 结束阻塞，Synchronized 状态无法查询锁的持有等待状态，reentrantlock 可查询当前线程是否持有锁和等待线程数。

synchronized 锁底层为什么是非公平的？

非公平锁可以通过、抢占式尝试、随机唤醒、避免精确排序，最大化减少上下文切换、锁空窗期和内存开销，显著提升吞吐量

然后公平锁的先到先得规则虽能保证顺序，但会引入高昂的性能代价，违背了 synchronized 作为基础锁工具的高效性设计目标；

且底层依赖的管程和操作系统互斥量，本身也为非公平实现提供了天然支持。

讲一下乐观锁是什么，怎么实现？

乐观锁是一种用于处理并发访问的机制，它基于一种乐观的假设，即认为在大多数情况下，并发访问不会导致数据冲突。

乐观锁假设在数据进行更新操作时，不会发生并发冲突。当需要更新数据时，先检查在读取数据后到更新数据前这段时间内，数据是否被其他线程修改过。如果没有被修改过，则执行更新操作；如果被修改过，则根据具体的策略来处理，如重新读取数据并再次尝试更新，或者放弃更新并提示用户。

常见实现方式有版本号机制，在数据库表中添加一个版本号字段。当数据被读取时，同时获取其版本号。在更新数据时，将当前数据的版本号与数据库中存储的版本号进行比较。如果两者相等，说明在读取数据后没有其他线程对该数据进行修改，允许执行更新操作，并将版本号加 1；如果不相等，说明数据已被其他线程修改，更新操作失败。

以及时间戳机制，与版本号机制类似，使用时间戳来记录数据的最后修改时间。在读取数据时记录其时间戳，更新数据时比较当前时间戳与数据库中存储的时间戳。如果当前时间戳大于数据库中的时间戳，说明数据在读取后没有被修改过，允许更新；否则，更新失败。不过时间戳机制可能存在精度问题，而且在分布式系统中，不同节点的时间同步也可能带来问题。

cas 是什么？

CAS 是一种用于实现多线程同步的原子操作。在并发编程中，CAS 操作是一种无锁算法，常用于解决多线程环境下的数据竞争问题，避免使用传统的锁机制带来的性能开销。

CAS 操作包含三个操作数：内存位置 V、预期原值 A 和新值 B。当执行 CAS 操作时，它会先比较内存位置 V 中的值是否等于预期原值 A，如果相等，则将内存位置 V 中的值更新为新值 B；如果不相等，则不进行更新操作。整个比较并交换的过程是原子性的，由硬件层面保证，在执行过程中不会被其他线程中断。

说一下线程池的创建和调用？

创建需要先根据任务特点选择合适的线程池类型。例如，固定大小线程池适用于已知并发任务数量且相对稳定的场景，它创建固定数量的线程，线程会一直存在并重复利用；可缓存线

线程池适合处理大量短时间任务，它会根据任务数量动态创建和回收线程，线程空闲一段时间后会销毁；定时线程池用于定时或周期性执行任务；单例线程池则是只有一个线程的线程池，保证任务按顺序执行。

再设置核心参数，包括核心线程数，即线程池中保持活动的最小线程数量；最大线程数，规定了线程池最多能创建的线程数；还有阻塞队列，用于存放暂时无法执行的任务，常见的有有界队列和无界队列，队列的选择会影响线程池的性能和资源占用；此外，还可以设置线程的存活时间，当线程数超过核心线程数时，多余的线程在空闲多长时间后会被销毁。

调用则是先将任务提交给线程池，线程池会按照一定的策略来执行提交的任务。当有任务提交时，线程池首先会检查核心线程是否都在执行任务，如果有空闲的核心线程，则将任务分配给它执行；如果核心线程都在忙碌，且阻塞队列未满，则将任务放入队列中等待；如果核心线程都在忙碌且阻塞队列已满，此时会根据最大线程数来判断是否需要创建新的线程来执行任务；如果线程数已经达到最大线程数，那么就会根据拒绝策略来处理新提交的任务，常见的拒绝策略有直接拒绝、丢弃最老的任务、由提交任务的线程直接执行任务等。

最后，当不再需要使用线程池时，应该关闭线程池以释放资源。可以调用 shutdown 方法来平滑地关闭线程池，它会等待所有正在执行的任务完成后再关闭线程池；如果希望立即停止所有任务并关闭线程池，则可以使用 shutdownNow 方法，该方法会尝试中断正在执行的任务，并返回未执行的任务列表。

说说 completablefuture?

使用 completableforFuture 可以在后台线程中异步执行任务，而不会阻塞主线程。这样可以提高程序的响应性和性能，特别是在处理耗时操作时，如网络请求、文件读取或复杂的计算。

有丰富的方法来组合和编排多个异步任务。可以将多个 CompletableFuture 对象链接在一起，以便在一个任务完成后自动触发另一个任务，或者将多个任务并行执行并在它们都完成后进行汇总结果。

提供了方便的方式来处理异步任务的结果和异常。可以使用回调方法来在任务完成时获取结果，并对结果进行进一步的处理。同时，也可以优雅地处理任务执行过程中抛出的异常，而不会导致程序崩溃。

支持非阻塞的编程风格，使得代码更易于阅读和维护。可以使用类似于链式调用的方式来编写异步代码，避免了传统异步编程中常见的回调地狱问题。

Future 和 completablefuture 的区别?

Future 和 CompletableFuture 都是 Java 中用于处理异步任务的接口，它们的主要区别体现在功能丰富度和使用灵活性上。

Future 是 Java 5 引入的接口，它提供了基本的异步任务处理能力，比如通过 get 方法获取任务结果、isDone 方法判断任务是否完成等。但它的功能比较有限，无法直接对多个异

步任务进行组合编排，也不能为任务完成后设置回调函数，必须通过阻塞或轮询的方式获取结果，在处理复杂异步场景时不够灵活。

CompletableFuture 是 Java 8 新增的类，它实现了 Future 接口，同时扩展了更强大的功能。它支持链式调用，可以通过 thenApply、thenAccept 等方法为任务设置回调，实现任务完成后的自动处理，避免了阻塞等待。此外，CompletableFuture 还提供了丰富的静态方法创建异步任务，以及 allOf、anyOf 等方法组合多个异步任务，能轻松应对复杂的异步流程编排场景。

说说 volatile 关键字？

Volatile/'vɒ:lətl/关键字可以保持可见性，在 Java 内存模型中，每个线程都有自己的工作内存，变量的值会被拷贝到工作内存中进行操作，操作完成后再写回主内存。当一个变量被声明为 volatile 时，它会保证对该变量的写操作会立即刷新到主内存中，而读操作会直接从主内存中读取最新的值。

可以禁止指令重排序，为了提高程序的执行效率，编译器和处理器可能会对指令进行重排序。但在某些情况下，指令重排序可能会导致程序出现错误。volatile 关键字可以禁止指令重排序，保证代码的执行顺序与编写顺序一致。

但不保证原子性，因为原子性是指一个操作是不可分割的，要么全部执行，要么全部不执行。volatile 关键字并不能保证对变量的操作是原子的。

如何防止 ABA？

可以使用版本号，给要更新的变量添加一个版本号，每次变量更新时，不仅要比较变量的值，还要比较版本号。只有当变量的值和版本号都与预期相符时，才执行更新操作。这样可以确保在变量值从 A 变为 B 再变回 A 的过程中，版本号会发生变化，从而避免 CAS 操作误判。

也可以使用标记，为变量添加一个标记位，用于表示变量是否被修改过。这个标记位可以是一个布尔值，当变量被修改时，标记位被设置为 true；在进行 CAS 操作时，同时检查变量的值和标记位。如果标记位发生了变化，即使变量的值看起来没有变化，也说明变量在中间被修改过，CAS 操作不会执行。

说说 aqs？

AQS 是 JUC 中的核心组件，是许多同步工具的基础框架。它通过原子状态管理和阻塞队列实现了高效的同步控制。AQS 的设计基于模板方法模式，它定义了一套同步器的通用框架，子类只需重写特定方法即可实现不同的同步语义。AQS 内部维护一个 state 变量，用于表示同步状态。对 state 的操作通过 cas 实现，保证线程安全。当线程获取同步状态失败时，会被封装成节点加入到内部的 FIFO 双向队列中，并阻塞等待。队列中的节点包含线程引用、等待状态、前驱和后继指针。

线程安全是什么？

线程安全是指在多线程环境下，一个程序或代码片段能够正确地执行，并且不会因为多个线程同时访问和修改共享资源而导致数据不一致、程序崩溃或出现其他错误的特性。

当一段代码是线程安全的，意味着无论有多少个线程同时访问它，都能保证其行为符合预期，不会出现数据竞争，多个线程同时访问和修改同一个共享数据，导致数据的最终结果依赖于线程执行的顺序，产生不可预测的结果。

也不会出现资源冲突，多个线程同时访问和使用同一个资源，如文件、网络连接等，可能导致资源被损坏或无法正常使用。

也不会出现状态不一致，由于多线程并发执行，可能导致对象或系统的状态处于不一致的状态，使程序出现错误或异常。

Java 中如何保证线程安全？

可以使用同步代码或同步方法，通过 `synchronized` 关键字来实现，它可以确保在同一时刻只有一个线程能够访问被同步的代码块或方法，从而避免数据冲突和不一致性。

也可以使用 `juc` 包下的 `Lock` 接口及其实现类提供了更灵活的锁机制。可以通过显式地获取和释放锁来控制对共享资源的访问，相比于 `synchronized` 关键字，`Lock` 锁提供了更多的功能，如可重入锁、公平锁等。

还可以使用线程安全的类和容器，如 `ConcurrentHashMap`、`CopyOnWriteArrayList` 等。这些类和容器内部已经进行了线程安全的设计和实现，可以在多线程环境中安全地使用。

以及使用 `juc` 包下的原子类提供了原子操作的原子类，这些原子类可以在多线程环境中对单个变量进行原子性的操作，避免了使用锁带来的性能开销。

以及创建不可变对象，即对象一旦创建就不能被修改。不可变对象在多线程环境中是线程安全的，因为它们的状态不会被改变，从而避免了数据竞争和不一致性问题。

还有使用 `ThreadLocal` 类为每个线程提供独立的变量副本，每个线程只能访问自己的副本，从而避免了线程之间的共享数据冲突。

如果别的线程调用另一个线程的 `threadlocal` 变量有什么函数可以实现吗？如果没有怎么实现？

`ThreadLocal` 设计的核心是让每个线程只能访问自己线程绑定的变量，其内部通过线程私有 `Map` 存储数据，默认情况下不提供直接让其他线程访问当前线程 `ThreadLocal` 变量的 API。

手动实现可以当前线程将 `ThreadLocal` 中存储的变量值取出，通过方法参数、共享容器等方式传递给其他线程。这是最常见的做法，本质上是传递数据而非直接访问对方的 `ThreadLocal`。

或者利用反射获取目标线程的 `threadLocals` 字段再从中提取对应 `ThreadLocal` 变量的值。但这种方式破坏了 `ThreadLocal` 的线程隔离设计，可能引发线程安全问题。

还可以通过一个全局容器如 `concurrenthashmap` 手动关联线程与变量，键为线程标识，值为需要共享的数据。

知道哪些锁？

`synchronized` 锁是 Java 中最基本的同步机制，可修饰实例方法、静态方法和代码块。当一个线程访问被 `synchronized` 修饰的方法或代码块时，会自动获取锁，其他线程必须等待该线程释放锁后才能进入。

可重入锁，支持可重入性，即同一个线程可以多次获取同一把锁而不会出现死锁。与 `synchronized` 相比，它提供了更灵活的锁机制，如可中断锁、公平锁等。

可重入读写锁，将读写操作分离，允许多个线程同时进行读操作，但在写操作时会独占锁。这样可以提高并发性能，适用于读多写少的场景。

邮戳锁，是 Java 8 引入的一种新的锁机制，它提供了乐观读锁、悲观读锁和写锁三种模式。乐观读锁在读取数据时不会加锁，只有在写入数据时才会检查数据是否被修改，从而提高了并发性能。

公平锁和非公平锁，公平锁是指多个线程按照申请锁的顺序来获取锁，而非公平锁则不保证线程获取锁的顺序，可能会导致某些线程长时间得不到锁。

自旋锁，是一种忙等待的锁机制，当一个线程尝试获取锁时，如果锁已经被其他线程持有，该线程会不断地循环检查锁的状态，而不是进入阻塞状态。自旋锁适用于锁的持有时间较短的场景。

锁的膨胀过程是怎样的？

锁膨胀是指无锁到偏向锁到轻量级锁到重量级锁的一个过程。

首先是无锁，无锁就是没有获得锁的时候。然后是偏向锁，偏向锁就是说一个线程在第一次进入这个代码块或者方法体的时候，若这个锁目前未被获取且偏向锁未被禁止，那么这个线程就会成功获取到这个锁，并且设置偏向锁标识为 1，表明这是个偏向锁的意思，同时会在对象头里的 `markword` 里存储当前线程的 `id`。

后面当有线程再想获得这个锁的时候，分两种情况，一种是重入，是该线程再次想获取这个锁，此时这个线程进入退出代码块或方法体的时候，就不需要再花费 CAS 操作来加锁、解锁了。

另一种是别的线程来获取这个锁，如果之前那个线程不活跃，则是偏向锁的拥有权转移一下，如果还活跃，则会撤销偏向锁，重置偏向锁标识，把锁标志位设置为 00，同时使用 CAS 操作来把对象头的 `Mark Word` 替换为指向锁记录的指针。

替换完之后，就升级为轻量级锁了，此时这个轻量级锁被新线程给拿着。后面当有线程再想获得这个锁的时候，还是会通过 CAS 操作来尝试将对象头的 `Mark Word` 替换为指向锁记录的指针。

如果之前那个线程不活跃了，那这个新线程会 CAS 成功，得到这个轻量级锁；如果之前线程还活跃那 CAS 会失败，但它不会说就不获取了，它是会使用自旋重复的获取这个锁，直达到自旋的次数上限。达到自旋的次数上限后，这个轻量级锁就会膨胀为重量级锁。

而那个一直自旋的线程会阻塞，等待之前线程执行完成后去唤醒自己，然后再去尝试获取这个重量级锁。

ThreadLocal 的原理是什么？

每个 Thread 对象内部都有一个 ThreadLocalMap 类型的成员变量。这个变量就像一个容器，专门用来存放该线程所对应的所有 ThreadLocal 变量及其对应的值。

以及有 ThreadLocalMap 对象，ThreadLocal 的一个静态内部类，类似于 HashMap，但实现细节有差异。采用数组来存储键值对，其中键是 ThreadLocal 对象，值是该 ThreadLocal 为当前线程存储的变量副本。这里的键是弱引用类型，这样做有助于在 ThreadLocal 对象不再被使用时能被垃圾回收，避免内存泄漏。

当线程调用 ThreadLocal 的 set 方法来存储一个值时，会先找到当前线程，然后查看该线程内部的 ThreadLocalMap 是否已经存在。

如果 ThreadLocalMap 已经存在，就把当前 ThreadLocal 对象当作键，要存储的值当作值，一起存到 ThreadLocalMap 里。

如果 ThreadLocalMap 还不存在，就会创建一个新的 ThreadLocalMap，并把当前 ThreadLocal 对象和要存储的值作为第一对键值对存进去，同时将这个新创建的 ThreadLocalMap 关联到当前线程。

当线程调用 ThreadLocal 的 get 方法来获取值时，同样先找到当前线程，接着查看其内部的 ThreadLocalMap。

如果 ThreadLocalMap 存在，就用当前 ThreadLocal 对象作为键去 ThreadLocalMap 里查找对应的值。

如果找到了对应的键值对，就返回存储的值；要是 ThreadLocalMap 不存在或者没找到对应的键值对，就会返回初始值。

当线程调用 ThreadLocal 的 remove 方法时，会从当前线程的 ThreadLocalMap 中把当前的 ThreadLocal 对象对应的键值对移除，释放相关的内存空间。

以及 ThreadLocalMap 中的键使用弱引用，主要是为了避免内存泄漏。当外部程序对 ThreadLocal 对象的强引用被释放后，由于 ThreadLocalMap 里的键是弱引用，在垃圾回收时，这个 ThreadLocal 对象就会被回收（键变为 null）。后续在对 ThreadLocalMap 进行操作时，会检查并清理那些键为 null 的键值对，防止无用数据一直占用内存。

说说 synchronized 底层的机制？

synchronized 底层机制主要基于 Java 对象头和监视器实现，Java 每个对象都有一个对象头，对象头中包含了一些与对象相关的元数据信息。

每个 Java 对象都可以关联一个监视器，当一个线程想要访问被 synchronized 修饰的代码块或方法时，需要先获取该对象的监视器。

当一个线程进入 synchronized 代码块或方法时，会尝试获取该对象的监视器。

如果监视器没有被其他线程持有，当前线程会成功获取监视器，并将其计数器加 1，表示该线程已经持有了锁。

如果当前线程已经持有了该对象的监视器，再次进入 synchronized 代码块或方法时，会将监视器的计数器再次加 1，即 synchronized 的可重入性。

当线程执行完 synchronized 代码块或方法后，会将监视器的计数器减 1。当计数器减为 0 时，表示该线程已经释放了锁，其他线程可以竞争获取该对象的监视器。

synchronized 关键字在编译后会在同步块的前后分别插入 monitorenter 和 monitorexit 字节码指令。monitorenter 指令会尝试获取对象的监视器，monitorexit 指令会释放对象的监视器。

JVM 在执行这些字节码指令时，会根据对象头中的锁状态信息来决定是使用偏向锁、轻量级锁还是重量级锁。

公平锁和非公平锁是怎么实现的？

两者都基于 AQS，主要差异体现在锁获取的逻辑上。公平锁在尝试获取锁时，会先检查等待队列中是否有线程排在前面，如果有则当前线程进入队列等待。而非公平锁在尝试获取锁时，会直接尝试抢占锁，只有抢占失败后才会进入等待队列。

HashMap 为什么要用红黑树？

因为链表在极端情况下存在性能问题，在 JDK 1.8 之前，HashMap 内部是数组加链表的结构。当大量元素通过哈希函数计算后映射到数组的同一个位置时，这些元素会在该位置形成一个链表。而链表的查找操作需要从头节点开始逐个遍历，时间复杂度是 $O(n)$ ，其中 n 是链表的长度。如果链表过长，查找效率会变得非常低。

同样，链表的插入和删除操作在需要查找特定节点时，也会受到遍历链表的时间开销影响，尤其是在链表较长时，性能会显著下降。

而红黑树是一种自平衡的二叉搜索树，它通过对节点进行着色以及特定的插入和删除操作来保证树的高度相对平衡。这意味着红黑树的左右子树的高度差不会过大，避免了普通二叉搜索树在极端情况下退化为链表的问题。

且查找、插入和删除操作效率高，由于红黑树的自平衡特性，其查找、插入和删除操作的时间复杂度都是 $O(\log n)$ ，其中 n 是树中节点的数量。相比于链表的 $O(n)$ 复杂度，红黑树在处理大量数据时性能优势明显。

而且在实际应用中，哈希冲突的分布可能是不均匀的。红黑树的引入使得 HashMap 能够更好地应对这种不均匀的哈希冲突，即使在某些位置出现大量冲突，也能保持较好的性能。

有了解过 CopyOnWriteList 的实现原理吗？

CopyOnWriteList 是 Java 并发包中提供的一种线程安全的 List 实现，其核心原理是“写时复制”。

它内部维护着一个不可变的数组，当进行读操作时，直接访问这个数组，不需要加锁，因此读操作效率很高。

而当执行添加、修改、删除等写操作时，它不会直接修改原数组，而是先创建一个新的数组，将原数组的内容复制到新数组中，再在新数组上执行写操作，最后用新数组替换原数组。这个过程会通过加锁来保证线程安全，避免多个写操作同时修改导致的数据不一致。

这种机制使得读操作无需同步，适合读多写少的场景。但缺点也很明显，写操作时需要复制整个数组，内存开销较大，而且写操作完成前，其他线程读到的还是旧数据，存在数据一致性的延迟。

所以 CopyOnWriteList 适合对实时性要求不高，但需要频繁读取、偶尔修改的并发场景。

刚提交一个任务，发现核心线程数满了，工作队列也满了，但是没有达到最大线程数，这个时候会创建一个就绪队列来执行，那执行的是刚提交过来的任务还是工作队列的任务？
刚提交过来的任务。

为什么是执行刚提交过来的任务？

因为线程池的设计逻辑是当队列已满且可以创建新线程时，新任务会直接由新线程处理，而不是放入已满的队列中等待。这种机制可以避免新任务因队列满而被拒绝，同时保证了新任务能更快得到处理。

MQ

消息队列的优点缺点有哪些？

优点：

解耦。比如，用户下单后，订单系统需要通知库存系统，假如库存系统无法访问，则订单减库存将失败，从而导致订单操作失败。订单系统与库存系统耦合，这个时候如果使用消息队列，可以返回给用户成功，先把消息持久化，等库存系统恢复后，就可以正常消费减去库存了。

异步。将消息写入消息队列，非必要的业务逻辑以异步的方式运行，不影响主流程业务。

削峰。消费端慢慢的按照数据库能处理的并发量，从消息队列中慢慢拉取消息。在生产中，这个短暂的高峰期积压是允许的。比如秒杀活动，一般会因为流量过大，从而导致流量暴增，应用挂掉。这个时候加上消息队列，服务器接收到用户的请求后，首先写入消息队列，如果消息队列长度超过最大数量，则直接抛弃用户请求或跳转到错误页面。

缺点：

系统可用性降低。引入消息队列之后，如果消息队列挂了，可能会影响到业务系统的可用性。系统复杂性增加。加入了消息队列，要多考虑很多方面的问题，比如一致性问题、如何保证消息不被重复消费、如何保证消息可靠性传输等。

各个消息队列的对比？

万级的 RabbitMQ 的吞吐量要比十万级甚至是百万级的 RocketMQ 和 Kafka 低一个数量级。都可以实现高可用。RabbitMQ 是基于主从架构实现高可用性。RocketMQ 基于分布式架构。kafka 也是分布式的，一个数据多个副本，少数机器宕机，不会丢失数据，不会导致不可用。RabbitMQ 基于 erlang 开发，所以并发能力很强，性能极其好，延时很低，达到微秒级。其他两个都是 ms 级。

除了 Kafka，其他两个功能都较为完备，Kafka 主要支持简单的 MQ 功能。

RabbitMQ 消息丢失的可能性非常低，RocketMQ 和 Kafka 理论上不会丢失。

如何保证消息顺序性？

消息顺序性分为全局顺序性和分区顺序性，全局顺序要求所有消息严格按发送顺序处理，rocketmq 通过单队列 + 单生产者 + 单消费者实现，创建只有 1 个队列的 Topic，生产者和消费者均使用单实例。

而分区顺序性指同一组消息如同一用户的操作按顺序处理，不同组之间无需保证顺序。生产者建立路由规则，同一规则的消息发送到固定队列，每个队列由一个消费者线程处理，确保顺序。

如何保证 rocketmq 的高可用？

Rocketmq 有 nameserver 和 broker，nameserver 因为是无状态，且不相互通信的，所以只要集群部署就可以保证高可用，broker 的高可用是体现在读和写的高可用，通过集群和主从实现。

Broker 可以配置两种角色，主和从，主角色的 Broker 支持读和写，从角色的 Broker 只支持读，主会向从同步消息。

也就是说生产者只能向主角色的 Broker 写入消息，消费者可以从主和从角色的 Broker 读取消息。

消费者的配置文件中，并不需要设置是通过主角色读还是通过从角色读，当主角色不可用或者繁忙的时候，消费者的读请求会被自动切换到从角色。有了自动切换消费者这种机制，当一个主角色的机器出现故障后，消费者仍然可以通过从角色读取消息，这就实现了读的高可用。

写的高可用性则是在创建主题的时候，把主题的多个 Message Queue 创建在多个 Broker 组上，这样当一个 Broker 组的主角色不可用后，其他组主角色仍然可用，生产者仍然可以发送消息。

如何避免消息重复消费？

可以为每条消息分配全局唯一 ID，并创建一张 mysql 表记录消息的全局唯一 id 或在 redis 中缓存全局唯一 id，消费前检查该 id 是否已在 mysql 或 redis 中有记录，若有记录则不再处理该消息。

介绍一下 rocketmq 的死信队列和延迟队列？

Rocketmq 的**死信队列**用于存储**无法被正常消费且重试次数耗尽**的消息，避免消息**丢失**并支持后续**排查与处理**。

一般是由于消费者消费消息时**抛出异常**，且消息的**重试次数达到预设上限**时消息进入死信队列，或通过消费者**主动将消息标记为死信**的方式让消息进入死信队列。

可以通过**创建监听死信队列的消费者处理死信消息**，或通过 **rocketmq 的控制台删除死信消息**。

延迟队列则用于实现**消息发送后延迟指定时间再被消费**的场景，RocketMQ 通过**延迟级别机制原生支持延迟队列**。

RocketMQ 而是预设了**多个延迟级别**，对应不同的延迟时长，延迟消息发送时，会先被存储在一个**内部延迟主题**中，**Broker** 根据延迟级别对应的时间，**到期后将消息转发到目标主题**，此时消费者可正常消费。

如何实现 rocketmq 的流量削峰、异步解耦？

流量削峰是**接口流量过高时**生产消息，**将部分请求的传参传到消息队列去**，当流量降下来后再用这些**请求传参去调用接口**，异步解耦则是**将原先完整的接口工作拆出一部分交给消息队列去处理**，一般是不需要返回结果的工作交给消息队列去处理。

Rocketmq 消息堆积了应该怎么办？

可以**临时扩容消费端**，新增消费实例，利用 RocketMQ **负载均衡机制**，让**多实例并行消费**堆积消息，

然后**暂停非核心生产**，如果堆积由**生产速度远超消费能力**导致，临时停止如**日志记录这样的非核心业务的消息发送**，优先保障**核心消息消费**，避免堆积进一步恶化。

以及**调整消费参数**，消费端开启**批量拉取**、**调大消费线程池**，在消费端处理能力范围内提升**单次消费效率**。

说说 rocketmq 的模型架构？

RocketMQ 的模型架构主要分为几个部分，

生产者，负责**发送消息**，支持**集群部署**。通过**负载均衡策略选择合适的 Broker 发送消息**，还支持**同步、异步、单向等多种发送方式**，以及**事务消息等高级特性**。

消费者，负责接收并处理消息，同样**支持集群和广播两种消费模式**。集群模式下，消息会被负载到某个消费者实例，广播模式下，每个消费者都会收到全量消息。**消费者还分为推送型和拉取型**。

Broker，核心**存储和转发节点**，负责**接收、存储消息**，并将消息**推送给消费者**。**Broker** 通常以**主从架构部署**，主节点负责处理读写请求，从节点同步主节点数据并提供容灾能力，保证消息可靠性。

NameServer，充当**路由注册中心**，**管理 Broker 节点的注册与发现**。生产者和消费者通过 **NameServer** 获取 **Broker** 的地址信息，然后直接与 **Broker** 交互。**NameServer** 是轻量级的，无状态，可集群部署，彼此独立不通信。

Topic 和 Queue，Topic 是消息的逻辑分类，相当于消息的主题，**每个 Topic 会被划分为多个 Queue**，**分布在不同 Broker 上**，通过队列实现**负载均衡和并行处理**。消息在 Queue 中按顺序存储，**消费者按队列维度消费**。

整体流程是生产者向 **NameServer** 获取 **Broker** 地址，将消息发送到对应 **Broker** 的 Queue 中；消费者同样通过 **NameServer** 找到 **Broker**，从 Queue 中获取消息并消费。这种架构通过

NameServer 的轻量路由、Broker 的主从备份、Queue 的并行处理，实现了高可用、高吞吐和低延迟的消息传递。

Topic 中的 Queue 是属于消费者还是生产者？

Topic 中的 Queue 既不属于生产者，也不属于消费者，而是属于 Broker 服务器的资源。Queue 是 RocketMQ 中消息存储和传输的基本单元，本质上是 Broker 上的一个物理存储结构。当生产者发送消息到某个 Topic 时，消息会被分发到该 Topic 在不同 Broker 上的 Queue 中；而消费者则是从 Broker 上的这些 Queue 中拉取或接收消息进行消费。

Queue 的创建和管理由 Broker 负责，数量在 Topic 创建时指定或动态调整，分布在不同的 Broker 节点上。

生产者和消费者只是通过 Queue 进行消息的发送和接收，并不拥有 Queue 的所有权。比如，一个 Queue 可以被多个生产者发送消息，也可以被多个消费者分配消费，但 Queue 本身始终属于存储它的 Broker。

这种设计让 Queue 成为连接生产者和消费者的中间载体，通过 Broker 对 Queue 的集中管理，实现了消息的可靠存储和高效分发。

一个 queue 可以被多个消费者消费么？

一个 Queue 能否被多个消费者消费，取决于 RocketMQ 的消费模式，在集群消费模式下，一个 Queue 只会被同一个消费组内的一个消费者实例消费。这是因为集群模式下，消费组内的消费者会通过负载均衡机制分配 Queue，每个 Queue 对应一个消费者，避免同一条消息被重复处理，保证消费的唯一性。

而在广播消费模式下，同一个消费组内的所有消费者实例都会消费同一个 Queue 中的消息。此时每个消费者都会收到 Queue 中的全量消息，适合需要多实例同时处理同一条消息的场景。

假如说 rocketMQ 消息丢失了，怎么处理？

RocketMQ 消息丢失通常发生在生产端、Broker 端或消费端，需要分场景针对性处理。

生产端丢失时，可通过开启消息确认机制，确保消息成功写入 broker；同时使用事务消息或本地消息表，结合业务逻辑保证消息生产的原子性，避免业务执行成功但消息未发出的情况。

Broker 端丢失主要与存储机制有关，可配置刷盘策略为同步刷盘，确保消息写入磁盘后再返回确认，而非仅存在内存中。同时开启主从同步机制并设置为同步复制，保证主节点消息成功同步到从节点，避免主节点宕机导致消息丢失。

消费端丢失则需确保消息被正确处理后再提交 offset，可通过关闭自动提交，改为手动提交机制，在业务逻辑执行完成且确认无误后，再提交消费进度，防止消费过程中发生异常导致消息未处理但已标记为消费完成的情况。

此外，还可通过监控工具实时跟踪消息流转状态，定期校验消息数量，结合 RocketMQ 的重试机制和死信队列，对丢失或处理失败的消息进行补偿处理，最大程度减少消息丢失带来的影响。

RocketMQ 可以保证消息可靠送达，你了解它是怎么做到的吗？

RocketMQ 实现消息可靠送达，核心是通过全链路的状态把控与异常补偿机制，从消息发送、存储到消费的每个环节，都设计了针对性方案来避免消息丢失。

消息发送环节中，生产者发送消息时，RocketMQ 并非一发即忘，而是通过**确认机制**确保消息**能到达 Broker**，默认情况下，生产者会**等待 Broker 回消息已成功存储的确认信号**，只有收到这个 ACK，才认为发送成功；如果遇到网络波动、Broker 临时故障等情况导致发送超时或失败，生产者还会**触发重试机制根据配置的重试次数自动重新发送**，**同时通过退避重试策略避免频繁重试给 Broker 带来压力**。另外，对于需要更高可靠性的场景，还能开启**事务消息模式**，通过**预发送+本地事务确认+提交/回滚的三步流程**，确保**业务逻辑与消息发送的原子性**，避免业务执行成功但消息没发出去的情况。

然后消息存储环节，这是可靠送达的核心保障。Broker 收到消息后，并不会直接写入磁盘文件，而是先写入**内存缓冲区**，再通过**刷盘机制持久化到磁盘**：默认采用**异步刷盘**，即消息**先存缓冲区，满足一定条件后批量刷盘**。如果业务对可靠性要求极高，可以配置为**同步刷盘**，只有消息真正写入磁盘并返回成功，**才会给生产者发送 ACK**，彻底避免 Broker 宕机导致内存中消息丢失的问题。同时，RocketMQ 还**支持主从复制**，主 Broker 会实时将消息同步到从 Broker，即使主 Broker 故障，从 Broker 也能快速切换为主节点提供服务，进一步保障消息存储的可靠性。

最后是消息消费环节，重点是避免**消息已消费但没处理完或消费成功但 Broker 没收到确认导致的重复消费或丢失**。消费者收到消息后，**并不会立即让 Broker 删除消息**，而是先执行**本地业务逻辑**。只有当业务逻辑执行成功，才会主动向 Broker 发送消费确认。Broker 收到 ACK 后，才会将该消息标记为已消费，并从消费队列中移除。如果消费者在执行业务逻辑时宕机、网络中断，或者业务执行失败，Broker 会因为没收到 ACK，在**重试间隔后将消息重新投递给消费者**，确保消息能被正确处理。不过这里需要注意，由于重试机制的存在，业务端需要做**幂等处理**，避免同一条消息被重复处理导致业务异常。

了解消费者组的概念吗？

RocketMQ 的消费者组是由**多个消费者实例组成的集合**，**这些实例通常部署在不同的进程或机器上**，共同消费同一类消息。

消费者组的核心作用是实现消息的**负载均衡**和**高可用**。在负载均衡方面，当多个消费者实例属于同一个消费者组时，**Broker 会将订阅的消息队列均匀分配给组内的消费者**，避免单个消费者处理压力过大。而在高可用方面，若组内某个消费者实例出现故障，其负责的消息队列会被自动分配给组内其他健康的消费者，保证消息消费不中断。

需要注意的是，同一个消费者组内的所有消费者必须订阅相同的主题，**否则会导致负载均衡异常**。此外，消费者组还支持**广播消费和集群消费两种模式**，集群消费下消息只会被组内一个消费者处理，广播消费则会被组内所有消费者处理，可根据业务场景选择合适的模式。

单个 Broker 挂掉，比如运行 Broker 的机器故障，消息会不会丢失？

单个 Broker 挂掉时消息是否丢失，核心取决于 Broker 的**部署模式和消息存储配置**，如果 Broker 采用的是单机模式，那么当机器故障导致 Broker 挂掉时，存在消息丢失的可能，具体取决于消息**是否已完成持久化**。如果消息刚写入内存缓存、还没来得及刷盘，此时 Broker

突然故障，这部分未刷盘的消息就会丢失；但如果消息已经完成刷盘，即使 Broker 挂掉，待机器恢复后，Broker 重启时会从磁盘的 CommitLog 中恢复消息，这部分消息不会丢失。不过单机模式下，Broker 故障期间无法对外提供服务，且一旦磁盘损坏，已刷盘的消息也会永久丢失，所以生产环境几乎不会用这种模式。

而生产环境更常用的是主从模式，这种模式下能极大降低消息丢失风险，核心依赖主从同步”和刷盘机制的配合，Master 节点负责接收生产者发送的消息，接收后会先完成本地刷盘，之后再将消息同步给 Slave 节点。当 Master 节点因机器故障挂掉时：如果消息已在 Master 完成刷盘并同步到 Slave，那么即使 Master 故障，Slave 可以切换为新的 Master 继续提供服务，消息不会丢失。但如果消息仅在 Master 完成刷盘、还没来得及同步到 Slave 时 Master 就挂了，且此时 Master 的磁盘因故障损坏，这部分未同步的消息就会丢失，不过这种情况概率较低，且可通过“同步双写”配置进一步降低风险，即 Master 和 Slave 都刷盘成功后才返回生产者成功，但会牺牲部分性能。

另外，从生产者角度看，也有机制能辅助避免消息丢失：比如生产者开启消息发送确认机制，只有收到 Broker 返回的发送成功确认后，才认为消息发送完成。如果 Broker 挂掉导致发送失败，生产者可以通过重试机制重新发送消息。

Partition 概念，你了解过吗？它和 Broker 之间是什么关系？

RocketMQ 中的 Partition 是消息存储和并行处理的基本单元，它本质上是一个有序的消息队列，用于在物理层面拆分 Topic 中的消息，实现消息的分片存储和并行处理。

Partition 与 Broker 的关系密切且相互关联。

每个 Partition 必然属于某个 Broker 节点，一个 Broker 可以管理多个 Partition，这些 Partition 可以是不同 Topic 的，也可以是同一 Topic 的不同分片。

当 Topic 创建时，会根据配置的分区数在集群的 Broker 节点间分配 Partition，通常会尽量均衡分布以实现负载均衡。

Partition 是消息存储的实际载体，消息发送时会被路由到 Topic 的某个 Partition 中，由所在的 Broker 负责持久化存储和管理。

消费端通过订阅 Partition 来获取消息，多个消费者可以并行消费同一 Topic 的不同 Partition，提高消费吞吐量，而每个 Partition 只能被同一消费组中的一个消费者独占消费，保证消息有序性。

备份的 Broker 和主 Broker 里的 Partition 是一样的吗？

数量、标识、数据内容都是一样的，确保了主节点故障时，备份节点能无缝接管服务，只是主 broker 能读能写，备份 broker 只能读不能写。

ElasticSearch

为什么用 es?

因为它基于倒排索引结构，能提供高效、精准的全文搜索，支持复杂的查询语法和多种搜索策略，可快速从海量数据中检索出相关信息。

具备良好的分布式架构，能轻松扩展到多个节点，以处理大规模数据和高并发的搜索请求。随着业务增长，数据量和用户访问量不断增加，ES 可以通过添加节点来提高系统的性能和容量，保证服务的稳定性和响应速度。

支持多种数据格式和类型，包括结构化数据、半结构化数据和非结构化数据，还允许用户根据具体需求自定义索引结构和映射关系，能适应不同的业务场景和数据特点。

提供了丰富的数据分析功能，如聚合操作，可对数据进行分组、统计和计算，还能与 Kibana 等可视化工具集成，方便用户直观地展示和分析数据，帮助用户更好地理解数据和发现问题。

es 为什么高效（快）？

采用了倒排索引结构，快速定位数据。采用分布式架构，可以将数据分布在多个节点上，通过增加节点来实现水平扩展。这种分布式架构能够处理大规模的数据，并且随着数据量的增加，性能不会受到明显的影响。

在分布式环境中，es 会自动进行负载均衡，将查询请求均匀地分配到各个节点上，避免单个节点出现过载的情况。这样可以充分利用集群中各个节点的资源，提高整体的查询处理能力和响应速度。

同时 es 使用内存缓存来存储经常访问的数据和查询结果。当一个查询被执行时，它会首先检查缓存中是否有对应的结果，如果有，则直接从缓存中返回，避免了再次执行查询操作，从而大大提高了查询效率。

还会对索引数据进行缓存，将经常访问的索引块存储在内存中，这样可以减少磁盘 I/O 操作，提高索引的读取速度，进而加快查询的执行。

以及支持对数据、索引数据和文档进行压缩。压缩后的数据可以大大减少存储空间占用，同时也降低了磁盘 I/O 的压力，提高了数据的读写速度。

且 es 在执行查询之前，会解析查询语句的语法，将其转换为内部的查询执行计划，并根据索引的结构和数据分布情况，选择最优的查询执行策略。

讲讲 es 架构（讲讲 es 集群架构）？

有主节点，负责管理集群的元数据，如索引的创建、删除，节点的加入、离开等操作。主节点通过选举产生，一个集群中只有一个主节点处于活跃状态，其他主节点候选则处于待命状态。

有数据节点，用于存储实际的数据，并执行数据的索引和搜索操作。数据节点可以根据集群的规模和数据量进行扩展，以提高集群的存储和处理能力。

有协调节点，负责接收客户端的请求，并将请求分发到相应的数据节点上执行。协调节点还会对数据节点返回的结果进行合并和排序，然后将最终结果返回给客户端。

有索引，具有相同结构和映射关系的文档集合，类似于关系型数据库中的表。在 ES 集群中，索引可以分布在多个节点上，以实现数据的分布式存储和并行处理。

有分片，索引可以被划分为多个分片，每个分片是一个独立的 Lucene 索引。分片可以分布在不同的节点上，从而实现数据的分布式存储和负载均衡。ES 集群会自动管理分片的分布和复制，确保数据的高可用性和容错性。

当客户端向 ES 集群写入数据时，数据首先会被发送到协调节点。协调节点根据文档的路由规则，将数据分发到对应的主分片所在的数据节点上。

主分片接收数据后，会将数据写入本地的 Lucene 索引，并将数据复制到对应的副本分片上。副本分片写入成功后，主分片会向协调节点返回写入成功的响应，协调节点再将响应返回给客户端。

需要读取数据时，客户端发送读取请求到协调节点，协调节点根据请求的内容，将请求分发到包含相关数据的分片所在的数据节点上。

数据节点执行查询操作，并将结果返回给协调节点。协调节点对各个数据节点返回的结果进行合并和排序，然后将最终结果返回给客户端。

es 的深分页是什么？

在 ES 中，分页是查询数据时常用的功能，比如用 from 和 size 参数控制返回从第 N 条开始、取 M 条数据。但当 from 值很大时，就会进入深分页场景。这背后的核心问题，和 ES 的分布式架构密切相关，ES 的索引数据是分片存储的，每个分片都是一个独立的 Lucene 索引，查询时需要协调多个分片共同工作

当发起一次分页查询时，ES 的协调节点会先把查询请求分发到索引的所有分片上。对于浅分页比如 from=10, size=10, 每个分片只需要查询出前 20 条数据，再把这 20 条数据返回给协调节点。协调节点汇总所有分片的结果后，按排序规则重新排序，最终取前 20 条里的后 10 条，就是想要的结果，这个过程开销很小。

但深分页不一样。比如要查 from=10000, size=10, 协调节点会要求每个分片都查询出前 10010 条数据，因为每个分片都不知道其他分片的数据排序情况，必须把可能进入最终结果池的所有数据都返回。假设索引有 5 个分片，那协调节点就要接收 50050 条数据，之后再对这 5 万多条数据做全局排序，最后截取从第 10000 条开始的 10 条。

这里的问题就很明显了。一方面，分片要查询并返回大量数据，会占用更多的内存和网络带宽；另一方面，协调节点要处理海量数据的排序，内存消耗会急剧增加，甚至可能触发内存溢出。而且随着 from 值越来越大，这种性能损耗会呈指数级上升，所以 ES 默认会限制 from+size 不超过 10000，就是为了避免深分页带来的性能风险。

因此如果需要处理海量数据分页的场景，一般不用 from+size 的深分页，而用滚动查询，滚动查询会在第一次查询时生成一个快照，后续分页都基于这个快照读取数据，避免重复排序。

讲讲 es 的倒排索引？

传统的索引结构是“文档 -> 词”的映射关系，即根据文档找到其中包含的词。而倒排索引则是“词 -> 文档”的映射，它记录了每个词在哪些文档中出现过，以及在文档中的位置信息。

当文档被添加到 ES 中时，首先会对文档内容进行分词处理，将文本拆分成一个个独立的词。

对于每个词，ES 会记录它出现的文档列表，以及在每个文档中的位置信息。

并将构建好的倒排索引存储在磁盘上，以便后续的查询操作。为了提高查询效率，ES 还会对索引进行压缩和优化。

倒排索引能够快速定位包含特定词的文档，无需遍历所有文档。当用户输入一个查询词时，ES 可以直接通过倒排索引找到包含该词的文档列表，大大提高了查询速度。

支持多种复杂的查询操作，如布尔查询、短语查询、模糊查询等。通过对多个倒排表进行逻辑组合，可以实现精准的搜索匹配。

在处理大规模数据时，倒排索引的优势更加明显。由于它只需要关注包含查询词的文档，而不是整个文档集合，因此可以在短时间内处理大量的数据。

JVM

说说 JVM 内存模型？

有程序计数器，一块较小的内存空间，可以看作是当前线程所执行的字节码的行号指示器。在 JVM 的概念模型里，字节码解释器工作时就是通过改变这个计数器的值来选取下一条需要执行的字节码指令，分支、循环、跳转、异常处理、线程恢复等基础功能都需要依赖这个计数器来完成，是线程私有的。

有虚拟机栈，也是线程私有的，它的生命周期与线程相同。虚拟机栈描述的是 Java 方法执行的内存模型，每个方法在运行的同时都会创建一个栈帧用于存储局部变量表、操

作数栈、动态链接、方法出口等信息。每一个方法从调用直至执行完成的过程，就对应着一个栈帧在虚拟机栈中入栈到出栈的过程。

有本地方法栈，与虚拟机栈所发挥的作用是非常相似的，它们之间的区别不过是虚拟机栈为虚拟机执行 Java 方法服务，而本地方法栈则为虚拟机使用到的本地方法服务。

有堆，被所有线程共享的一块内存区域，在虚拟机启动时创建。此内存区域的唯一目的就是存放对象实例，几乎所有的对象实例都在这里分配内存。这也是 Java 垃圾回收器管理的主要区域，因此也被称为“GC 堆”。

有方法区，与 Java 堆一样，是各个线程共享的内存区域，它用于存储已被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码等数据。

Jvm 中数组存放位置在哪，字符串在哪？（这个来不及可以先不看，问的频率很低）

数组的引用，其存储位置和普通局部变量一致，如果是方法内定义的局部数组变量，引用会存在虚拟机栈的当前栈帧中。如果是类的成员变量，引用则会随对象实例或类信息，分别存在堆或方法区和元空间中。数组的实际元素，无论数组是基本类型数组还是引用类型数组，这些元素的存储本体一定在堆中。

字符串的存放位置，需要结合 String 的创建方式和 JVM 的字符串常量池来分析

在 JDK 1.7 及之后，字符串常量池从方法区迁移到了堆中，而 JDK 1.6 及之前，它是存放在方法区的。

当用字面量 abc 创建字符串时，JVM 会先去字符串常量池里检查是否已存在这个字符串，如果存在，直接让引用 s 指向常量池中的这个对象。如果不存在，会先在常量池中创建 abc 对象，再让 s 指向它。此时，字符串对象的本体是在堆中的常量池或方法区的常量池中。

当用 new String 创建字符串时，这个过程会产生两个对象，首先，abc 作为字面量，会触发上述常量池的检查逻辑。其次，new 关键字会在堆的普通区域中再创建一个内容为 abc 的字符串对象，最终引用 s 指向的是堆普通区域的这个对象。也就是说，此时存在两个 abc 对象：一个在常量池，一个在堆的普通区域。

gc 是什么？gc 是怎样的执行过程？

GC 是垃圾回收，GC 的主要作用就是自动检测并回收不再使用的对象所占用的内存，让程序员无需手动管理内存的分配和释放，从而降低了内存管理的复杂度和出错的可能性。

执行过程是先确定哪些对象是存活的，哪些是可回收的，常用可达性分析算法，从 gc roots 对象开始，递归遍历所有引用的对象，被引用的对象标记为存活，再通过垃圾回收器回收垃圾对象占用的内存空间，常用算法有标记清除、标记整理、复制算法等。最后进行内存压缩，整理内存空间，消除碎片，提高内存分配效率。

什么是 gc roots、标记清除、标记整理、复制算法？

Gc roots 包括静态变量、栈帧中的本地变量、JNI 引用等。

（本地变量表中引用的对象、方法区中静态属性引用的对象、正在运行的线程）

标记清除算法是直接删除标记为垃圾的对象，可能产生内存碎片。

标记整理算法是先标记垃圾，再将存活对象移动到连续内存空间，避免碎片。

复制算法是将内存分为两半，存活对象复制到另一半，清空原区域。

说说常见的垃圾回收器？

串行垃圾回收器，单线程，在进行垃圾回收时，会暂停所有的用户线程。

并行垃圾回收器，多线程，关注系统的吞吐量，在进行垃圾回收时也会暂停用户线程，但通过多线程并行回收可以提高回收效率。

CMS 垃圾回收器，以获取最短回收停顿时间为目标，在进行垃圾回收时，尽量减少用户线程的停顿时间。8

G1 垃圾回收器，一种面向服务器的垃圾回收器，能够同时兼顾吞吐量和低延迟。它将堆内存划分为多个大小相等的区域，根据每个区域中垃圾的数量和回收价值进行优先回收。9

Zgc 垃圾回收器，进行垃圾回收时暂停所有的用户线程时间极短，不超过 10ms，内存不分代，视为一个整体，使用着色指针和读屏障进行垃圾回收。11

什么是 jvm 的分代？

将内存分为多个代，针对不同代的对象特性采用不同的 GC 策略。大部分垃圾回收器都会采用内存分代方案。

有新生代，分为 eden 区和两个 survivor 区，新对象优先分配在 eden 区，eden 区满时触发 gc，存活对象移至 survivor，新生代 gc 频繁但 gc 速度快。

有老年代，存放新生代中多次 gc 后仍存活的对象，默认 15 次，空间不足时触发 gc，gc 频率较低但速度慢。

串行、并行、cms、g1 垃圾回收器对新生代都采用复制算法清除，对于老年代除了 cms 是采用标记清除算法，串行、并行、g1 垃圾回收器都是采用标记整理算法。

以及 jdk8 前的永久代和 jdk8 后的元空间，存储类元数据、静态变量等，空间不足时触发 gc。

串行、并行、cms 垃圾回收器对永久代采用标记整理算法，串行、并行、cms、g1 垃圾回收器对元空间均采用标记-清除算法。

你知道哪些 jvm 调优策略？

合理设置堆大小，堆大小设置过小会导致频繁的垃圾回收，甚至引发内存溢出错误，设置过大则可能会增加垃圾回收的停顿时间。可以分别设置堆的初始大小和最大大小，通常将这两个值设置为相同，避免堆在运行过程中频繁扩容和缩容。

调整新生代和老年代比例，对于创建对象频繁的应用程序，可以适当增大新生代的比例，减少对象在新生代和老年代之间的移动，从而减少垃圾回收的次数。

设置永久代或元空间大小，如果应用程序加载的类较多，可能需要适当增大元空间的大小。

以及根据应用场景选择合适的垃圾回收器和调整垃圾回收器相关参数。

还可以分析垃圾回收日志，了解垃圾回收的频率、停顿时间、回收对象的大小等信息，从而判断垃圾回收器的性能和内存使用情况。根据分析结果，调整堆大小、垃圾回收器等参数。

类加载机制是怎样的？

类加载主要包括加载、连接和初始化三个阶段。先是加载，类加载器根据类的全限定名查找对应的字节码文件，将找到的字节码文件读取到内存中，根据字节码内容在内存中创建一个 `java.lang.Class` 对象，该对象是类在 JVM 中的唯一表示。

再是连接，先验证确保加载的字节码文件符合 JVM 的规范，不会对 JVM 造成安全威胁。验证内容包括文件格式验证、元数据验证、字节码验证和符号引用验证等。

然后为类的静态变量分配内存，并设置默认初始值。最后解析，将类中的符号引用转换为直接引用。

最后初始化，对执行类的静态代码块和静态变量进行显式赋值操作。静态代码块和静态变量的初始化顺序按照它们在类中定义的顺序执行。

双亲委派机制是怎样的？

当一个类加载器接收到类加载请求时，它会先检查该类是否已经被加载过。如果未被加载，它会将请求委托给父类加载器。比如，应用程序类加载器收到请求后，会先将请求委托给扩展类加载器，扩展类加载器再将请求委托给启动类加载器。

启动类加载器会首先尝试加载该类，如果它能找到并加载该类，则完成类加载过程。如果启动类加载器无法加载，它会将请求回传给扩展类加载器，让扩展类加载器尝试加载。

若扩展类加载器也无法加载，再回传给应用程序类加载器进行加载。如果应用程序类加载器也无法加载，则会抛出异常。

通过双亲委派机制，所有的类加载请求最终都会先由启动类加载器尝试加载，这可以防止用户自定义的类覆盖核心的 Java 类，具有较高的安全性。

同时无论一个类在程序的何处被加载，因为类加载器在加载类之前会先检查该类是否已经被加载过，只有未被加载的类才会被加载，只要它的全限定名相同，都只会被加载一次。这样可以避免同一个类被多次加载，从而保证了类的唯一性和一致性。

如何打破双亲委派机制？

可以自定义类加载器，继承 `ClassLoader` 类，重写 `loadClass` 方法，因为在默认的 `loadClass` 方法实现中，遵循了双亲委派机制。要打破该机制，就需要修改这个方法的逻辑，不先委派给父类加载器，而是直接尝试自己加载类。

什么是 full gc?

FullGC 是 jvm 中针对老年代进行的垃圾回收操作，会回收整个堆内存中不再使用的对象，包括年轻代和老年代。

FullGC 的触发通常与老年代空间不足有关，比如当新对象晋升到老年代后导致空间不足，或永久代、元空间满了，也可能由 `System` 类的 `gc` 方法手动触发。

FullGC 的执行成本较高，因为需要扫描整个堆内存，且过程中通常会暂停所有用户线程，这会导致应用程序出现卡顿。尤其是在堆内存较大的情况下，FullGC 的耗时可能更长，对系统响应时间影响显著。

如果是你负责的服务器频繁的去发生 Full GC 的话，这个时候你要怎么处理？

首先，我会先通过监控工具查看 Full GC 的触发频率、持续时间以及内存变化情况，比如使用 JVM 自带的 `jstat`、`jmap` 等工具，或者可视化工具如 `VisualVM`，确定 Full GC 的具体原因。

如果是因为堆内存设置不合理，比如新生代或老年代空间过小，导致对象频繁进入老年代触发 Full GC，我会适当调整 JVM 参数，比如增大 `-Xms`、`-Xmx` 设置，或调整新生代与老年代的比例，让对象能在新生代被回收，减少进入老年代的频率。

如果是存在内存泄漏，导致老年代内存被不断占用且无法回收，我会通过 dump 内存快照，分析是否有大量对象长期存活且未被释放，比如静态集合未清理、资源未关闭等，找到内存泄漏点并修复代码。

另外，还可能是垃圾回收算法不合理，导致短时间内创建大量大对象直接进入老年代，这时需要优化对象创建逻辑，比如复用对象、使用对象池，或调整大对象的阈值参数。

同时，也会考虑垃圾回收器的选择是否合适，比如对于大内存应用，可能需要将 Serial GC 换成 Parallel GC 或 G1 GC，通过调整垃圾回收器参数提高回收效率，减少 Full GC 的影响。

总之，处理的核心是先定位问题根源，再针对性地从配置优化、代码修复或回收策略调整等方面解决，最终降低 Full GC 频率，保证服务器稳定运行。

什么对象算大的？

通常将占用内存空间较大、包含大量数据或复杂结构的对象视为大对象，存储了大量元素的集合如包含数万条数据的 `List`、`Map`，或者包含大字段的对象如长字符串、字节数组，还有由多层嵌套对象组成的复杂数据结构。

Spring、SpringBoot、SpringCloud

讲讲 Spring 中 bean 的生命周期？

Spring 中 Bean 的生命周期是指从 Bean 被创建到销毁的整个过程，

Spring 容器通过构造函数或者工厂方法等方式创建 Bean 的实例。如果 Bean 有多个构造函数，Spring 会根据依赖关系选择合适的构造函数进行实例化。

Bean 实例创建完成后，Spring 会按照配置将 Bean 的属性值注入到相应的属性中。这可以通过注解或者 XML 配置来实现。

在 Bean 属性赋值完成后，会执行 BeanPostProcessor 的方法，在 Bean 初始化之前对其进行一些额外的处理，例如修改 Bean 的属性值、检查 Bean 的状态等。

然后 Spring 会调用 Bean 的初始化方法。可以通过注解或者实现 InitializingBean 接口来定义初始化方法。在初始化方法中，可以进行一些资源的初始化、数据的加载等操作。

Bean 初始化完后，会执行 BeanPostProcessor 的方法，对其进行进一步的处理，例如代理 Bean、添加额外的功能等。

经过前面的步骤后，就可以在应用程序中被使用了。此时，Bean 已经处于可用状态，可以被其他 Bean 依赖和调用。

当 Spring 容器关闭时，会先调用 DestructionAwareBeanPostProcessor 的方法，在 Bean 销毁之前进行一些清理工作，如释放资源、关闭连接等。

最后，Spring 会调用 Bean 的销毁方法。可以通过注解或者实现 DisposableBean 接口来定义销毁方法。在销毁方法中，应该释放 Bean 占用的所有资源，以避免资源泄漏。

说说 springboot 自动装配原理？

Spring Boot 的自动装配原理本质上是通过“约定优于配置”的理念，让框架自动识别应用所需的配置并完成初始化，从而避免开发者手动编写大量样板代码。其核心机制围绕着启动类上的 @SpringBootApplication 注解展开，这个注解整合了 @EnableAutoConfiguration、@ComponentScan 等功能，其中 @EnableAutoConfiguration 是触发自动装配的关键——它会借助 AutoConfigurationImportSelector 类去扫描 META-INF/spring.factories 文件，该文件中记录了所有可能的自动配置类，比如数据库连接、Web MVC 框架等相关配置。

这些配置类并非全部生效，而是通过 @Conditional 系列注解进行条件筛选。

@ConditionalOnMissingBean 会确保容器中没有用户自定义的同类 Bean 时才使用默认配置，@ConditionalOnProperty 则根据配置文件中的属性值决定是否启用配置。这种条件化加载机制使得框架能根据应用的依赖和配置动态调整加载的配置项，避免无效配置的加载。

当符合条件的配置类被加载后，它们会在 Spring 容器中注册所需的 Bean。比如 WebMvcAutoConfiguration 会注册处理 HTTP 请求的相关组件，RedisAutoConfiguration 会配置 Redis 连接工厂等。这些配置类还会通过 @ConfigurationProperties 绑定配置文件中的属性，让用户可以通过 application.yml 或 application.properties 轻松修改默认行为，比如调整端口号、数据库连接参数等。

AOP 如何绑定到 IoC 上面的？

AOP 与 IoC 的绑定主要通过容器的整合实现，核心是利用 IoC 容器对 Bean 的生命周期管理能力来植入 AOP 的增强逻辑。

当 IoC 容器初始化 Bean 时，会通过 AOP 的核心组件进行匹配判断，如果某个 Bean 需要被增强，容器就不会直接返回原始 Bean 实例，而是动态生成一个代理对象。

这个代理对象会被纳入 IoC 容器的管理，替代原始 Bean 存在。当开发者从容器中获取该 Bean 并调用其方法时，实际执行的是代理对象的逻辑，先触发切面中定义的增强操作，再执行原始 Bean 的业务方法，从而实现了在不修改业务代码的情况下对功能的增强。

这种绑定方式让 AOP 能够依托 IoC 容器的依赖注入能力，自动完成切面与目标 Bean 的关联，既保持了业务逻辑的纯净性，又通过容器统一管理实现了横切逻辑的复用，是 Spring 等框架中 AOP 与 IoC 协同工作的核心机制。

为什么用 AOP，直接用一些工具类实现不行吗？

工具类确实能实现代码复用，比如把日志、事务等通用逻辑封装成静态方法，在需要的地方主动调用。但这种方式会导致业务代码和通用逻辑紧耦合，每个需要日志的方法里都要显式调用工具类，不仅冗余，而且如果后续通用逻辑修改，得逐个修改所有调用点，维护成本很高。

而 AOP 采用的是横向切入的思想，能在不修改业务代码的前提下，通过预定义的切入点，自动将通用逻辑织入到目标方法中。这样一来，业务代码只需要关注核心逻辑，通用功能的开启、关闭或修改都在一个地方统一管理，实现了关注点分离。

这种方式在处理权限校验、事务控制、异常处理等跨模块的通用需求时，优势尤其明显，能大幅提升代码的整洁性和可维护性。

Jdk 动态代理和 cglib 代理的区别？

代理机制不同，JDK 动态代理基于接口，必须实现至少一个接口，CGLIB 代理基于继承，通过继承目标类生成子类代理，性能不同，JDK 动态代理创建快，直接通过反射调用，但调用慢，每次调用需通过反射，性能更低。CGLIB 代理创建慢，需生成子类字节码并加载，但调用快，直接继承父类方法，无需反射，性能较高。

以及代理机制决定了 cglib 可代理普通类的方法，但无法处理 final 类和方法。

有了解反射吗？

反射允许程序在运行时获取类的信息如类名、方法、字段、构造器等，并动态操作类或对象的成员，而无需在编译期就知道具体的类信息。反射无需通过 new 关键字，可以直接通过类名在运行时创建对象实例。也可以调用任意类的任何方法，无需在编译期确定调用的具体类和方法，可以读取或修改对象的任何字段值，不受访问权限限制。

一般用于框架开发、动态代理、序列化和反序列化。优点是灵活性较高，缺点是性能开销大，且破坏了封装性。

用过 spring 的注解吗？

用过常见的几个注解，@Component，注册 bean 到 spring 容器，@controller 和@service，注册 bean 到 spring 容器的同时标识是接口层和服务层，@Resource，注入依赖给当前的类使用。

事务具体怎么执行的？

事务的执行从启动开始，系统会为其分配唯一标识，并记录执行上下文，明确后续所有操作都归属于该事务。

事务启动后，所有业务操作并非直接修改磁盘上的物理数据，而是先通过日志机制记录操作逻辑，再更新内存中的缓冲数据，这一步是保证事务原子性和持久性的核心。

如果多个事务同时操作同一批数据，可能出现脏读、不可重复读、幻读等问题。因此事务执行过程中，系统会通过锁机制和 MVCC 实现隔离性，确保多个事务互不干扰。

事务内所有操作执行完毕后，会通过 COMMIT 或 ROLLBACK 决定事务的最终状态，保证原子性。

最后事务提交或回滚，进入资源释放阶段：系统会回收事务占用的锁、清空事务上下文、标记 Undo Log 为可清理状态。至此，整个事务的执行流程完全结束。

Spring 中关于数据库事务的传播方式有哪些？

必须使用事务

如果当前存在事务，则加入该事务；如果当前没有事务，则创建一个新的事务。这是 Spring 默认的事务传播方式。

支持事务

如果当前存在事务，则加入该事务；如果当前没有事务，则以非事务的方式执行。

强制使用事务

如果当前存在事务，则加入该事务；如果当前没有事务，则抛出异常。

新建事务

无论当前是否存在事务，都会创建一个新的事务。如果当前存在事务，则将当前事务挂起，直到新事务执行完毕。

不支持事务

以非事务的方式执行操作，如果当前存在事务，则将当前事务挂起。

绝不使用事务

以非事务的方式执行，如果当前存在事务，则抛出异常。

嵌套事务

如果当前存在事务，则在嵌套事务中执行；如果当前没有事务，则创建一个新事务，与 PROPAGATION_REQUIRED 类似。嵌套事务可以有多个保存点，当内部事务回滚时，只会回滚到最近的保存点，而不会影响外部事务。

讲一下 Spring 的 ioc、aop、di?

IoC 是一种设计思想，它将对象的创建和对对象之间的依赖关系的管理从应用程序代码本身转移到了外部 spring 容器中。在传统的编程方式中，对象的创建和依赖关系的维护是由开发者在代码中直接进行的，这使得代码的耦合度较高，难以维护和扩展。而在 IoC 模式下，对象的创建和依赖关系的注入由容器负责，应用程序只需要从容器中获取所需的对象，而不需要关心对象是如何创建和依赖关系是如何解决的。

通过 IoC，实现了对象之间的解耦，提高了代码的可维护性和可扩展性。使得组件的依赖关系更加清晰，便于代码的复用和单元测试。

DI 是 IoC 的一种实现方式。它是指在运行时，由容器将依赖关系注入到组件中，从而使组件能够正常工作。依赖注入有三种常见的方式，分别是构造函数注入、Setter 方法注入和接口注入。

构造函数注入可以确保依赖关系在对象创建时就被正确初始化，适用于对象的依赖关系在创建后不会改变的情况。Setter 方法注入则更加灵活，允许在对象创建后动态地设置依赖关系，适用于需要在运行时根据不同条件来设置依赖的场景。

AOP 是一种编程范式，它允许开发者将横切关注点从业务逻辑中分离出来，形成独立的切面，然后在运行时将这些切面织入到目标对象的方法执行过程中。

其中切面是一个模块化的横切关注点，它包含了一组相关的通知和切入点。例如，一个用于日志记录的切面可能包含了在方法执行前记录方法调用信息、在方法执行后记录方法返回结果等通知。

通知定义了切入点处要执行的具体逻辑，包括前置通知、后置通知、环绕通知、异常通知和最终通知等类型。

切入点定义在哪些方法或代码块上应用切面的规则。可以通过方法签名、注解等方式来定义切入点。

使用 AOP 可以有效地减少代码的重复，提高代码的可维护性和可扩展性。将横切关注点与业务逻辑分离，使得业务逻辑更加清晰和简洁，同时也方便对横切关注点进行统一的管理和维护。

Spring Bean 的作用域有哪些？

单例，Spring Bean 的默认作用域。在整个 Spring 容器中，无论在何处引用该 Bean，都只会存在一个共享的实例。

原型，每次请求获取 Bean 时，Spring 容器都会创建一个新的实例。

请求，在一次 HTTP 请求中，Spring 容器会为该请求创建一个 Bean 实例，且仅在当前请求范围内有效。当请求处理完成后，该 Bean 实例会被销毁。

会话，在一个 HTTP Session 范围内，Spring 容器会创建一个 Bean 实例，该实例在整个 Session 期间有效。当 Session 过期或被销毁时，对应的 Bean 实例也会被销毁。

应用，在整个 Web 应用程序范围内共享一个 Bean 实例，类似于 ServletContext 级别。它在应用程序启动时创建，在应用程序关闭时销毁。

讲讲 session、cookie、token？

Cookie 是由服务器发送到用户浏览器并存储在本地的一小段文本数据。它通常包含键值对，用于在用户访问网站时记录用户的相关信息。

当用户访问网站时，服务器可以通过设置 Set - Cookie 响应头来向浏览器发送 Cookie。浏览器会将 Cookie 存储在本地，之后每次向同一服务器发送请求时，都会在请求头中包含相应的 Cookie 信息，以便服务器识别用户身份和获取相关用户数据。浏览器会自动在后续请求中携带 Cookie，无需用户手动操作。

同时由于 Cookie 存储在客户端，容易被用户篡改或窃取，因此不适合存储敏感信息。

Session 是服务器端用于跟踪用户会话的一种机制。它通过在服务器内存或数据库中存储用户相关信息，为每个用户创建一个唯一的会话标识。

用户首次访问网站时，服务器会为其创建一个 Session，并生成一个唯一的 Session ID。服务器将 Session ID 通过 Cookie 发送给浏览器，浏览器在后续请求中会携带该 Session ID。服务器根据 Session ID 来查找对应的 Session 对象，从而获取用户的相关信息，实现对用户状态的跟踪。

数据存储在服务器端，相对安全，不会被客户端直接篡改。

能够有效地跟踪用户在整个会话期间的操作和状态变化。

但随着并发用户数的增加，Session 会占用服务器的内存资源，因此需要合理设置 Session 的过期时间，以释放不再使用的 Session 资源。

Token 是一种用于身份验证和授权的令牌，通常是一个加密的字符串。它包含了用户的相关信息，如用户 ID、权限等，用于在不同的系统或服务之间进行身份验证和数据传输。

用户在登录时，向服务器发送包含用户名和密码等凭证的请求。服务器验证用户身份后，生成一个 Token，并将其返回给客户端。客户端在后续请求中，将 Token 放在请求头或其他

指定位置发送给服务器。服务器接收到请求后，通过验证 Token 的有效性来确定用户的身份和权限，从而决定是否允许用户访问相应的资源。

Token 中包含了用户的所有必要信息，服务器无需在内存中存储用户的会话状态，减轻了服务器的负担，便于实现分布式部署和扩展。

且 Token 通常是经过加密和签名处理的，难以被篡改和伪造。同时，可以设置 Token 的有效期，一旦过期，需要重新获取 Token，提高了系统的安全性。

以及可以根据不同的需求，在 Token 中包含不同的信息，如用户角色、权限范围等，方便进行灵活的授权和访问控制。

讲讲 jwt 加密的算法？

有对称加密算法，使用同一个密钥进行签名和验证，密钥需在服务端和验证端之间安全共享。适用于服务端内部的 token 验证，不适合分布式系统。

和非对称加密算法，使用私钥签名、公钥验证，私钥仅由签发方保存，公钥可公开分发，安全性更高。适用于分布式系统、跨服务 / 跨平台的 token 验证。

以及无加密算法，不使用任何加密算法，仅对 JWT 进行 Base64 编码，无签名验证。这种算法任何人都可以篡改 payload，仅用于调试场景。

Jwt 是如何保证安全的？

服务端生成 jwt 时，用对称加密的密钥或非对称加密的私钥生成签名，客户端请求携带 jwt 时，服务端用相同密钥或公钥验证签名，若签名不一致，说明 Token 被篡改，直接拒绝。

说说你对循环依赖的认识？

循环依赖是两个或多个类相互引用，形成依赖关系的闭环。

会导致对象创建和初始化困难，在存在循环依赖的情况下，对象的创建和初始化过程可能会变得复杂且容易出错。因为每个对象都依赖于其他对象的先初始化，可能导致初始化顺序难以确定，甚至出现死锁情况，即两个或多个对象相互等待对方初始化完成，从而导致程序无法继续执行。

以及循环依赖会使代码结构变得复杂，难以理解各个组件之间的关系。当需要对代码进行修改、扩展或调试时，由于依赖关系的复杂性，可能会引发一系列意想不到的问题，增加了维护成本。

可以通过分析依赖关系，对代码进行重构，打破循环依赖。可以将一些公共的功能提取出来，形成独立的模块或类，让相互依赖的组件通过这个公共模块进行交互，从而避免直接的循环依赖。

或者使用依赖注入容器，帮助管理对象的创建和依赖关系。它可以按照一定的规则和顺序来初始化对象，并解决对象之间的依赖问题。例如，在 Spring 框架中，通过使用依赖注入，可以有效地解决类之间的循环依赖问题。

讲一下微服务中组件的作用，以及有哪些组件？

服务发现组件，允许微服务在运行时动态地发现彼此的位置和状态。它使得服务之间能够相互通信，而无需事先知道对方的具体网络地址。当一个微服务启动时，它会向服务发现组件注册自己的信息，包括**服务名称**、**IP 地址**、**端口号**等。其他微服务可以通过查询服务发现组件来获取所需服务的连接信息，从而实现服务间的调用。

配置中心组件，用于集中管理微服务的配置信息。它允许开发人员在不重新部署微服务的情况下，动态地修改配置参数。配置中心提供了一个统一的配置管理平台，使得配置的修改和分发更加便捷和高效，同时也保证了配置的一致性和安全性。

网关组件，作为微服务架构的入口点，它接收所有来自外部的请求，并根据请求的路径、参数等信息将请求路由到相应的微服务。网关还可以实现一些额外的功能，如**请求过滤**、**身份验证**、**授权**、**限流**、**缓存**等，从而为微服务提供统一的安全防护和性能优化。

熔断器组件，用于**监控微服务之间的调用关系**，当某个微服务出现故障或响应时间过长时，熔断器会自动切断对该服务的调用，以防止**故障的扩散和级联效应**。当故障恢复后，熔断器会自动恢复对该服务的调用。熔断器能够提高微服务架构的**稳定性和可靠性**，避免因单个服务的故障导致整个系统的崩溃。

监控组件，用于**实时收集微服务的运行状态信息**，如 CPU 使用率、内存使用率、请求响应时间、吞吐量等，以便运维人员及时了解系统的运行情况，发现潜在的问题并进行性能优化。

了解什么是分布式事务吗？

分布式事务是指**对数据库的多个操作在多个服务中执行时**确保这些**分散操作**要么全部成功，要么全部失败，最终保持数据的一致性，也就是事务在分布式环境中的延伸。

会涉及到 **CAP** 理论，分布式系统无法同时满足三个特性，最多只能满足其中两个，**一致性**，所有节点在同一时间看到的数据完全相同（如 A 转账后，所有查询 A 账户的请求都能看到最新余额）

可用性，只要不是所有节点故障，系统就能正常响应请求（如部分数据库节点故障，仍能处理下单、查询请求）

分区容错性，网络分区出故障（如机房断网、节点间通信中断发生）时，系统仍能继续运行。由于分布式系统必须满足分区容错性，因此实际设计中**只能在一致性和可用性之间权衡**，要么优先保证强一致性，牺牲部分可用性，要么优先保证高可用性，接受短期数据不一致，后续修复。

以及 **BASE** 理论，对 CAP 理论的延伸，承认**强一致性无法完全实现**，转而追求**最终一致性**，即短期数据可能不一致，但经过一段时间后，所有节点数据会自动同步为一致状态，核心包含三点，**基本可用**，系统核心功能可用（如电商秒杀时，非核心功能如“历史订单查询”可降级，但“下单、支付”必须可用）

软状态，允许数据存在中间状态（如“订单已创建 - 待支付”“库存扣减中”，且中间状态不影响系统可用性）

最终一致性，经过一定时间（如秒级、分钟级），所有节点数据会自动达成一致（如支付成功后，订单状态从“待支付”更新为“已支付”，库存最终扣减完成）。

BASE 理论是目前绝大多数分布式事务方案的设计基础（如支付宝、微信支付的跨系统对账），因为它更符合互联网系统“高可用优先”的需求。

有了了解三个常用方案，方案一是**两阶段提交**，第一阶段是准备阶段，协调者向所有参与者发送“准备请求”，**询问是否可以执行事务**（如“能否新增订单？能否扣减库存？”）。

参与者执行本地事务但不提交，记录事务日志。参与者向协调者返回同意或拒绝。

此时所有参与者都处于事务已执行但未提交的中间状态，若任何一个参与者拒绝，整个事务后续会回滚。

第二阶段是提交回滚阶段，若所有参与者均返回同意，协调者发送提交请求，参与者执行本地事务提交，之后返回提交成功。

若有任一参与者返回拒绝协调者发送回滚请求，参与者执行本地事务回滚，之后返回回滚成功。

方案二是补偿事务，先是尝试阶段，检查资源、预留资源而不执行实际业务，然后确认阶段，确认执行业务，最后取消阶段，释放尝试阶段预留的资源，若尝试失败，直接执行取消。若尝试成功但确认失败，需重试确认或人工干预，直到成功，保证最终一致性。

方案三，分段补偿方案，针对“长事务”（如跨多个微服务的复杂流程，如“用户注册→开通会员→发送优惠券→通知短信”），将分布式事务拆分为多个“本地事务步骤”，每个步骤对应一个“补偿步骤”，

先按顺序执行每个本地事务步骤（如 Step1: 注册用户→Step2: 开通会员→Step3: 发优惠券）；

若某个步骤失败（如 Step3 发优惠券失败），则从失败步骤的前一步开始，反向执行补偿步骤（如先补偿 Step2: 关闭会员→再补偿 Step1: 删除用户），直到所有步骤回滚完成。实现方式有编排式，由一个编排器统一管理步骤顺序和补偿逻辑（如通过流程引擎控制），耦合度低，适合复杂流程

和协同式，每个服务自己知道下一步要调用哪个服务，以及失败时如何补偿（如 Step1 执行完后主动调用 Step2，Step2 失败后主动通知 Step1 补偿），耦合度高，适合简单流程。

备注：括号里的不背，只是辅助理解

微服务里面，比如说有 ab 两个系统，a 系统去给 b 系统发请求的过程中，rpc 是怎么处理的？

首先，A 系统的应用层会通过本地调用的方式发起请求，调用的其实是 RPC 框架生成的代理对象。这个代理对象会负责将调用的方法名、参数等信息进行序列化，转换成可在网络传输的字节流。

接着，序列化后的请求会通过 RPC 框架的网络传输层，经过协议封装后发送到 B 系统所在的服务器。这里的协议通常包含了通信协议和应用层协议。

当 B 系统的网络层接收到请求后，会将数据转发给对应的代理对象，由它负责反序列化，解析出方法名和参数，然后真正调用 B 系统中对应的业务方法。

业务方法执行完成后，B 系统会将返回结果按照同样的流程：序列化→网络传输→A 系统反序列化，最终由 A 系统的代理对象将结果返回给应用层，完成一次 RPC 调用。

整个过程中，RPC 框架还会处理服务发现、负载均衡、超时控制、重试等问题，让开发人员无需关注网络通信细节，像调用本地方法一样调用远程服务。

Spring 事务的失效情况？

常见的情况有以下几种。

首先，方法不是 public 的。Spring 事务默认通过 AOP 实现，而 AOP 对非 public 方法的增强可能失效，导致事务注解无法生效。

其次，事务方法内部调用。如果在同一个类中，一个没有事务的方法调用了有事务的方法，由于 Spring AOP 是基于代理的，这种内部调用不会触发代理对象的事务增强，导致事务失效。

第三，异常被捕获且未重新抛出。事务回滚通常依赖未被捕获的异常，如果在方法中捕获了

异常却没有抛出，Spring 无法感知异常，也就不会执行回滚。

第四，抛出的异常类型不匹配。默认情况下，Spring 事务只对 RuntimeException 和 Error 类型的异常回滚，如果抛出的是受检异常，且没有通过 rollbackFor 属性指定，事务不会回滚。

第五，数据源未配置事务管理器。如果没有为数据源配置对应的事务管理器，Spring 无法获取事务管理器，事务自然无法生效。

第六，事务传播行为设置不当。例如，将传播行为设置为不支持事务，可能导致事务无法按预期工作。

第七，数据库本身不支持事务。比如使用了 MyISAM 存储引擎，即使配置了 Spring 事务，也无法实现事务控制。

操作系统

进程和线程的区别是什么？

进程是资源分配的基本单位，而线程是 CPU 调度的基本单位。进程有独立的地址空间，一个进程崩溃不会影响其他进程。线程共享进程的地址空间，一个线程崩溃可能导致整个进程崩溃。

进程的创建、切换和销毁开销较大，线程的开销相对较小。

进程和线程分别有什么通信方式？

进程的通信方式有管道、命名管道、消息队列、共享内存、信号量、套接字，线程的通信方式则有共享内存、条件变量、信号量、互斥锁。

进程间通信方式的特点，以及哪种方式更快？

管道是半双工通信，只能在父子进程或兄弟进程间使用，数据单向流动，容量有限，适用于简单的字节流传递。

命名管道，突破了管道的亲缘关系限制，允许任意进程通信，仍为半双工，数据存在于内存中，用完即消失，适合无亲缘关系的进程间简单通信。

信号，用于处理异步事件，携带信息少，不能传递大量数据，主要用于通知进程发生特定事件。

消息队列，按消息类型组织的链表，存于内核中，可双向通信，允许不同进程按类型读取，有容量限制，适合需要分类传递数据的场景。

共享内存，多个进程直接共享同一块物理内存，无需数据拷贝，是所有 IPC 中速度最快的。但需要配合信号量等机制实现同步互斥，避免并发冲突。

信号量，本身不传递数据，用于进程间的同步与互斥，通过控制临界资源的访问权限，保证多进程有序协作。

套接字，可用于同一主机或不同主机的进程通信，支持 TCP/UDP 协议，灵活性高，但开销较大，适用于网络通信场景。

共享内存是最快的，因为它跳过了数据在用户态和内核态之间的拷贝过程，进程直接操作共享的内存区域。

了解过虚拟内存吗？有什么用？

虚拟内存是操作系统为解决物理内存不足而引入的一种内存管理技术，核心是通过软硬件配合，让程序认为自己拥有连续的可用内存，而实际物理内存可能是分散的，甚至部分数据存储在磁盘上。

主要作用有扩展内存容量，当物理内存不足时，系统会把暂时不用的数据从物理内存转移到磁盘的交换区，释放出的物理内存供其他程序使用，让程序可以运行在比实际物理内存更大的地址空间中。以及隔离进程地址空间，每个进程都有独立的虚拟地址空间，彼此不可见，避免了进程间的内存干扰，提高了系统安全性和稳定性。

和内存管理，对程序来说，虚拟地址是连续的，无需关心实际物理内存的碎片化问题；对操作系统来说，可更灵活地管理物理内存分配。

还有提高内存利用率，通过按需加载和交换机制，让物理内存得到更高效的利用，同时支持更多程序并发运行。

操作系统为进程分配内存的过程？

首先，进程会发起内存申请，此时操作系统第一步会先做合法性校验，比如检查申请的内存大小是否超过进程能使用的用户空间上限，或者是否超出系统给该进程分配的内存配额，不合法的话会直接返回错误，合法则进入下一步。

接着是选择内存分配策略，现在主流操作系统都用“离散分配”，因为早期的“连续分配”容易产生外部碎片，比如空闲块分散，凑不出大进程需要的连续空间。离散分配会把物理内存切成固定大小的页框，给进程分配不连续的页框，再用页表记录映射关系，这样就不会有外部碎片问题，只存在可忽略的内部碎片。

然后内存分配，第一步会把进程申请的内存大小按页框对齐，之后操作系统会用专门的算法比如伙伴系统找空闲页框，找到后就把这些页框标记为已占用，同时更新空闲页框链表、进程页表这些核心数据结构。

最后，当进程退出或主动释放内存，操作系统会把之前分配的页框标记为空闲，放回空闲链表，同时销毁进程的页表，把内存回收给系统，供其他进程使用。

进程空间的大小是由什么决定的？

主要由硬件架构限制和进程自身需求共同决定。

硬件层面是根本限制，核心是 CPU 的寻址范围，比如 32 位 CPU 寻址范围远小于 64 位，因此 64 位系统中进程空间远超 32 位。

然后进程会根据自身的需求动态调整，操作系统会根据进程的实际内存申请，动态分配虚拟空间中的可用区域。

了解过 cpu 的缓存吗？为什么要做 cpu 缓存？有几级缓存？速度如何？

Cpu 缓存是 CPU 与内存之间的高速数据缓冲层，核心作用是解决 CPU 运算速度与内存读写速度不匹配的问题。做缓存原因是现在主流 CPU 的运算速度远快于内存的读写速度，如果 CPU 每次运算都直接从内存取数据，会因为等待内存响应而频繁闲置，严重浪费 CPU 的运算能力。有三级缓存，一级到三级缓存速度逐级递减。

说说 linux 常用的命令？

ps -ef grep 查看某进程的端口号，如果该进程未运行则查看不到 cd 进入某层目录，如果输入..则是退到上一层目录，从根路径开始输入则可以跳转到任何地方，相对路径只能从当前路径往下跳转 kill -9 输入进程端口号杀死进程 ls 遍历某文件夹下的所有文件和文件夹

`Nohup java -jar nohup.out&` 启动 java 的 jar 包并保持后台运行，日志输入到默认文件中

`Tail -f` 查看某日志的实时更新 `rm -rf` 删除文件 `pwd` 得到某文件夹从根路径开始的文件路径

备注：`nohup.out&`可以不读后边那个`&`，如果面试官问是不是少了参数或者符号再讲后边还要加个 java 的按位与运算符。

什么是死锁？产生死锁的原因和必要条件是什么？

死锁是指两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法推进下去。

产生死锁的原因主要有资源竞争和进程推进顺序不当。产生死锁的必要条件有资源非共享一次只有一个进程可以使用、一个进程至少占有一个资源并等待另一个资源、资源不能被抢占、进程间互相循环等待。

怎么避免死锁？

系统动态监测可能导致死锁的情况，避免进入死锁状态，比如每次资源申请时判断分配后系统是否仍处于安全状态，如果资源分配会导致死锁则拒绝该请求，常用资源分片图算法和等待图算法来检测，以及如果检测到死锁可以通过中止进程、回滚进程状态、强行从其他进程抢占资源重新分配等方式解除死锁。

什么是 IO 多路复用？

IO 多路复用是一种高效的 IO 模型，允许单个线程同时监听多个文件描述符，并在某个文件描述符可读、可写或出现异常时得到通知。这样可以避免无效的等待，充分利用 CPU 资源。常用的有 `select`、`poll`、`epoll` 模式，其中 `epoll` 相比于 `poll` 和 `select`，能支持更大的并发连接数、性能更高。

什么是用户态、内核态？

用户态是指应用程序运行在非特权模式下的状态，此时应用程序只能访问被授权的资源，不能直接操作硬件资源和底层系统资源。在用户态下，应用程序只能执行一些受限的指令集，不能直接执行特权指令。

当应用程序需要访问系统资源时，需要通过系统调用的方式切换到内核态，请求操作系统提供服务。内核态和用户态之间的切换是通过操作系统内核提供的中断或异常机制实现的。

内核态是操作系统运行在特权模式下的状态，此时操作系统具有最高的权限，可以访问所有的硬件资源和底层系统资源，如处理器、内存、I/O 等。在内核态下，操作系统可以执行所有的指令，而且不受访问权限的限制。

常见的进程调度算法有哪些？

有先来先服务，非抢占式，按照请求的顺序进行调度。

最短作业优先，非抢占式，按估计运行时间最短的顺序进行调度。

最短剩余时间优先，最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。

时间片轮转调度，将所有就绪进程按 FCFS 的原则排成一个队列，每次调度时，把 CPU 时间分配给队首进程，该进程可以执行一个时间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 CPU 时间分配给队首的进程。

优先级调度，为每个进程分配一个优先级，按优先级进行调度。

什么是孤儿进程，什么是僵尸线程，如何处理僵尸线程？

1 孤儿进程指的就是一个进程的父进程已经退出或者终止了，但是这个子进程本身还在运行的情况，僵尸进程指的是一个进程已经退出，但他还存在于进程表。

僵尸进程仍占据着系统的某些资源如进程号，可以通过清除僵尸进程的父线程、重启僵尸进程所属的服务来清理僵尸线程。

什么是内存管理？

内存管理分为连续内存管理和非连续内存管理，其中连续内存管理为一个用户程序分配一个连续的内存空间，内存利用率一般不高，容易造成内存碎片

非连续内存管理允许一个程序使用的内存分布在离散的内存中，相对更加灵活一些。

有段式管理，将应用程序的虚拟地址空间划分为多个逻辑段，每个段是一段连续的物理内存，有页式管理，把物理内存分为连续等长的物理页，应用程序的虚拟地址空间也被划分为连续等长的虚拟页，有段页式管理，结合了段式管理和页式管理，把物理内存先分成若干段，每个段又继续分成若干大小相等的页。

计算机网络

说下 OSI 七层模型？

1. **物理层**：是 OSI 模型的最底层，主要负责处理物理介质上的信号传输，确保数据能够在。
2. **数据链路层**：主要功能是将物理层接收到的原始比特流转换为有意义的数数据帧，并进行错误检测和纠正
3. **网络层**：负责在不同的网络之间进行数据路由和转发，使数据能够从源节点到达目标节点，还负责处理网络拥塞、数据包分片和重组等问题。
4. **传输层**：主要为应用程序提供端到端的通信服务，确保数据在不同主机之间的可靠传输，常见的协议有 TCP 和 UDP。
5. **会话层**：用于建立、维护和管理会话连接，协调不同主机上应用程序之间的通信。它可以实现会话的同步、断点续传等功能，确保通信双方能够按照正确的顺序进行数据交换。
6. **表示层**：主要负责处理数据的表示和转换，包括数据加密和解密、压缩和解压缩、字符编码转换等。它确保不同系统之间能够正确地理解和处理数据，使应用层能够专注于数据的语义和逻辑处理。
7. **应用层**：是 OSI 模型的最高层，直接面向用户和应用程序，为用户提供各种网络服务，应用层的协议规定了应用程序如何与网络进行交互，以实现特定的功能。

讲讲 tcp、udp、http、https 等网络协议？

TCP 是一种面向连接的协议，在数据传输之前需要先建立连接，传输完成后再释放连接。通

通过三次握手建立连接，确保通信双方都做好了数据传输的准备；通过四次挥手来释放连接，保证数据传输的完整性。

具有高可靠性，它通过序列号、确认应答、重传机制等保证数据的准确传输。如果发送方发送的数据在规定时间内没有收到接收方的确认应答，就会自动重传数据，直到接收方正确接收为止。

能够进行流量控制，通过滑动窗口机制让发送方根据接收方的接收能力来调整发送数据的速度，防止接收方缓冲区溢出。还具备拥塞控制机制，当网络出现拥塞时，自动降低发送数据的速率，以避免网络进一步拥塞。

UDP 是一种无连接的协议，发送方不需要事先与接收方建立连接就可以直接发送数据。这种方式使得 UDP 的传输速度较快，但不保证数据一定能准确到达接收方，没有确认应答、重传等机制。数据发送出去后，发送方不知道数据是否被正确接收，也不会主动去处理数据丢失或错误的情况。

具有低延迟、高效率的特点，因为它不需要像 TCP 那样进行复杂的连接建立和维护操作，也没有流量控制和拥塞控制机制，所以在传输小数据量、对实时性要求较高的场景中表现出色。

HTTP 是一种应用层协议，主要用于在 Web 浏览器和 Web 服务器之间传输超文本数据，如 HTML 页面、图片、视频等。它规定了客户端和服务器之间数据交互的格式和规则。

采用请求 - 响应模式，客户端向服务器发送请求，服务器接收到请求后进行处理，并向客户端返回响应。请求和响应消息都包含报头字段和消息体，报头字段用于传递一些元信息，如请求方法、数据类型、缓存策略等，消息体则包含了实际传输的数据。

具有无状态性，即服务器不会在不同的请求之间记住客户端的状态信息。每个请求都是独立的，服务器只根据当前接收到的请求进行处理并返回响应，不依赖于之前的请求或交互历史。通常基于 TCP 协议来实现，默认使用 80 端口进行数据传输。在建立连接后，客户端和服务器通过 TCP 连接进行 HTTP 数据的传输。

HTTPS 实际上是 HTTP 协议与 SSL/TLS 协议的结合，在 HTTP 的基础上增加了加密和认证功能，确保数据在传输过程中的安全性和完整性。它通过 SSL/TLS 协议对数据进行加密，使得数据在网络传输过程中变成密文，只有合法的接收方才能解密和读取数据。

利用数字证书来进行身份认证，服务器会向客户端提供由权威证书颁发机构颁发的数字证书，客户端可以通过验证证书来确认服务器的身份是否可信。这可以防止中间人攻击，确保客户端连接到的真正的目标服务器，而不是被假冒的服务器。

默认使用 443 端口进行数据传输。由于增加了加密和解密等操作，HTTPS 的性能相对 HTTP 会稍低一些，但随着硬件性能的提升和加密算法的优化，这种性能差异在大多数情况下并不明显，而且其安全性优势远远超过了性能上的损失。

讲讲三次握手？

第一次握手：客户端向服务器发送一个 SYN 同步报文段，该报文段中包含客户端随机生成的初始序列号 x 。此时客户端进入同步已发送状态，表明客户端希望与服务器建立连接，并已准备好发送数据。

第二次握手：服务器接收到客户端的 SYN 报文段后，会向客户端发送一个 SYN+ACK 报文段。其中，SYN 表示同步，ACK 表示确认。服务器会为自己生成一个初始序列号 y ，并将客户端的序列号加 1 作为确认号 $x+1$ 。此时服务器进入同步已接收状态，表明服务器已收到客户端的连接请求，并同意建立连接。

第三次握手：客户端收到服务器的 SYN+ACK 报文段后，会向服务器发送一个 ACK 报文段。客户端将服务器的序列号加 1 作为确认号 $y+1$ ，自己的序列号则为 $x+1$ 。服务器收到这个

ACK 报文段后，连接建立成功，双方进入建立连接状态，此时客户端和服务端就可以开始进行数据传输了。

TCP 为什么 3 次握手，两次行不行？

TCP 三次握手是为了确保通信双方都能确认彼此的发送和接收能力，建立可靠的连接。两次握手不行，因为如果只有两次握手，服务器在第二次握手时发送的确认信息无法确定客户端是否能够成功接收，无法确认客户端接收能力。

比如，客户端发送的第一次握手请求丢失，客户端会认为服务器没有收到请求而重新发送请求。

但服务器在收到第二个请求后会认为客户端是第一次发送请求，于是发送确认信息并建立连接。

然而，如果这个确认信息在传输过程中丢失，服务器会认为连接已经建立并开始发送数据，而客户端由于没有收到确认信息，不会认为连接已建立，从而导致数据丢失，连接建立失败。且易造成资源浪费，比如客户端发送的第一次握手请求丢失，客户端会认为服务器没有收到请求而重新发送请求，以及服务器在收到客户端的连接请求后就会为该连接分配资源，如果客户端因为某些原因不再发送数据，服务器就会一直等待，造成资源浪费。

而三次握手可以避免这种情况，因为只有客户端收到服务器的确认信息并发送第三次握手的确认后，双方才真正建立连接并开始分配资源。

讲讲四次挥手？

第一次挥手：客户端向服务器发送一个 FIN 结束报文段，该报文段中包含客户端当前的序列号 x ，用于告知服务器自己不再发送数据，但仍可接收数据。此时客户端进入终止等待 1 状态，标志着客户端主动发起连接断开请求。

第二次挥手：服务器接收到客户端的 FIN 报文段后，会向客户端发送一个 ACK 确认报文段。服务器将客户端的序列号加 1 作为确认号 $x+1$ ，自己的序列号为 y 。此时服务器进入关闭等待状态，表明服务器已收到断开请求，正在处理剩余数据；客户端收到 ACK 后，进入终止等待 2 状态，等待服务器完成数据传输。

第三次挥手：当服务器处理完所有剩余数据后，向客户端发送 FIN 结束报文段，其中序列号为 $y+1$ ，确认号仍为 $x+1$ 。此时服务器进入最后确认状态，表明自己也不再发送数据，等待客户端的最终确认。

第四次挥手：客户端接收到服务器的 FIN 报文段后，向服务器发送 ACK 确认报文段。客户端将服务器的序列号加 1 作为确认号 $y+2$ ，自己的序列号为 $x+1$ 。发送后，客户端进入时间等待状态，服务器收到 ACK 后，直接进入关闭状态，连接正式终止。

Tcp 为什么四次挥手？

关闭连接时，客户端向服务端发送 FIN 报文，仅表示客户端不再发送数据，但还能接受数据而服务端收到客户端的 FIN 报文后，先回一个 ACK 报文，而服务端可能还有数据需要处理和发送，等到服务端处理完毕，再发送一个 FIN 报文给客户端

服务端通常需要等待数据完成处理和发送，所以服务端的 ACK 报文和 FIN 报文是分开来发送的，总计四次

http 状态码有哪些，代表的意义？

常见的有

2xx 成功状态码

200 OK: 请求成功，服务器已成功处理了请求并返回了请求的资源。

201 Created: 请求成功，并且服务器创建了新的资源。

202 Accepted: 服务器已接受请求，但尚未处理完成。该请求可能在后台进行处理。

204 No Content: 请求成功，但服务器没有返回任何内容。通常用于只需要执行服务器端操作而不需要返回数据的情况。

3xx 重定向状态码

301 Moved Permanently: 请求的资源已被永久移动到新的 URL。客户端应使用新的 URL 进行后续请求。

302 Found: 请求的资源临时位于不同的 URL。客户端应使用新的 URL 进行本次请求，但后续请求仍可使用原 URL。

303 See Other: 服务器建议客户端通过另一个 URL 获取资源，通常用于 POST 请求后，让客户端重定向到一个 GET 请求的 URL 以获取结果。

304 Not Modified: 客户端发送了附带条件的请求，且服务器认为资源未被修改，客户端可以使用缓存的资源。

4xx 客户端错误状态码

400 Bad Request: 客户端发送的请求有错误，服务器无法理解请求的语法。

401 Unauthorized: 客户端请求需要身份验证，而客户端未提供有效的身份凭证或凭证无效。

403 Forbidden: 服务器理解请求，但拒绝执行该请求。客户端可能没有足够的权限访问请求的资源。

404 Not Found: 服务器无法找到请求的资源。可能是 URL 输入错误或资源已被删除。

405 Method Not Allowed: 客户端请求的方法（如 GET、POST 等）不被服务器允许用于请求的资源。

5xx 服务器错误状态码

500 Internal Server Error: 服务器内部发生错误，无法完成请求。这是一个通用的错误代码，通常表示服务器端出现了未预料到的问题。

501 Not Implemented: 服务器不支持请求中所使用的功能或方法。

502 Bad Gateway: 服务器作为网关或代理，从上游服务器接收到了无效的响应。

503 Service Unavailable: 服务器暂时无法处理请求，通常是由于服务器过载或正在维护。

504 Gateway Time - out: 服务器作为网关或代理，在等待上游服务器响应时超时。

网页 URL 输入到浏览器上的流程？

浏览器首先检查自身的缓存，看是否有该 URL 对应的 IP 地址缓存。如果没有，就会向操作系统的 DNS 缓存查询。若操作系统的 DNS 缓存中也没有，浏览器会向本地配置的 DNS 服务器发送请求，进行域名解析，获取 URL 对应的 IP 地址。

获取 IP 后浏览器与目标服务器通过 TCP 协议进行连接，连接建立后，浏览器会根据输入的 URL 生成 HTTP 请求报文，包括请求方法、请求头字段以及请求体，并将其发送到服务器。服务器接收到 HTTP 请求后，根据请求的 URL 和其他信息进行处理，可能会涉及到查询数据库、执行脚本等操作，以生成响应内容。

服务器将生成的响应内容封装成 HTTP 响应报文，包括响应状态码、响应头字段以及响应体，并发送给浏览器。

然后浏览器接收并解析 HTTP 响应报文，提取出 HTML、CSS、JavaScript 等资源。

浏览器根据这些资源生成渲染树，再根据渲染树进行布局和绘制，将页面呈现给用户。

https 加密流程？

客户端向服务器发送一个 HTTPS 请求，请求中包含客户端支持的加密算法、密钥交换算法等信息。

服务器收到客户端请求后，根据客户端提供的信息，选择一种双方都支持的加密算法和密钥交换算法，并将服务器的数字证书发送给客户端。

客户端收到服务器的数字证书后，会验证证书的合法性，如果证书验证通过，客户端会从证书中提取出服务器的公钥。客户端使用服务器的公钥对一个随机生成的会话密钥进行加密，并将加密后的会话密钥发送给服务器。这个会话密钥将用于后续的加密通信。

而服务器使用自己的私钥对客户端发送的加密会话密钥进行解密，得到会话密钥。当通信结束后，客户端和服务器会关闭连接。在关闭连接之前，双方可以协商是否需要重新进行密钥交换，以便进行下一次的加密通信。

http 中 Get 和 post 请求的区别？

首先是请求目的和数据传递方式不同。GET 用于从服务器获取资源，数据通过 URL 参数传递，会暴露在地址栏中；而 POST 主要用于向服务器提交数据，数据放在请求体中，不会直接显示在 URL 里，相对更安全。

其次是数据长度限制不同。GET 受限于 URL 长度，不同浏览器有差异，通常几 KB，不适合传递大量数据；POST 则没有明确的长度限制，适合传输表单、文件等较大数据。

另外，缓存和幂等性方面也有区别。GET 请求是幂等的，会被浏览器主动缓存，可在历史

记录中查看；POST 不幂等，默认不会被缓存，历史记录中也不会保存具体数据。
还有安全性差异，GET 的参数明文可见，不适合传递敏感信息；POST 数据在请求体中，虽不是绝对安全，但相对更适合传输密码等敏感数据。

http1.0 和 http2.0 的区别？

HTTP 1.0 是早期的版本，采用明文传输，每次请求都需要建立新的 TCP 连接，完成后就断开，这种短连接模式会带来较多的连接建立和断开开销，且同一时间一个连接只能处理一个请求，容易出现队头阻塞问题，导致性能受限。

而 HTTP 2.0 做了很多优化。引入了二进制分帧机制，将数据分割成更小的帧进行传输，使得同一 TCP 连接可以并行处理多个请求和响应，解决了 HTTP 1.0 的队头阻塞问题。同时，HTTP 2.0 支持长连接复用，多个请求可以复用同一个 TCP 连接，减少了连接建立的开销。此外还提供了头部压缩功能，能有效减少头部数据的传输量，并且支持服务器推送，即服务器可以主动向客户端推送相关资源，提升加载效率。